

pyGANDALF - a Geometric, ANimation, Directed, Algorithmic, Learning Framework for Computer Graphics

John Petropoulos

Thesis submitted in partial fulfillment of the requirements for the
Masters' of Science degree in Computer Science and Engineering

University of Crete
School of Sciences and Engineering
Computer Science Department
Voutes University Campus, 700 13 Heraklion, Crete, Greece

Thesis Advisor: Professor *George Papagiannakis*

This work has been performed at the University of Crete, School of Sciences and Engineering, Computer Science Department.

The work has been supported by the Foundation for Research and Technology - Hellas (FORTH), Institute of Computer Science (ICS).

UNIVERSITY OF CRETE
COMPUTER SCIENCE DEPARTMENT

**pyGANDALF - a Geometric, ANimation, Directed, Algorithmic,
Learning Framework for Computer Graphics**

Thesis submitted by
John Petropoulos
in partial fulfillment of the requirements for the
Masters' of Science degree in Computer Science

THESIS APPROVAL

Author: _____
John Petropoulos

Committee approvals: _____
George Papagiannakis
Professor, Thesis Supervisor

Yannis Tzitzikas
Professor, Committee Member

Xenophon Zabulis
Principal Researcher, Committee Member

Departmental approval: _____
Polyvios Pratikakis
Associate Professor, Director of Graduate Studies

Heraklion, July 2024

pyGANDALF - a Geometric, ANimation, Directed, Algorithmic, Learning Framework for Computer Graphics

Abstract

In the realm of computer graphics (CG) education and development, the lack of versatile and modern tools presents a significant challenge, particularly in providing a comprehensive learning experience that encompasses both legacy and cutting-edge technologies. Traditional frameworks often rely on outdated APIs, such as OpenGL and Direct3D11 or below, which limit the exploration of advanced graphics techniques and hinder the educational process.

In this thesis, we introduce pyGANDALF, a powerful, lightweight, open-source CG framework that addresses these limitations through its pure Entity-Component-System (ECS) design. This framework seamlessly integrates an editor layer within its ECS architecture, streamlining the development process for both educators and developers. The framework natively supports two graphics APIs, OpenGL as well as WebGPU, a modern low-level graphics API, ensuring compatibility and versatility across various platforms. This feature bridges a critical gap in educational tools by enabling the use of state-of-the-art technologies such as compute and ray tracing pipelines, enabled by the WebGPU API, which are essential for a contemporary CG curriculum.

The framework’s Physically Based Rendering (PBR) capabilities produce stunning visuals, enhancing the quality of both practical applications and educational demonstrations. It also integrates the Universal Scene Description (USD) open-source software, supporting the depiction of robust and scalable 3D scenes composed of diverse assets with complex hierarchies. This fosters highly collaborative workflows and broadens the scope of potential applications.

To aid in learning and exploration, the framework includes a plethora of examples that demonstrate a wide range of graphics techniques, from fundamental to advanced concepts. These examples serve as valuable resources for researchers, and developers, facilitating the transition from educational settings to real-world applications. Furthermore, CG educators can enhance CG curricula by leveraging pyGANDALF to efficiently instill modern CG principles to students, a statement that is validated by the expert-based evaluation that was conducted.

pyGANDALF - Ένα Γεωμετρικό, Κατάλληλο για Σύνθετη Κίνηση, Κατευθυνόμενο, Αλγοριθμικό, Μαθησιακό Σύστημα για Γραφικά Υπολογιστών

Περίληψη

Στον τομέα της εκπαίδευσης και ανάπτυξης γραφικών υπολογιστών, η έλλειψη ευέλικτων και σύγχρονων εργαλείων παρουσιάζει μια σημαντική πρόκληση, ιδιαίτερα στην παροχή μιας ολοκληρωμένης μαθησιακής εμπειρίας που περιλαμβάνει τόσο τις παραδοσιακές όσο και τις πιο σύγχρονες τεχνολογίες. Τα παραδοσιακά συστήματα συχνά βασίζονται σε παρωχημένες διεπαφές προγραμματισμού εφαρμογών που περιορίζουν την εξερεύνηση προηγμένων τεχνικών στα γραφικά υπολογιστών και δημιουργούν εμπόδια στη εκμάθησή τους.

Σε αυτή τη διπλωματική εργασία, παρουσιάζουμε το pyGANDALF, ένα ισχυρό, ελαφρύ, ανοιχτού κώδικα σύστημα γραφικών υπολογιστών που αντιμετωπίζει αυτούς τους περιορισμούς μέσω της χρήσης αρχιτεκτονικής οντοτήτων-στοιχείων-συστημάτων (ΟΣΣ). Αυτό το σύστημα ενσωματώνει και ένα επίπεδο διεπαφής για επεξεργασία σκηνών εντός της αρχιτεκτονικής ΟΣΣ, διευκολύνοντας τη διαδικασία ανάπτυξης τόσο για τους εκπαιδευτικούς όσο και για τους προγραμματιστές. Το σύστημα υποστηρίζει εγγενώς δύο διαφορετικές διεπαφές προγραμματισμού γραφικών εφαρμογών, την OpenGL καθώς επίσης και την WebGPU, μιας σύγχρονη χαμηλού επιπέδου διεπαφή προγραμματισμού γραφικών εφαρμογών, κάτι το οποίο διασφαλίζει τη συμβατότητα και την ευελιξία σε διάφορες πλατφόρμες. Αυτό το χαρακτηριστικό γεφυρώνει ένα κρίσιμο κενό στα εκπαιδευτικά εργαλεία, επιτρέποντας τη χρήση προηγμένων τεχνολογιών οι οποίες υποστηρίζονται από την WebGPU και είναι απαραίτητες σε ένα σύγχρονο μάθημα γραφικών υπολογιστών.

Το σύστημα μπορεί να χρησιμοποιήσει τη τεχνική "Απόδοσης Σκηνής με βάση την Φυσική" για να δημιουργήσει αποτελέσματα με αυξημένο φωτορεαλισμό, βελτιώνοντας την ποιότητα τόσο των πρακτικών όσο και των εκπαιδευτικών εφαρμογών. Ενσωματώνει επίσης ένα λογισμικό ανοιχτού κώδικα για αναπαράσταση και περιγραφή σκηνών, το Universal Scene Description (USD), υποστηρίζοντας την απεικόνιση τρισδιάστατων σκηνών που αποτελούνται από διάφορα στοιχεία τα οποία μπορούν να έχουν περίπλοκες ιεραρχίες. Αυτό ενισχύει τις πιθανές συνεργασίες/συνδυασμούς με άλλα εργαλεία και διευρύνει το πεδίο των πιθανών εφαρμογών.

Για να υποστηρίξει τη μάθηση και την εξερεύνηση στον τομέα της γραφικής υπολογιστών, το σύστημα περιλαμβάνει μια πληθώρα παραδειγμάτων που επιδεικνύουν ένα ευρύ φάσμα τεχνικών στον τομέα αυτό, από θεμελιώδεις έννοιες έως προηγμένες τεχνικές. Αυτά τα παραδείγματα λειτουργούν ως πολύτιμοι πόροι για ερευνητές και προγραμματιστές, διευκολύνοντας τη μετάβαση από τα εκπαιδευτικά περιβάλλοντα στην υλοποίηση κανονικών εφαρμογών. Επιπλέον, οι εκπαιδευτικοί που διδάσκουν γραφικά υπολογιστών μπορούν να ενισχύσουν τα προγράμματα σπουδών τους αξιοποιώντας το pyGANDALF για να μεταδώσουν αποτελεσματικά στους φοιτητές τις σύγχρονες αρχές των γραφικών υπολογιστών, όπως υποδυκνείται από την αξιολόγηση απο ειδικούς που πραγματοποιήθηκε.

Acknowledgements

I want to sincerely thank my supervisor Prof. George Papagiannakis for his invaluable guidance, mentorship, and consistent support throughout this endeavor, which empowered my journey and impacted the direction and quality of this work.

I would like to extend my sincere appreciation to Dr. Manos Kamarianakis and Dr. Antonis Protopsaltis, whose expertise and insights have greatly enriched this work.

I am also indebted to my friends and colleagues Achilleas Filippidis, Dimitrios Angelis, Efstratios Geronikolakis, Eleftherios Toupis, Michael Dodis, Michael Tamiolakis and Stelios Georgiou for providing, with their knowledge and expertise, invaluable impact in the evaluation of this work.

Finally, I would like to thank my family for the unwavering support and encouragement that they showed me throughout this journey that helped me get through the difficulties that I encountered.

Explore, Experiment, Learn

Contents

1	Introduction	1
2	Related Work	9
2.1	Generic Frameworks	9
2.1.1	Three.js, Babylon.js	9
2.1.2	Unity	10
2.2	Learning Focused frameworks	11
2.2.1	The glGA Project	11
2.2.2	The ShaderLab Framework	12
2.2.3	The CodeRunnerGL Framework	13
2.2.4	Vulkan All The Way	14
2.2.5	The Elements Project	15
2.3	Beyond State of the Art	15
2.4	Our publications related to this work	16
3	The pyGANDALF Framework	19
3.1	Entity Component System	20
3.1.1	Design Philosophy	20
3.1.1.1	Performance Considerations	20
3.1.2	Implementation Details	21
3.1.2.1	ECS Structure Deep Dive	21
3.1.2.2	Systems Deep Dive	22
3.2	Editor layer	25
3.2.1	Design Philosophy	25
3.2.1.1	Dear ImGui Bundle Package Limitation	26
3.2.2	Implementation Details	27
3.2.2.1	Editor Panels	27
3.2.2.2	Seamless Scene Switching	27
3.2.2.3	Serialization	28
3.3	Renderer	28
3.3.1	Design Philosophy	28
3.3.2	Implementation Details	29
3.3.2.1	Renderer class	29

3.3.2.2	Renderering Systems	30
3.3.2.3	Handling Resources	31
3.4	Code Availability	33
4	Using pyGANDALF in CG Curriculum	35
4.1	Introductory Examples	35
4.2	Beginner Examples	37
4.2.1	Empty Scene	37
4.2.2	Triangle	38
4.2.3	Quad	39
4.2.4	Camera	41
4.2.5	Textured Quad	41
4.2.6	ECSS Cube	42
4.3	Intermediate Examples	43
4.3.1	Blinn-Phong Model	43
4.3.2	Blinn-Phong Textured Model	46
4.3.3	Cube Mapping	47
4.3.4	Environment Mapping	48
4.3.5	Basic Compute	51
4.3.6	User Defined System &Component	52
4.3.7	Multiple Scenes	52
4.3.8	The Scene Editor	53
4.4	Advanced Examples	54
4.4.1	Tessellation Shaders	54
4.4.2	Geometry Shaders	56
4.4.3	Normal Mapping	58
4.4.4	Parallax Mapping	62
4.4.5	Shadow Mapping	66
4.4.6	Physical Based Rendering - Sphere	68
4.4.7	Physical Based Rendering - Model	73
4.4.8	Advanced Compute	74
4.5	Summary	75
5	Evaluation	77
5.1	Performance evaluation	77
5.1.1	Experimental Setup	77
5.1.2	Results &Discussion	78
5.1.2.1	Frames Per Second	78
5.1.2.2	Memory Usage	79
5.1.3	Summary	80
5.2	Educational evaluation	80
5.2.1	Experimental Setup	80
5.2.2	Results &Discussion	82
5.2.2.1	Overall Results	82

5.2.2.2	Per Task Results	86
5.2.3	Summary	89
6	Conclusion	91
6.1	Summary	91
6.2	Comparison of Educational Frameworks	92
6.3	Limitations &Future Work	92
6.3.1	Renderer	92
6.3.2	Editor Layer	92
6.3.3	Educational Examples	93
	Bibliography	95
	Appendices	99
A	Core Framework Code	101
B	Core Renderer Code	111
C	Resource Handling Code	119

Listings

3.1	Entity class	21
3.2	Example Gravity Component	21
3.3	Gravity System example	21
3.4	Scene creation - entity registration - adding of components	22
3.5	System instantiation	23
3.6	Custom component serialization rule	28
3.7	Resource management in OpenGL	32
3.8	Resource management in WebGPU	32
4.1	GLSL Shader Code for for Unlit Geometry with Projections	38
4.2	WGSL Shader Code for Unlit Geometry with Projections	38
4.3	GLSL Shader Code with Support for Textures and Projections	39
4.4	WGSL Shader Code with Support for Textures and Projections	40
4.5	Blinn-Phong GLSL Shader Code	43
4.6	Blinn-Phong WGSL Shader Code	45
4.7	Skybox GLSL Shader Code	47
4.8	Skybox WGSL Shader Code	48
4.9	Skybox Reflection GLSL Shader Code	48
4.10	Skybox Refraction GLSL Shader Code	49
4.11	Skybox Reflection WGSL Shader Code	50
4.12	Skybox Refraction WGSL Shader Code	50
4.13	Compute WGSL Shader Code	51
4.14	Tessellation GLSL Shader Code	54
4.15	Geometry GLSL Shader Code	57
4.16	Normal Mapping GLSL Shader Code	58
4.17	Normal Mapping WGSL Shader Code	60
4.18	Parallax Mapping GLSL Shader Code	62
4.19	Parallax Mapping WGSL Shader Code	64
4.20	Shadow Mapping GLSL Shader Code	66
4.21	PBR GLSL Shader Code	69
4.22	PBR WGSL Shader Code	71
4.23	Advanced Compute WGSL Shader Code	74
A.1	Application Class	101
A.2	Base System Class	102
A.3	Scene Class	105

A.4	SceneManager Class	108
B.1	OpenGLRenderer Class	111
B.2	WebGPURenderer Class	114
C.1	OpenGLTextureLib Class	119
C.2	OpenGLShaderLib Class	122
C.3	OpenGLMaterialLib Class	125
C.4	WebGPUTextureLib Class	129
C.5	WebGPUShaderLib Class	131
C.6	WebGPUMaterialLib Class	134
C.7	MeshLib Class	140

List of Tables

5.1	Frames Per Second (Higher is Better).	79
5.2	A comparison of CPU Memory Usage between OpenGL, OpenGL + Editor and WebGPU (Lower is Better).	79
5.3	A comparison of GPU Memory Usage between OpenGL, OpenGL + Editor and WebGPU (Lower is Better).	80
6.1	Comparison of different educational frameworks and their provided features.	92

List of Figures

3.1	Entity Component System setup	22
3.2	Editor Layer	26
4.1	Code snippet from a jupyter notebook step by step example. . . .	36
4.2	An Empty Scene.	37
4.3	Rendering of a Triangle.	38
4.4	Rendering of a Quad.	40
4.5	Rendering of a Quad using a Perspective Camera.	41
4.6	Rendering of a Textured Quad.	42
4.7	A Cube is Constructed Using Six Quads, Each Representing a Face of the Cube.	43
4.8	Blinn-Phong Model.	44
4.9	Blinn-Phong Textured Model.	46
4.10	Rendering of a Skybox Using Cube Mapping.	47
4.11	Rendering of Meshes Simulating Reflection(Right) and Refraction(Left) by Utilizing Environment Mapping.	49
4.12	The Scene Editor.	53
4.13	Rendering of a Height Map with Dynamic Level of Detail Using Tessellation Shaders.	55
4.14	Example of an Exploding Mesh Using a Geometry Shader.	57
4.15	Showcase of Normal Mapping in a Brick Textured Wall.	58
4.16	Showcase of Different Parallax Mapping Techniques.	62
4.17	Rendering of a Scene Using Shadow Mapping. Also, in the Upper Left, the Depth Only Texture that was Produced During the First Depth Only Pre-Pass is Rendered.	66
4.18	Rendering of a Rusted Sphere Using Physical Based Rendering. . .	69
4.19	Rendering of a 3D Model Using Physical Based Rendering.	74
4.20	Applying Filters to Images Using Compute Shaders.	75
5.1	Overall Results for Group 1 that used pyGANDAL, showing the percentage of responses (y-axis) in the 1-5 range per question (x- axis).	83
5.2	Overall Results for Group 2 that used no framework, showing the percentage of responses (y-axis) in the 1-5 range per question (x-axis). .	84

5.3	Average Implementation Times for Both Groups.	84
5.4	Standard Deviation Results for Group 1 that used pyGANDALF. .	85
5.5	Standard Deviation Results for Group 2 that used pyGANDALF. .	85
5.6	Group 1: Per Task Average Response in the Range (1-5).	87
5.7	Group 1: Per Task Standard Deviation.	87
5.8	Group 2: Per Task Average Response in the Range (1-5).	88
5.9	Group 2: Per Task Standard Deviation.	88

Chapter 1

Introduction

Learning computer graphics presents several challenges for students, particularly in areas such as mathematics, transformations and projections, and logical problem solving. These challenges are discussed by Suselo et al. [1], who identify these areas as preparatory obstacles that students must overcome to grasp graphics programming effectively. A solid understanding of linear algebra, calculus, and geometry is essential for grasping computer graphics concepts. Many students struggle with these mathematical foundations, which can impede their ability to understand transformations, lighting calculations, and other core graphics principles. Furthermore, understanding how to manipulate objects in 3D space and project them onto a 2D screen is fundamental yet challenging. Concepts like matrix multiplication, coordinate systems, and perspective projections often confuse students. Graphics programming involves complex problem-solving skills, including debugging shader programs, optimizing rendering pipelines, and understanding hardware constraints. These tasks require a deep level of logical reasoning and systematic thinking, which can be difficult for students to develop.

A wide range of tools and frameworks already exist to aid students in overcoming the difficulties mentioned above. These tools aim to simplify the learning process, provide hands-on experience, and bridge the gap between theoretical concepts and practical application. However, based on the analysis that was conducted by Balreira et al. [2], it can be concluded that OpenGL was the most widely used graphics API in university education in 2017. This conclusion is drawn from the notable absence of any mention of other graphics APIs in their study. Its portability, low barrier to entry, and extensive documentation and tutorials [3] have made OpenGL an ideal tool for this purpose. OpenGL's design as a cross-platform API provided a reliable foundation for educators to introduce students to the principles of computer graphics, fostering a widespread understanding of rendering techniques and graphics pipeline operations.

However, in recent years, the emergence of new, more modern APIs, which are significantly closer to the hardware, has prompted many educators to reconsider their approach to teaching computer graphics. One major shift is driven by

the evolving demands of the industry and the need to equip students with skills that align with current and future technologies. Modern APIs like Vulkan, Metal, and WebGPU offer low-level access to the GPU, providing developers with more control over rendering processes and performance optimizations. This granularity enables the exploitation of advanced GPU features and the creation of more efficient graphics applications, which are crucial for industries such as gaming, virtual reality, and simulation.

It has become clear that the structure of the OpenGL API, essentially a sophisticated state machine, does not accommodate the latest and most exciting features supported by contemporary GPUs. These features include hardware-accelerated ray tracing, which enables real-time rendering of photorealistic scenes with complex lighting and shadows, and push constants, which allow for efficient parameter updates within shaders. Additionally, more optimized bindless designs facilitate faster resource management and rendering performance, essential for high-fidelity visual experiences.

Furthermore, the landscape of debugging and error reporting has vastly improved with modern APIs. OpenGL’s historically inadequate error reporting and debugging capabilities have become even more apparent in light of the substantial improvements offered by modern APIs. Tools such as Vulkan’s validation layers and Metal’s detailed error messages provide developers with critical insights into the rendering process, helping to identify and resolve issues more effectively. This enhancement is particularly beneficial in an educational context, where understanding and troubleshooting are integral to the learning process.

The shift towards modern graphics APIs also aligns with the broader trends in software development, where efficiency, performance, and scalability are paramount. By integrating these advanced tools into the curriculum, educators can provide students with a more comprehensive and relevant education in computer graphics, preparing them for the demands of contemporary development environments. This transition not only bridges the gap between legacy and current technologies but also ensures that the next generation of computer graphics professionals is well-versed in the latest advancements, ultimately contributing to the continuous evolution of the field.

Several modern low-level APIs have emerged, including Metal [4], Direct3D12 [5], Vulkan [6], and, more recently, WebGPU [7]. These APIs provide developers with low level access to GPU hardware, enabling more efficient and sophisticated graphics programming. Among these, only Vulkan and WebGPU possess the potential to replace OpenGL in graphics programming curricula, as they are the only cross-platform APIs, a crucial feature for an accessible teaching process. This cross-platform capability ensures that educational tools and projects can be developed and executed across a variety of operating systems and devices, promoting inclusivity and flexibility in computer graphics education.

In this thesis, we introduce the pyGANDALF framework, which aims to fill the gap in teaching-focused frameworks by utilizing a modern low-level API, employing an Entity-Component-System (ECS) architecture, and offering an editor layer for

quick iteration in creation and development. All of this is encapsulated within a Pythonic framework to maximize accessibility, ease of use, and adherence to good programming principles. Let us delve deeper into the rationale behind our choices when designing the framework.

Why WebGPU As previously mentioned, the primary choice for a cross-platform graphics API boils down to Vulkan and WebGPU. Vulkan, being a more mature and well-established API, is supported by the Khronos Group, which includes all major GPU manufacturers, operating system vendors, and various individual, academic, and industry members. This widespread support ensures that Vulkan is robust, extensively tested, and continuously updated to support the latest GPU features and performance enhancements.

However, Vulkan is notoriously verbose and requires extensive manual management of low-level concepts such as synchronization and resource handling. This complexity makes it cumbersome and time-consuming to begin with, as a simple triangle rendering program can require nearly 1000 lines of code. This steep learning curve can be a significant barrier for students and educators, particularly in an educational setting where the goal is to teach fundamental graphics concepts efficiently and effectively.

Choosing Vulkan would necessitate creating an abstraction layer to assist students and ease their learning process. This abstraction layer would simplify the Vulkan API, reducing the boilerplate code and complexity involved in setting up basic rendering operations. Ironically, this abstraction layer would likely resemble the WebGPU API, as WebGPU is designed to simplify the complexities of low-level graphics programming while still providing sufficient control over hardware operations.

The WebGPU API, specifically its native desktop variant, serves as a layer on top of low-level APIs like Direct3D12 or Vulkan on Windows, Vulkan on Linux, and Metal on MacOS. It reduces the verbosity and complexity of these low-level APIs just enough to make them easier and faster to code with, while still maintaining low-level abstractions that provide fine-grained control over the hardware. This balance makes WebGPU an appealing choice for educational purposes, as it allows students to focus on learning graphics programming concepts without getting bogged down by intricate low-level details.

Therefore, WebGPU emerged as the clear choice for our purposes. Its design strikes a balance between ease of use and the ability to access advanced graphics features, making it suitable for both beginners and more advanced students. By reducing the amount of boilerplate code and abstracting some of the more complex aspects of GPU programming, WebGPU enables students to achieve meaningful results more quickly, which is crucial for maintaining engagement and fostering a deeper understanding of the material.

The main downside is that WebGPU is still relatively young and not entirely stable, which can result in delays in supporting the latest features and occasional

API changes or deprecations. These issues can pose challenges, particularly in an educational setting where stability and consistency are important. However, the advantages of WebGPU’s design outweigh these drawbacks, especially given its rapid development and growing support within the community.

Moreover, WebGPU’s emphasis on being a web-first API extends its applicability beyond traditional desktop applications. As web technologies continue to advance, the ability to create high-performance graphics applications that run in the browser becomes increasingly valuable. This opens up new opportunities for students to learn and experiment with graphics programming in a web context, preparing them for a future where web-based applications play a significant role.

Many state-of-the-art examples using the WebGPU API have been created [8] [9] [10], showcasing the framework’s capabilities in a learning-focused environment. These examples demonstrate how WebGPU can be leveraged to produce complex and visually stunning graphics directly in the browser, providing an accessible platform for students and educators to explore and understand modern graphics programming. By engaging with these cutting-edge examples, learners can gain hands-on experience with advanced rendering techniques, shader programming, and performance optimization, all within the convenience of a web browser. This not only enriches their educational experience but also equips them with the skills needed to thrive in an increasingly web-centric technological landscape.

In conclusion, WebGPU’s balance between ease of use and maintaining sufficient control makes it the most suitable option for teaching modern graphics programming. Its cross-platform nature, simplified API design, and growing ecosystem support create an ideal environment for both learning and developing advanced graphics applications. Despite its relative youth and occasional instability, WebGPU’s potential to revolutionize graphics education by making advanced techniques more accessible cannot be overstated.

Why Python Another crucial decision in designing a framework for educational purposes is selecting the programming language. Experiences with using Vulkan and the C++ programming language, as described by Unterguggenberger et al. [11], indicate that students found the intricacies of C++ more challenging than using the Vulkan API itself, even through an abstraction layer. This observation highlights a significant barrier in computer graphics education: the complexity of the programming language can overshadow the learning of graphics concepts. To address this, we chose Python as the programming language for our framework, pyGANDALF.

Python is renowned for its **simplicity and readability**. Its syntax is clean and intuitive, which makes it an excellent choice for beginners. Students can focus on understanding graphics programming concepts without being bogged down by the complexities of the language itself. This ease of use ensures that students can quickly start experimenting with graphics programming, leading to a more engaging and productive learning experience.

The recent remarkable advancements in the field of **Deep Learning**—which is closely related to many computer graphics developments—made Python an easy choice for our framework’s language. Python is the predominant language in machine learning, supported by a vast ecosystem of libraries such as TensorFlow, PyTorch, and Keras. This integration provides a unique opportunity for students to explore advanced topics in graphics programming, such as deep learning super sampling (DLSS), procedural content generation using GANs, and AI-driven animation techniques. Python’s extensive library support simplifies these integrations, making sophisticated techniques accessible to learners. By using Python, pyGANDALF can natively support extensions related to deep learning, allowing students to explore the intersection of graphics and AI seamlessly.

Python has a **vast and active community**. This community support means that there are extensive resources, tutorials, and documentation available, making it easier for students to find help and continue learning independently. Additionally, the rich ecosystem of libraries and frameworks in Python provides powerful tools for various aspects of computer graphics, from image processing to scientific computing.

Python’s high-level nature and dynamic typing enable **rapid development and prototyping**. Students can quickly implement and test their ideas without worrying about the lower-level details of memory management and compilation. This speed of development is particularly beneficial in an educational setting, where the ability to iterate quickly can significantly enhance the learning process.

Experiences with using Python as the framework language, as described by Papagiannakis et al. [12], indicate that Pythonic frameworks have a lot of potential and positive impact on students’ performance. These studies show that students can easily adapt to Python, even if they are not initially familiar with it. The positive reception of Python-based educational tools supports our choice, reinforcing the idea that Python can effectively lower the entry barrier to graphics programming.

Python is a **cross-platform** language, running seamlessly on Windows, macOS, and Linux. This ensures that pyGANDALF can be used in diverse educational environments, accommodating students with different operating systems. This flexibility is crucial for an educational tool intended for widespread use.

Furthermore, Python’s beginner-friendly nature enables students to concentrate on graphics programming without struggling with the compiler and the complexities of a lower-level language like C++. This focus on ease of use ensures that students can maximize their learning experience in graphics programming. By reducing the cognitive load associated with learning a complex language, students can achieve a deeper understanding of graphics principles and algorithms.

In conclusion, the choice of Python as the programming language for pyGANDALF is driven by its user-friendliness, integration with deep learning, extensive community support, rapid development capabilities, and positive educational experiences. By leveraging Python, pyGANDALF provides an accessible and powerful platform for teaching modern computer graphics, empowering students to explore

and innovate without being hindered by language complexities.

Why ECS The last and perhaps most fascinating choice in creating our framework was the decision to use an Entity-Component-System (ECS) architecture. The ECS pattern [13] [14], widely used in 3D applications and game development, decouples data from behavior, simplifying development. It is based on data-oriented design and composition, where entities are assigned independent components, contrasting with the inheritance model of object-oriented design. ECS offers advantages such as enhanced performance in applications with numerous objects, improved maintenance, and parallelization, and a clearer understanding of the application's objects.

In ECS, entities are simple containers that hold components, which are data structures without behavior. Systems, which are responsible for the behavior, operate on these components. This **separation of data and behavior** simplifies the architecture, making the codebase easier to understand and maintain. Students can clearly see the distinction between what the data represents and how it is processed, which aids in learning and debugging.

Furthermore, ECS promotes **data-oriented design**, which can lead to more efficient use of CPU cache and better performance. Although our implementation in Python does not focus on maximal performance or optimizing for cache-friendliness, introducing students to this paradigm helps them understand the principles behind writing efficient code. This knowledge is transferable to other, more performance-critical languages and environments.

ECS is well-suited for applications that manage numerous entities, such as games or simulations. By **decoupling data and behavior**, ECS allows for more straightforward **parallelization**. Systems can operate independently on their respective components, making it easier to distribute work across multiple threads or processors. This aspect of ECS provides a practical example for students to learn about parallel processing and scalability in software design.

ECS's modular nature makes it **highly maintainable and extensible**. Adding new features or modifying existing ones often involves adding or modifying components and systems, without affecting unrelated parts of the codebase. This modularity is a crucial aspect of software engineering, as it facilitates easier updates, bug fixes, and feature additions. Students learn to write code that is flexible and adaptable to changes, a valuable skill in real-world software development.

Implementing an ECS architecture in pyGANDALF promotes **good coding practices**. By introducing students to a programming paradigm they may not have encountered before, ECS helps them develop habits that will enable them to write performant and parallelizable software in the future.

Many students are primarily exposed to object-oriented design (OOD) in their early studies. While OOD is powerful and widely used, it has limitations, particularly in scenarios involving complex interactions and numerous entities. ECS

provides a contrasting approach, highlighting the benefits and drawbacks of different architectural patterns. This comparison enriches students' understanding and prepares them to choose the appropriate design patterns based on the requirements of their projects.

Our implementation does not focus on maximal performance, rendering as many entities as possible, or optimizing for cache-friendliness—given that we're using Python—but on promoting good coding practices. The primary goal is to introduce students to the ECS paradigm, helping them understand its structure and benefits. By working with an ECS-based framework, students can experience firsthand how data-oriented design and decoupling of data and behavior simplify complex software development.

The ECS architecture in pyGANDALF serves as a practical example of how to manage and organize large-scale applications efficiently. Students can see how different systems operate independently yet cohesively within the framework, learning how to design software that is both scalable and maintainable. This experience prepares them for real-world development challenges, where efficient data management and parallel processing are often critical.

In conclusion, the decision to use an ECS architecture in pyGANDALF was driven by its ability to simplify development, improve maintainability, and introduce students to advanced software design principles. By leveraging ECS, we aim to provide an educational tool that not only teaches computer graphics but also instills good coding practices and prepares students for future challenges in software development.

Chapter 2

Related Work

Over the years, numerous frameworks and libraries have been developed, some specifically designed for educational use, and others that, while general-purpose, have found application in classroom settings. In this section, we will briefly examine both categories: frameworks explicitly designed for educational purposes and those that, although more general in nature, are suitable for use in educational environments.

2.1 Generic Frameworks

2.1.1 Three.js, Babylon.js

Both Three.js and Babylon.js are open-source JavaScript libraries renowned for their capabilities in creating and displaying animated 3D graphics within web browsers. Initially developed to abstract the complexities of WebGL, they have since expanded to support WebGPU, a cutting-edge graphics API designed to offer better performance and more advanced features than its predecessor. However, the WebGPU integration in both is still a work in progress, with no official release yet. Additionally, browser support for WebGPU is often hidden behind a developer flag or may not be supported at all. Despite their ease of use, as they require no setup and run directly in the browser, these limitations make these web frameworks less ideal for educational purposes at this time.

Three.js Three.js [15] is a JavaScript library that simplifies the creation of 3D graphics in web applications. It provides a robust API that abstracts many of the lower-level details of WebGL, making it accessible to developers and students with varying levels of experience. With Three.js, users can create complex 3D scenes, animations, and interactive graphics with relatively little code. It has a large and active community, extensive documentation, and numerous examples, which make it a popular choice for both hobbyists and professionals. However, its current WebGPU support is not fully realized, and browser compatibility issues

hinder its potential for leveraging the latest advancements in web graphics.

Babylon.js Babylon.js is another powerful JavaScript library for creating 3D graphics on the web. It offers a comprehensive set of features, including support for physics, animations, materials, and more. Babylon.js also provides a playground environment, which is an online editor where users can write, test, and share their 3D scenes directly in the browser. This feature is particularly useful for educational purposes, as it allows students to experiment and learn in an interactive setting. Like Three.js, Babylon.js is working on integrating WebGPU support, but it is not yet fully available. The limitations in browser support and the experimental nature of WebGPU integration make it less than ideal for stable educational use at this time.

Three.js and Babylon.js are powerful and accessible tools for creating 3D graphics on the web. Their simplicity and ease of use make them excellent for quick prototyping and learning the basics of 3D graphics. However, the ongoing development and limited browser support for WebGPU integration present challenges for their use in a stable educational environment. As such, while these libraries are excellent for introductory and intermediate learning experiences, more robust solutions may be necessary for advanced graphics education that requires the latest technology and consistent performance.

2.1.2 Unity

Unity [16] is renowned in game development for its versatility and robust feature set. Despite not being explicitly tailored for educational use, it has gained popularity in academic circles due to its comprehensive documentation, strong community support, and advanced capabilities. Unity's ease of use and cross-platform compatibility make it particularly valuable for teaching computer graphics concepts, both basic and advanced.

Unity provides extensive documentation, tutorials, and a wide range of learning resources essential for educators and students alike. These resources include step-by-step guides and examples that simplify complex concepts. Courses available on Unity Learn cater to learners at all proficiency levels, making the platform accessible to a diverse audience. The platform benefits from a large and active community comprising developers, educators, and enthusiasts. This community support enables students to find answers, share projects, and collaborate effectively. Forums, online groups, and social media platforms serve as vibrant hubs for learning and problem-solving.

Unity's cross-platform development capabilities extend to Windows, macOS, Linux, iOS, Android, and various game consoles. This versatility is advantageous in educational settings, allowing students to create projects deployable across different devices and operating systems. It also exposes them to the intricacies of cross-platform development, a crucial aspect of modern software engineering. Advanced graphics features in Unity, such as high-fidelity rendering and real-time

lighting, empower educators to teach cutting-edge graphics techniques. The platform’s Scriptable Render Pipeline (SRP) supports custom rendering, offering students insights into graphics programming beyond basic concepts.

Despite its advanced capabilities, Unity remains user-friendly with its drag-and-drop interface and visual scripting tools like Bolt. These features cater to beginners while providing robust scripting capabilities in C# for advanced users. Real-time collaboration features such as Unity Collaborate facilitate teamwork and collaborative learning, essential for group projects common in educational settings.

In teaching computer graphics, Unity serves diverse purposes. For introductory courses, it introduces fundamental game development and 3D graphics concepts. Visual scripting tools enable students to create interactions and behaviors without extensive coding. Advanced courses leverage Unity’s graphics prowess to delve into shader programming, physics-based rendering, and optimization techniques.

Unity also incorporates experimental WebGPU support, expanding its capabilities in interactive web graphics. This addition further enhances its versatility in educational contexts by enabling students to explore cutting-edge web-based graphics programming. WebGPU, an emerging standard for low-level graphics and compute on the web, allows Unity users to leverage modern GPU capabilities directly within web browsers. This experimental support not only facilitates teaching advanced graphics concepts in a web environment but also prepares students for future trends in web development and interactive media.

In conclusion, Unity stands out in computer graphics education due to its extensive support infrastructure, cross-platform capabilities, advanced graphics features, and user-friendly design. Originally developed for game development, its versatility has made it a preferred tool in educational environments, equipping students with essential skills for modern software development and interactive media creation. However, Unity’s higher-level approach abstracts many details and lower-level concepts of modern GPUs, which can limit students’ understanding of what occurs beneath the surface in a graphics pipeline. While Unity facilitates rapid development and enables students to create complex projects efficiently, it may not provide as deep an insight into the intricacies of GPU architecture and low-level graphics programming compared to frameworks that emphasize these aspects. Educators should consider supplementing Unity with additional learning experiences or tools that delve into more detailed GPU concepts, ensuring students gain a well-rounded understanding of graphics programming principles.

2.2 Learning Focused frameworks

2.2.1 The glGA Project

The glGA project, initiated over a decade ago by Papagiannakis et al. [17], addresses the crucial need for a robust framework that simplifies the teaching of computer graphics. At the time of its inception, shader-based graphics pipelines had taken precedence in the industry but posed significant complexity challenges

compared to the deprecated fixed-function OpenGL pipeline. Many educators continued to teach the older OpenGL version or relied on complex game engines and toolkits that shielded students from these complexities.

glGA aimed to bridge this gap by providing a modern, shader-based OpenGL framework built on C++. This framework was specifically designed for use in undergraduate and postgraduate courses. By embracing modern OpenGL practices, glGA empowered educators to introduce students to contemporary graphics programming concepts without the overhead of legacy methods.

The adoption of glGA in educational settings demonstrated its efficacy in facilitating learning and understanding of computer graphics:

- **Simplicity and Power:** glGA offered a balance of simplicity and capability, making it accessible for students while supporting advanced shader-based techniques.
- **Example-driven Learning:** The framework included numerous examples ranging from basic to advanced, which helped students grasp concepts effectively and achieve high academic results.
- **Overcoming Challenges:** By providing a structured environment with clear examples and a modern approach, glGA enabled students to overcome initial hurdles and develop confidence in graphics programming.

The glGA framework exemplifies how a well-designed educational tool can facilitate the teaching and learning of computer graphics effectively. By transitioning from deprecated OpenGL practices to modern shader-based pipelines, glGA not only met educational needs but also prepared students for contemporary graphics programming challenges in both academic and professional settings.

The influence of the glGA project on pyGANDALF underscores the importance of a structured and modern approach to teaching computer graphics. By leveraging the lessons and successes of glGA, pyGANDALF not only builds on established educational principles but also prepares students for the demands of contemporary graphics development in both educational and professional settings. This continuity ensures that pyGANDALF serves as a robust platform for advancing the computer graphics educational process, much like its predecessor, glGA.

2.2.2 The ShaderLab Framework

The ShaderLab Framework, developed by Antoine Toisoul et al. [18], introduces a transformative approach to teaching computer graphics through an Integrated Development Environment (IDE). This IDE enhances the learning experience by providing students with intuitive tools to write, modify, and visualize shader code effectively. Unlike traditional text-based interfaces, ShaderLab's advanced GUI editor environment allows students at Imperial College London to interact with

various components of the graphics pipeline—such as vertex, geometry, and pixel shaders—through intuitive controls and visual feedback.

By incorporating ShaderLab into their curriculum, the authors have structured a comprehensive course syllabus complemented by detailed lab exercises. These exercises guide students through progressively complex graphics programming concepts, from foundational shader techniques to advanced rendering effects. This structured approach not only demystifies shader development but also empowers students to explore creative and conceptual aspects of graphics programming without being hindered by syntax or low-level API interactions.

The influence of ShaderLab on the design of the pyGANDALF framework is evident in its emphasis on user interface integration and educational empowerment. Like ShaderLab, pyGANDALF incorporates a versatile editor layer that supports intuitive scene creation, editing, and serialization in USD format. This functionality enhances accessibility and usability for educators and students, facilitating easier manipulation of scenes and components. By providing similar GUI-based editing capabilities and supporting a structured learning path, pyGANDALF builds on ShaderLab’s legacy to deepen understanding and proficiency in graphics programming within educational settings.

2.2.3 The CodeRunnerGL Framework

CodeRunnerGL, proposed by Burkhard et al. [19], introduces a unique approach to teaching computer graphics through a quiz-based framework. Departing from traditional instructional methods, CodeRunnerGL offers a flexible structure for presenting questions, applying penalties, and providing feedback. Central to its methodology is the ability for students to manipulate a 3D scene in real-time by modifying code directly within their answers.

Key features of CodeRunnerGL include an automated feedback mechanism that evaluates student solutions against example solutions. This evaluation encompasses parameters, data structures, intermediate representations, and OpenGL states, providing detailed feedback based on these comparisons. By leveraging real-time code manipulation and automated feedback, CodeRunnerGL offers a dynamic learning environment where students can interactively explore and refine their understanding of graphics programming concepts.

The framework proposed by Burkhard et al [19]. underscores the diverse approaches available in computer graphics education. By integrating interactive quizzes with real-time code modification and automated evaluation, CodeRunnerGL exemplifies how educational tools can adapt to engage students actively in the learning process. This approach not only facilitates deeper comprehension of graphics programming principles but also encourages experimentation and iterative learning, highlighting the potential for innovation in educational methodologies within the field of computer graphics.

2.2.4 Vulkan All The Way

An exciting endeavor in modernizing computer graphics education is described by Unterguggenberger et al. [11], where they successfully integrated the Vulkan API into the curriculum at TU Wien, Institute of Visual Computing & Human-Centered Technology, Vienna, Austria. Rather than completely replacing the existing OpenGL-based course, they offered students the option to choose between the two APIs for their assignments. To mitigate the unavoidable complexity associated with Vulkan, especially for undergraduate students, they developed an abstraction layer on top of Vulkan to simplify development and setup.

The decision to offer Vulkan alongside OpenGL allowed students to choose based on their comfort level and learning objectives. This approach ensured continuity while introducing students to modern graphics programming techniques supported by Vulkan. Recognizing the inherent complexity of Vulkan, the development of an abstraction layer was crucial. This layer abstracted lower-level Vulkan concepts, such as synchronization and resource management, into higher-level constructs that were easier for students to grasp. This approach aimed to reduce the initial learning curve and enable faster development without sacrificing the power and capabilities of Vulkan.

Surveys conducted as part of the study indicated a predominantly positive response from students who chose the Vulkan-based route. They found the option helpful and interesting, appreciating the exposure to advanced graphics programming concepts. Interestingly, while Vulkan's complexity was acknowledged, students reported more difficulties with the C++ programming language than with Vulkan itself. This insight underscores the challenges inherent in teaching low-level programming languages alongside advanced APIs.

Given the challenges highlighted by Unterguggenberger et al. [11], the focus on using Python in our framework becomes particularly relevant. Python's simplicity and ease of use make it an ideal language for introducing students to programming concepts without the overhead of memory management and complex syntax associated with languages like C++.

The integration of Vulkan into computer graphics education, as demonstrated by Unterguggenberger et al. [11], highlights the importance of adapting teaching methods to include modern APIs while managing complexity. By offering Vulkan alongside OpenGL and developing abstraction layers, educators can effectively introduce students to advanced graphics programming concepts. Our framework, pyGANDALF, extends this approach by leveraging Python to enhance accessibility and manage complexity, ensuring that students can focus on learning and excelling in computer graphics without unnecessary barriers. Through this approach, we aim to empower the next generation of graphics programmers with the skills needed to succeed in both academic and professional environments.

2.2.5 The Elements Project

The Elements Project, as described by Papagiannakis et al. [12], represents a significant initiative in advancing computer graphics education through a transition to a Python-based framework featuring an Entity-Component-System (ECS) within a scenegraph architecture. Deployed at the University of Crete, Department of Computer Science, Herakleion, Greece, and the University of Western Macedonia, Department of Electrical and Computer Engineering, Kozani, Greece, this project aimed to modernize teaching methodologies while maintaining continuity in graphics programming education.

The transition from a C++ based framework to a Python-based one marked a pivotal shift in the Elements Project. While OpenGL remained the chosen graphics API, the adoption of Python introduced several advantages. Python's simplicity and readability facilitated a smoother learning curve for students, allowing them to focus more on understanding graphics concepts rather than grappling with language intricacies.

The incorporation of an ECS within a scenegraph architecture was a strategic decision to enhance modularity and flexibility in the framework. ECS separates data and behavior, providing a more intuitive approach to managing complex interactions within 3D scenes. This architectural choice aligns with modern software design principles prevalent in game development and interactive graphics. The framework also incorporates a very simple and thin editor layer that enables component manipulation such as transformations and camera settings.

Surveys conducted as part of the Elements Project indicated that students adapted well to the Python-based framework. Despite the shift from C++, students demonstrated effective learning and performance in assignments. The transition was supported by positive feedback regarding Python's accessibility and the framework's design, which emphasized practical application and project-based learning.

The Elements Project exemplifies the transformative impact of adopting Python and modern software architectures in computer graphics education. By embracing Python's accessibility and the modular ECS design, educators can enhance student engagement and learning outcomes. The success of the Elements Project reinforces our decision to utilize Python in our framework, pyGANDALF. By leveraging Python's accessibility and the benefits of ECS architecture, we aim to provide a user-friendly environment for learning computer graphics. Through this approach, we aim to contribute to the evolution of graphics programming education and prepare students for future challenges in the field.

2.3 Beyond State of the Art

In the rapidly evolving field of computer graphics, significant advancements have been made in educational methodologies, as highlighted by the extensive research

previously discussed. Despite these advancements, critical challenges and opportunities for further innovation remain. This section outlines how the research presented in this thesis advances beyond the current state of the art, addressing existing gaps and proposing contributions that enhance understanding and capabilities in the field.

Current state-of-the-art techniques and methodologies in computer science education have laid a robust foundation, enabling advancements in several key areas. These include modernizing the education curricula by incorporating contemporary low-level APIs [11] [20], improving accessibility and lowering the barrier to entry through the use of simple and intuitive programming languages that promote experimentation and rapid prototyping [12], deploying user-friendly GUI interfaces for efficient scene creation, editing [18] [19] and assessment [21] [22], and adopting entity-component-system (ECS) architectures and data-oriented design over traditional object-oriented methods [12].

These advancements have been achieved through incremental improvements in various frameworks, each contributing to the field’s progression. This thesis aims to consolidate these pioneering ideas into a single framework, creating a comprehensive solution for teaching computer graphics while also integrating some new ideas. The proposed framework leverages modern low-level APIs with the integration of **WebGPU**, offering students exposure to contemporary graphics programming while maintaining **OpenGL** for comparative learning and easier transition. It also includes an editor layer to streamline the development and creation processes for both students and educators. By utilizing an **ECS** architecture, the framework promotes performance, ease of use, and good programming practices within a **Pythonic** environment that is accessible to all and does not hinder the learning process. The framework also supports **Physically Based Rendering** (PBR) [23] workflows to enhance visual fidelity and provide insight into the intricate process of setting up and understanding the various techniques used to closely simulate the behavior of light in the physical world.

In summary, this thesis merges various pioneering ideas into a cohesive framework, taking the field of computer graphics education a small step further. By integrating modern low-level APIs, leveraging Python for accessibility and deep learning integration, adopting an ECS architecture, and providing a user-friendly editor layer, while also introducing new ideas to educational frameworks such as Physically Based Rendering, the pyGANDALF framework addresses existing educational challenges and sets a new standard for teaching computer graphics.

2.4 Our publications related to this work

In this section we present a short overview of our previous publications related to this work.

MAGES 4.0: Accelerating the World’s Transition to VR Training and Democratizing the Authoring of the Medical Metaverse [24] In this work, we propose MAGES 4.0, a novel software development kit to accelerate the creation of collaborative medical training applications in virtual/augmented reality (VR/AR). Our solution is essentially a low-code metaverse authoring platform for developers to rapidly prototype high-fidelity and high-complexity medical simulations. MAGES breaks the authoring boundaries across extended reality, since networked participants can also collaborate using different VR/AR as well as mobile and desktop devices, in the same metaverse world. With MAGES we propose an upgrade to the outdated 150-year-old master-apprentice medical training model. Our platform incorporates, in a nutshell, the following novelties: 1) 5G edge-cloud remote rendering and physics dissection layer, 2) realistic real-time simulation of organic tissues as soft-bodies under 10 ms, 3) a highly realistic cutting and tearing algorithm, 4) neural network assessment for user profiling and, 5) a VR recorder to record and replay or debrief the training simulation from any perspective.

pyGANDALF - a Geometric, ANimation, Directed, Algorithmic, Learning Framework for Computer Graphics (Submitted) In computer graphics (CG) education and development, the lack of versatile and modern tools is a significant challenge, particularly for providing comprehensive learning experiences with both legacy and cutting-edge technologies. Traditional frameworks often use outdated APIs like OpenGL and Direct3D11, limiting exploration of advanced graphics techniques. We introduce pyGandalf, a powerful, lightweight, open-source CG framework with a pure Entity-Component-System (ECS) design. This framework integrates an editor layer within its ECS architecture, streamlining development for educators and developers. It supports a range of graphics APIs, including the WebGPU and OpenGL APIs, ensuring compatibility across various platforms. By supporting the WebGPU API we aim to bridge the gap in educational tools by enabling the use of state-of-the-art technologies such as compute and ray tracing pipelines, essential for contemporary CG education. Its high fidelity Physically Based Rendering (PBR) capabilities produce stunning visuals, enhancing practical applications and educational demonstrations. The framework integrates the Universal Scene Description (USD) open-source software, supporting scalable 3D scenes with diverse assets and animations, fostering collaborative workflows. The framework includes numerous examples demonstrating various graphics techniques, from fundamental concepts to advanced techniques. These resources aid educators, researchers, and developers, facilitating the transition from educational settings to real-world applications. The framework was evaluated by several expert users to assess the workload and difficulty of developing with pyGandalf in order to enable users to unlock the full potential of computer graphics, ensuring a comprehensive and current education that prepares students for modern CG development demands.

Chapter 3

The pyGANDALF Framework

In this chapter, we introduce pyGANDALF, a framework designed to advance the computer graphics educational process. Let’s take a closer look at the framework, examining its structure and the features it provides to enable educators and students alike to enhance their teaching and learning capabilities. pyGANDALF aims to modernize computer graphics curricula and address the challenges associated with teaching this complex subject.

As mentioned before, our framework employs a similar Entity Component System (ECS) architecture to existing solutions [12] but distinguishes itself by adopting a pure ECS approach with an easy-to-use API for creating components and systems. This approach emphasizes good programming practices and clean code, providing a clear and structured methodology for students.

Like other successful frameworks [12], we choose Python as the framework’s language due to its simplicity, ease of use, and close integration with deep learning frameworks. This decision empowers students to learn computer graphics alongside deep learning techniques, highlighting the intersection between these fields.

Inspired by the success of integrating Vulkan, a modern low-level API, into educational curricula by [11], we choose to support the WebGPU API. WebGPU is a more user-friendly yet powerful API that enables students to understand computer graphics in a deeper, more hardware-focused way. Additionally, we maintain an OpenGL backend to facilitate smooth transitions and comparisons, providing a comprehensive learning experience.

Our framework, inspired by other IDE-like frameworks [18], goes a step further by integrating a full-fledged editor layer. This editor layer encourages students and educators to create and edit more complex scenes with ease and speed, enhancing the overall educational process.

Our framework comes equipped with a wide range of educational examples, covering basic concepts to advanced, state-of-the-art techniques. These examples are designed to facilitate student learning, inspire creativity, and demonstrate the vast possibilities in the field of computer graphics. Through this comprehensive

approach, we aim to provide students with a robust understanding of both fundamental and cutting-edge graphics programming practices. By combining these elements, our framework offers a modern, accessible, and effective tool for teaching computer graphics.

3.1 Entity Component System

We have designed a custom Entity-Component-System (ECS) implementation tailored to our framework’s needs. This implementation focuses on creating a simple and easy-to-understand API while adhering to the principles of a pure ECS architecture.

3.1.1 Design Philosophy

Let us take a closer look on the main pillars that drove our decisions when designing our Entity Component System Implementation:

- **Simplicity and Clarity:** Our ECS implementation emphasizes simplicity and clarity, making it accessible to students who may be new to graphics programming and software design patterns. The API is designed to be intuitive, allowing students to focus on learning and experimenting with ECS concepts without being overwhelmed by complex syntax or low-level details.
- **Adherence to ECS Principles:** Despite the emphasis on simplicity, our implementation adheres to the core principles of a pure ECS architecture. This involves strict separation of data (components) and behavior (systems), promoting a clear and modular design. Entities in our system are merely identifiers, while components are plain data containers and systems encapsulate logic and behavior.
- **Educational Focus:** By prioritizing educational goals, our ECS framework serves as a teaching tool that introduces students to the benefits and challenges of using ECS in real-world applications. This focus ensures that students not only learn how to use the framework but also understand the underlying concepts and design patterns.

3.1.1.1 Performance Considerations

Recognizing Python’s inherent performance limitations compared to lower-level languages like C++, our ECS implementation does not aim to achieve maximum performance or cache efficiency. Instead, it balances performance with readability and ease of use. Python’s dynamic nature and interpreted execution model make it a less ideal choice for performance-critical applications but highly suitable for educational purposes. Despite these constraints, our performance evaluation shows that the ECS implementation performs quite well for educational and prototyping

purposes. By leveraging Python’s strengths and carefully managing resources, we achieve respectable performance that is sufficient for most educational scenarios.

3.1.2 Implementation Details

As mentioned before our ECS implementation adheres to pure ECS principles meaning that:

- Entities are represented by Universally Unique Identifiers (UUIDs) (see Listing 3.1).
- Components consist solely of data, encapsulating no behavior (see Listing 3.2).
- Systems manage functionality and behavior (see Listing A.2).

All the elements described above are contained in a Scene, which holds the entire ECS world. The framework supports multiple scenes and allows for easy and fast switching between them.

```
1 class Entity:
2     def __init__(self):
3         self.id = uuid.uuid4()
```

Listing 3.1: Entity class

```
1 class GravityComponent(Component):
2     def __init__(self):
3         self.force = 5.0
```

Listing 3.2: Example Gravity Component

```
1 class GravitySystem(System):
2     def on_create_entity(self, entity, components):
3         pass
4
5     def on_update_entity(self, ts, entity, components):
6         gravity, transform = components
7         transform.y -= gravity.force * ts
```

Listing 3.3: Gravity System example

3.1.2.1 ECS Structure Deep Dive

In order to better understand the data handling in our ECS implementation, let’s examine Figure 3.1. We maintain an array of all the entities and dedicated arrays for each component type to keep them grouped for easy and fast access. To determine which component is attached to which entity, we use an intermediate structure that holds entity-component references. This structure is essentially a dictionary with the component type as the key and the index in the component array as the value. This way, given an entity, we can retrieve the requested component, if it exists. In Listing 3.4 we see an example usage of the resulting API where we create a scene, enroll entities to that scene and then adding components to the entities.

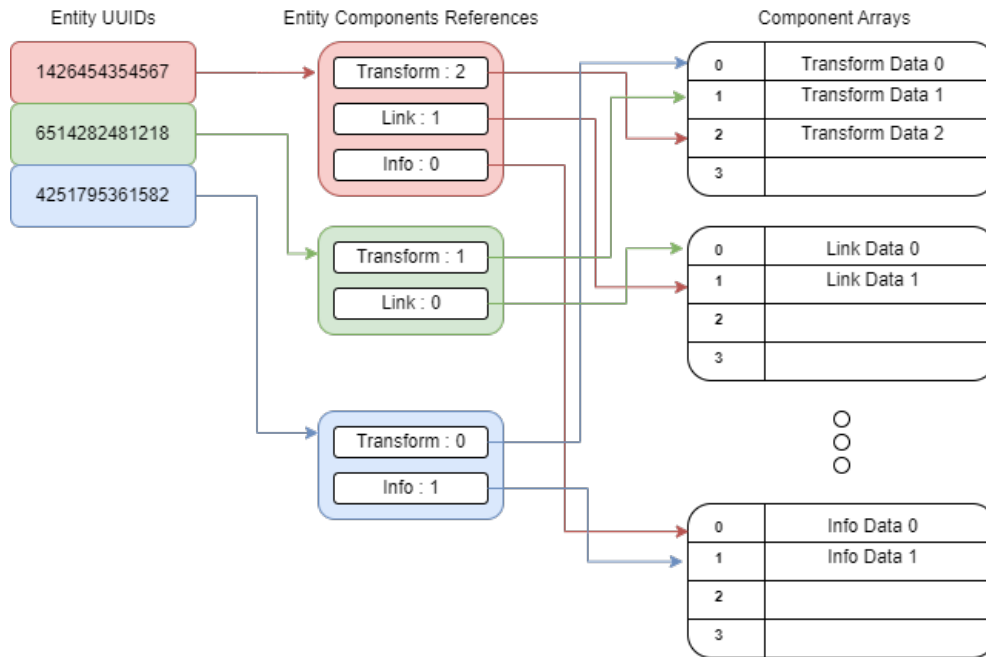


Figure 3.1: Entity Component System setup

```

1 # Create a scene
2 scene = Scene()
3
4 # Enroll entities to scene
5 root = scene.enroll_entity()
6 entity = scene.enroll_entity()
7
8 # Register components to root
9 scene.add_component(root, TransformComponent())
10 scene.add_component(root, InfoComponent('root'))
11 scene.add_component(root, LinkComponent(None))
12
13 # Register components to entity
14 scene.add_component(entity, TransformComponent())
15 scene.add_component(entity, InfoComponent('entity'))
16 scene.add_component(entity, LinkComponent(root))

```

Listing 3.4: Scene creation - entity registration - adding of components

3.1.2.2 Systems Deep Dive

In an Entity-Component-System (ECS) framework, systems play a crucial role in adding behavior and functionality to applications by processing entities based on their components. It is important to note that systems encapsulate specific behaviors, promoting modularity and reusability across different parts of the application. In addition, by focusing on filtered entities, systems optimize performance by processing only relevant components, reducing unnecessary computations. Finally, each system is responsible for a specific aspect of application logic, promoting cleaner and more maintainable code. Here, we'll delve into how systems are implemented and how users can create their own within the pyGANDALF framework.

System Base Class The base System class offers several overridable methods for derived classes to utilize:

- `on_create_system`: Called once when the system is initialized.
- `on_create_entity`: Called once in the first frame for every entity that the system operates on.
- `on_update_system`: Called once every frame to update the system.
- `on_update_entity`: Called once every frame for every entity that the system operates on.
- `on_gui_create_entity` and `on_gui_update_entity`: Used when UI drawing is required and follow similar concepts to the other methods.

An important remark is why there are both create and update methods for the system and the entities. When implementing complex systems, you might need to process the entities/components multiple times within a single frame. For example, in a sophisticated rendering system, multiple passes over the renderable entities might be required to achieve visual effects and implement advanced rendering techniques. The system create and update methods allow for such implementations. However, for simpler systems and behaviors, the `on_create_entity` and `on_update_entity` methods are recommended for a cleaner and simpler API.

User Defined Systems In Listing A.2, we present a simple gravity system that operates on a gravity component and a transform component. Each frame, it applies a primitive gravity force to the transform of each entity it processes. The current entity being processed is passed to the ‘`on_update_entity`’ method, giving users the flexibility to manipulate the entity as needed. This design allows for complex behaviors such as getting, adding, or removing components at runtime. However, it is important to note that such modifications should be avoided within the main update loop due to potential performance implications.

As seen in Listing 3.5, when instantiating a system, we define the components on which the system will operate. An entity must have all specified components to be processed by the system. This filtering of entities and components happens once at startup, and then the entities and components are cached in the system for immediate access. Runtime addition and removal of components are also supported by updating the cached entities and components of each system.

```

1 class GravityComponent(Component):
2     def __init__(self):
3         self.force = 5.0
4
5 class TransformComponent(Component):
6     def __init__(self, translation, rotation, scale):
7         self.translation = translation
8         self.rotation = rotation
9         self.scale = scale
10
11 GravitySystem([GravityComponent, TransformComponent])

```

Listing 3.5: System instantiation

It is rather important that the order that the components are unpacked in the system derived methods matches the order which they were defined when instantiating the system.

Built-In Systems The pyGANDALF framework comes with several built-in systems and components designed to facilitate various aspects of scene management, rendering, and interaction. These systems and components provide essential functionality out of the box, enabling users to focus on higher-level tasks without having to implement basic operations from scratch.

- **TransformSystem**
 - **Components:** Operates on entities that have a **TransformComponent**
 - **Responsibility:** This system calculates the transformation matrix from the translation, rotation, and scale vectors of each entity. The transformation matrix is essential for positioning entities within the 3D space.
- **LinkSystem**
 - **Components:** Operates on entities that have a **LinkComponent** and a **TransformComponent**
 - **Responsibility:** This system calculates the world matrix of an entity's transform based on its position in the scene graph hierarchy. The **LinkComponent** contains a parent node to define the parent entity and maintains a list of child entities. This structure allows for hierarchical transformations, ensuring that child entities move and rotate relative to their parent entities.
- **CameraSystem**
 - **Components:** Operates on entities that have a **CameraControllerComponent**, a **CameraComponent** and a **TransformComponent**
 - **Responsibility:** This system calculates the view and projection matrices for camera entities. These matrices are crucial for rendering the scene from the camera's perspective.
- **CameraControllerSystem**
 - **Components:** Operates on entities that have a **CameraComponent** and a **TransformComponent**
 - **Responsibility:** This system adds FPS-like camera movement to entities. It processes input to move and rotate the camera, providing an interactive way to navigate the 3D scene.
- **MeshRenderingSystem**

- Components: Operates on entities that have a **MeshComponent**, a **MaterialComponent** and a **TransformComponent**
- Responsibility: This system handles the rendering of entities. It interfaces with the GPU to draw meshes on the screen using the specified materials and transformation matrices. In section 3.3, we will discuss in detail how the systems are structured to manage dual APIs (OpenGL and WebGPU) for rendering.
- **LightSystem**
 - Components: Operates on entities that have a **LightComponent** and a **TransformComponent**
 - Responsibility: This system groups light entities together for easy access during rendering. It ensures that all light sources are considered when calculating lighting effects in the scene.

These built-in systems and components form the backbone of the pyGANDALF framework, providing essential functionality for scene management and rendering. By leveraging these pre-built systems, users can quickly set up and manipulate complex scenes, focusing on learning and experimenting with computer graphics concepts.

3.2 Editor layer

The framework includes an editor layer that offers a user-friendly UI for editing and creating scenes quickly and efficiently. This editor layer supports full serialization, allowing scenes to be saved and loaded in the Universal Scene Description (USD) format. The editor layer can be enabled or disabled via a simple flag, enhancing its versatility.

3.2.1 Design Philosophy

The editor (see Figure 3.2) implementation is seamlessly integrated into the ECS architecture. Each panel in the editor is represented as an entity, with dedicated components that facilitate the construction of various UI layouts. This design leverages the Dear ImGui Bundle [25] package, which is widely recognized for its efficiency in creating immediate mode and real-time user interfaces.

By integrating the editor into the ECS, we achieve several benefits:

- **Consistency:** The editor’s functionality is encapsulated within systems, maintaining a consistent approach throughout the framework.
- **Extensibility:** The editor can be easily extended using the same API, allowing for future enhancements without significant restructuring.

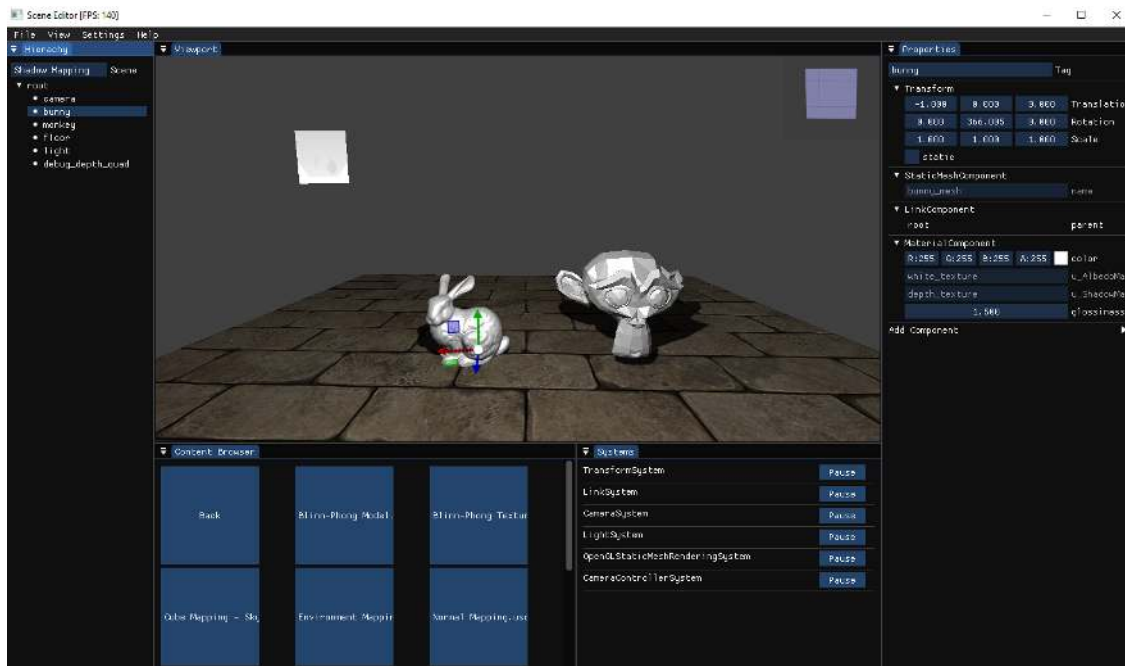


Figure 3.2: Editor Layer

The Dear ImGui Bundle package is chosen for its numerous advantages in creating an editor layer:

- Immediate Mode GUI: Facilitates rapid development and iteration, which is ideal for educational environments.
- Real-time Interaction: Provides immediate feedback to user actions, enhancing the learning experience.
- Popularity and Support: Dear ImGui is widely adopted and well-documented, making it a reliable choice for developers.
- Extensions: There is a wide range of extensions and add-ons that come bundled (Gizmos, Node Editor)

3.2.1.1 Dear ImGui Bundle Package Limitation

One minor setback is that the editor layer is currently available only when using the OpenGL API. This limitation arises because the WebGPU API is relatively new, and there is no implementation for it in the Dear ImGui Bundle package yet. This limitation means that while students can benefit from a rich, interactive editor interface with OpenGL, they will have to forego this convenience when working with WebGPU until further support is developed.

3.2.2 Implementation Details

As described earlier, the editor layer is seamlessly integrated into the Entity Component System (ECS) architecture. This integration is achieved through an `EditorPanelSystem`:

- `EditorPanelSystem`
 - Components: Operates on entities that have an **`EditorPanelComponent`** and an **`EditorSelectableComponent`**
 - Responsibility: This system is responsible for rendering the UI of entities, essentially treating them as editor panels. The `EditorPanel` system leverages the aforementioned `on_gui_update` method from the base `System` class to render its UI.

3.2.2.1 Editor Panels

Several built-in editor panels are provided:

- Hierarchy: Displays the structure of entities.
- Viewport: Renders the 3D view of the scene.
- Inspector: Allows inspection and modification of entity components.
- Content Browser: Facilitates browsing of assets.
- Menu Bar: Provides access to various editor functions and settings.
- Systems: Allows for pausing and resuming the available systems of the scene.

These panels can be extended easily by creating new entities with the appropriate components.

3.2.2.2 Seamless Scene Switching

To enable seamless switching between scenes, all editor-related entities, components, and systems are registered in a separate scene, or ECS world, which runs "in parallel" to the user-created scenes. This approach ensures that the editor remains active and responsive, preventing it from being reloaded and recreated during scene transitions. This design choice avoids performance slowdowns and jarring visual artifacts, maintaining a smooth user experience.

3.2.2.3 Serialization

To maximize utility and impact, the editor layer in the pyGANDALF framework features a powerful and flexible serialization system. This functionality allows users to create scenes and save them in a custom Universal Scene Description (USD) format, capturing comprehensive information about the entity-component system (ECS) world, including various settings and resources required for the scene's functionality.

Key Features of Editor Serialization The serialization system allows users to save and load their scenes to a custom USD format, preserving all ECS-related data such as entities, components, and the relationships between them, as well as settings and resources. For more complex or custom components where automatic serialization might not suffice, the framework provides an API that allows users to define how their components should be serialized and deserialized, accommodating edge cases and custom behaviors.

The ability to load existing scenes and modify them is a crucial feature, enabling users to make quick adjustments to scenes without delving into code, making it easier to fine-tune variables and achieve the desired visual results.

```

1 class MyComponent(Component):
2     def __init__(self, name: str):
3         self.name = name
4
5 def serialize_my_component(prim, component):
6     component_prim_name = prim.CreateAttribute("name", Sdf.ValueTypeNames.String)
7     component_prim_name.Set(component.name)
8     return prim
9
10 USDSerializer().add_serialization_rule(MyComponent, serialize_my_component)
11
12 def deserialize_my_component(prim):
13     name = prim.GetAttribute("name").Get()
14     return MyComponent(str(name))
15
16 USDSerializer().add_deserialization_rule(MyComponent, deserialize_my_component)

```

Listing 3.6: Custom component serialization rule

In this example on Listing 3.6, a custom component is defined and then we register a serialization and deserialization rule using the USDSerializer class. The component can be manipulated as normal with no further action needed.

The serialization feature streamlines the workflow by allowing users to save and load scenes efficiently, reducing the need for manual coding adjustments and speeding up the development process. The custom serialization API ensures that even the most complex and unique components can be serialized correctly, offering flexibility and customization.

3.3 Renderer

3.3.1 Design Philosophy

Given the versatile ECS implementation described earlier, supporting both OpenGL and WebGPU was achieved by creating two separate systems, each responsible for

rendering entities in their respective API. Thanks to Python’s dynamic typing, both systems utilize the same Mesh and Material components, preventing unnecessary duplication and simplifying the overall design.

By maintaining a unified set of components and leveraging Python’s flexibility, we ensure that the same entity data can be processed by either rendering system. This dual-API support allows students to get familiar with and gain insights into both OpenGL and WebGPU without the need for redundant code or data structures.

To handle dual APIs, we implemented two distinct rendering systems:

- OpenGL Rendering System:
 - Processes entities with Mesh, Material and Transform components.
 - Uses OpenGL-specific calls to render the scene.
- WebGPU Rendering System:
 - Also processes entities with Mesh and Material components.
 - Uses WebGPU-specific calls to render the scene.

This approach ensures that students can explore the unique aspects and capabilities of both OpenGL and WebGPU, fostering a deeper understanding of modern graphics programming. Additionally, it highlights the adaptability and reusability of the ECS architecture within different graphics API contexts.

3.3.2 Implementation Details

3.3.2.1 Renderer class

The pyGANDALF framework includes a basic base renderer class designed to structure the implementation of API-specific renderers and the ImGui renderer. This class provides a coherent interface that is easy to understand and extend. While the framework does not aim to switch renderer backends on demand—a feature that would add unnecessary complications and obscure low-level details from users, potentially detracting from the educational objectives—it does offer the minimal required structure to ensure clean and coherent code.

Base Renderer Class The base renderer class serves several critical functions. It organizes the rendering code, ensuring that API-specific details are encapsulated within their respective classes. By bundling common functionality and helper methods within a single class, the framework minimizes code duplication. Furthermore, developers can extend the base class to implement custom rendering features without altering the core rendering system.

The class can be extended to create implementations for different graphics APIs, such as OpenGL and WebGPU, allowing for a clean separation of concerns

where the core rendering logic is abstracted away from the specifics of each API. It also integrates the ImGui renderer, which is essential for the editor layer’s UI components, ensuring that the ImGui renderer can work seamlessly alongside the main rendering system. Additionally, the class provides a centralized place to implement rendering-specific features in order to be used in other systems, such as post-processing effects, without cluttering the main rendering system.

Conclusion The base renderer class in pyGANDALF provides a structured and extensible approach to implementing rendering logic. By separating API-specific details and centralizing common functionality, the framework ensures clean code organization, ease of extension, and reusability. This design not only simplifies the implementation of new features but also makes the framework more robust and maintainable.

3.3.2.2 Rendering Systems

To enhance performance and optimize the rendering systems in pyGANDALF, the framework employs instanced drawing techniques for the WebGPU API. WebGPU is set up for instanced drawing out of the box and does not use custom draw commands, making its implementation simple and straightforward. On the other hand, the OpenGL rendering system uses regular simple drawing.

In the WebGPU rendering system, instead of issuing a separate draw call for each renderable entity—a naive approach that can be highly inefficient—the framework bundles renderable entities based on their material instance and vertex data. This strategy allows the framework to dispatch draw instanced commands in WebGPU, thereby rendering multiple entities with the same material and mesh in a single draw call. In contrast, OpenGL handles these entities with individual draw calls per material instance to avoid complicating the implementation by utilizing instanced drawing, which requires custom draw commands. In the future, a new and separate rendering system could be implemented in OpenGL that utilizes instanced drawing as well to improve rendering performance.

Instanced Drawing Explained Instanced drawing is a technique that allows for the rendering of multiple instances of the same geometry with a single draw call. This is achieved by utilizing the GPU to process multiple instances of vertex data efficiently, significantly reducing the overhead associated with multiple draw calls.

Grouping by Material and Mesh The framework organizes renderable entities into groups based on their material instances and vertex data. This grouping ensures that entities sharing the same material and mesh are drawn together, leveraging the efficiency of instanced drawing. The process involves the following steps:

- **Grouping Entities by Material:** First, entities are grouped based on their material instance. This step minimizes state changes in the GPU, which can be costly.
- **Grouping by Vertex Data:** Within each material group, entities are further grouped by their vertex data (mesh). This allows for efficient batching of draw calls.

To facilitate the instanced drawing technique adequately in the WebGPU API, the use of storage buffers is required. Storage buffers are used to read and write data to and from the GPU and are an alternative to uniform buffers. They are suitable for instances where the volume of data is too large to be passed in a uniform buffer. For each group of renderable entities, discussed above, a storage buffer is created that stores an array of all the model matrices for the objects belonging to that group. This way, in the shader code, the instance ID built-in variable can be utilized to index the storage buffer and retrieve the appropriate model matrix required for our calculations.

Benefits of Instanced Drawing The benefits of Instanced Drawing include:

- **Performance:** By reducing the number of draw calls, instanced drawing significantly improves rendering performance, especially in scenes with a large number of similar entities.
- **Efficiency:** Minimizing state changes in the GPU and maximizing the use of GPU resources.
- **Scalability:** Facilitates the rendering of complex scenes with many entities without a corresponding linear increase in draw calls.

Conclusion By utilizing instanced drawing in the WebGPU rendering system, pyGANDALF optimizes the rendering process, making it more efficient and performant. Grouping entities based on material instance and vertex data, and using draw instanced commands, reduces the overhead associated with multiple draw calls. In the OpenGL rendering system, while not utilizing instanced drawing, entities are still grouped based on their material instance in order to not waste resources by dispatching too many unnecessary draw calls. This approach ensures that the framework can handle complex scenes and large numbers of entities efficiently, providing a robust foundation for teaching and experimenting with modern computer graphics techniques.

3.3.2.3 Handling Resources

Supporting multiple graphics APIs requires efficient resource management, including textures, shaders, and materials. To facilitate this, we implemented helper

singleton classes specific to each API. These classes manage resource creation, usage, and reuse, ensuring efficient performance and a streamlined development experience.

Resource Management Approach

- Textures:
 - Managed by API-specific singleton classes.
 - Ensure textures are loaded, stored, and accessed efficiently.
- Shaders:
 - API-specific singleton classes handle shader compilation and management.
 - Maintain a repository of compiled shaders to avoid redundant compilations.
- Materials:
 - Special attention was given to material management to optimize performance.
 - Materials are reused whenever possible, reducing the overhead of creating new instances.

In Listing 3.7 and 3.8 are examples illustrating the creation of a material that utilizes a shader and a texture. These examples demonstrate the straightforward API provided by our framework for resource management, ensuring ease of use and efficiency.

```

1 # Build texture
2 OpenGLTextureLib().build('pistol_albedo', TextureData(TEXTURES_PATH/'fa_flintlockPistol_albedo.jpg'))
3
4 # Build shader
5 OpenGLShaderLib().build('default_mesh', SHADERS_PATH/'opengl/'/'lit_blinn_phong.vs', SHADERS_PATH/'
6   opengl/'/'lit_blinn_phong.fs')
7
8 # Build Material
9 OpenGLMaterialLib().build('M_Pistol', MaterialData('default_mesh', ['pistol_albedo'], glossiness=2.0)
10 )

```

Listing 3.7: Resource management in OpenGL

```

1 # Build texture
2 WebGPUTextureLib().build('pistol_albedo', TextureData(TEXTURES_PATH/'fa_flintlockPistol_albedo.jpg'))
3
4 # Build shader
5 WebGPUShaderLib().build('default_mesh', SHADERS_PATH/'webgpu/'/'lit_blinn_phong.wgsl')
6
7 # Build Material
8 WebGPUMaterialLib().build('M_Pistol', MaterialData('default_mesh', ['pistol_albedo']))

```

Listing 3.8: Resource management in WebGPU

Shader Reflection A particularly interesting aspect of our framework is the way shaders are handled and preprocessed to provide a dynamic and extensible solution. In many production-ready computer graphics solutions, such as game engines and frameworks, a meta language is often used to facilitate the preprocessing of shaders. This meta language simplifies the shader creation process but abstracts away the original shader languages (GLSL, HLSL, WGSL, etc.).

In our educational framework, we intentionally avoid using a meta language to ensure that students interact directly with the shader language. This direct interaction is crucial as it exposes students to the finer details of shader programming, which are essential for a deeper understanding of the graphics pipeline.

To offer a dynamic solution for shader management while maintaining direct interaction with shader languages, we implemented a simple shader reflection system that supports both GLSL and WGSL. This system allows users to query the ShaderLib of each API to retrieve various pieces of information about the provided shaders, such as uniforms and attributes for both input and output and other useful information. This approach ensures that students can manage shaders effectively without abstracting away the complexities of the shader languages themselves, thereby fostering a more comprehensive learning experience.

3.4 Code Availability

The source code of the pyGANDALF framework can be accessed from this github repository: <https://github.com/papagiannakis/pyGandalf>

Chapter 4

Using pyGANDALF in CG Curriculum

To aid in learning and exploration, pyGANDALF includes a plethora of examples that demonstrate a wide range of graphics techniques, from fundamental concepts to advanced techniques. These examples serve as valuable resources for educators, researchers, and students, facilitating the transition from educational settings to real-world applications.

The examples provided with the framework are categorized based on their level of complexity and difficulty. This structured approach ensures that students gradually build their knowledge and skills, starting from basic concepts and progressing to advanced techniques.

4.1 Introductory Examples

These examples use Jupyter notebooks in Python to guide users step-by-step in creating their first computer graphics applications using the framework. Detailed explanations accompany each step (see Figure 4.1), aiming to familiarize students with the framework API and prepare them to develop their projects independently.

These Introductory tutorials include:

- Triangle: Detailed tutorial on how to render a triangle.
- Textured Quad: Detailed tutorial on how to render a textured quad.
- Textured Quad with Camera: Detailed tutorial on how to render a textured quad using a perspective camera.
- Model Loading: Detailed tutorial on how to load and render a 3D model.
- Scene Graph: Detailed tutorial on how to utilize the scene hierarchy feature to build entity hierarchies.

First, we create a new application. To achieve this, we need to specify two main things:

- The window
 - We create an `OpenGLWindow` and we specify the name, width, height and vertical sync mode accordingly.
- The renderer
 - We specify that we will use the `OpenGLRenderer`.

```
Application().create(OpenGLWindow('Triangle', 1280, 720, True), OpenGLRenderer)
```

Then, we build a shader, lets take a closer look at how this can be achieved.

- 1. From the singleton class `OpenGLShaderLib` we call the `build` method to build our shader.
- 2. We give our shader a name, in this case we chose: "unlit".
- 3. We provide the build function with the path to the source code of the shader stages that we want our shader to utilize, in this case we use a vertex and fragment shader.

```
OpenGLShaderLib().build('unlit', SHADERS_PATH + '/opengl' + '/unlit_simple.vs', SHADERS_PATH + '/opengl' + '/unlit_simple.fs')
```

Then we build a material, lets take a closer look at how this can be achieved.

- 1. From the singleton class `OpenGLMaterialLib` we call the `build` method to build our material.
- 2. We give the material a name, in this case we chose: "M_Unlit".
- 3. We give specify the desired data of the material through the `MaterialData` class.
 - As the first argument, we give them shader name that we want to use, we give the name of the shader we created above, meaning: "unlit".
 - The second argument is used to define which textures the material we use, in our case we dont have textures so we leave the array empty.

```
OpenGLMaterialLib().build('M_Unlit', MaterialData('unlit', []))
```

We create a new scene by instatiating an object of the class `Scene`, we give it the name "Triangle".

```
scene = Scene('Triangle')
```

Then, we enroll a new entity to the scene by calling the `enroll_entity` method. This entity will represent our triangle.

```
triangle = scene.enroll_entity()
```

Figure 4.1: Code snippet from a jupyter notebook step by step example.



Figure 4.2: An Empty Scene.

- User Defined Systems and Components: Detailed tutorial on how to create a new system and component.
- Multiple Scenes: Detailed tutorial on how to create and switch between multiple scenes.

4.2 Beginner Examples

4.2.1 Empty Scene

The "Empty Scene" example is designed to be the introductory point for users learning to create graphics applications with the pyGANDALF framework. This example serves as a foundational exercise, illustrating the simplest graphics application possible: an empty window with a customizable background color. Implemented in both OpenGL and WebGPU, this example demonstrates the basic setup required for a graphics application, including the creation of a window and interaction with the renderer to set the background color (see Figure 4.2).

Objective The primary goal of this example is to familiarize users with the fundamental setup of a graphics application. By completing this example, users will learn: (1) how to initialize a new application, (2) how to create and configure a window, (3) how to interface with the renderer to set the background color, (4) how to create an empty scene.

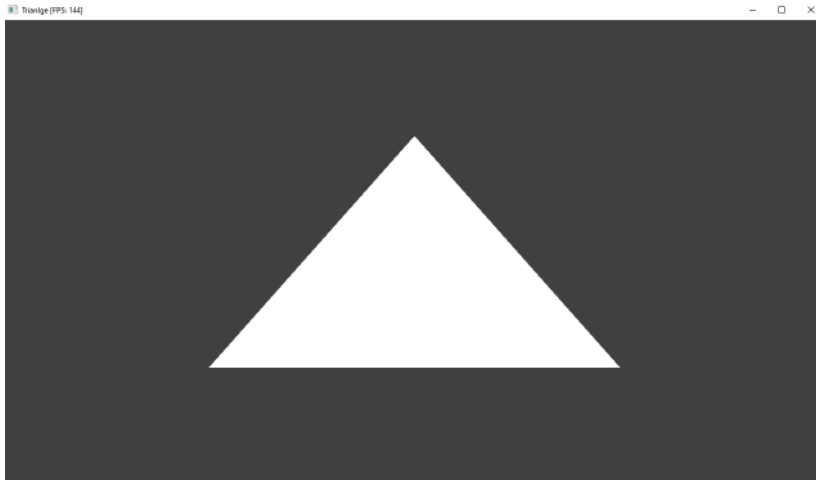


Figure 4.3: Rendering of a Triangle.

4.2.2 Triangle

The "Triangle" example is a cornerstone in computer graphics education, similar to a "Hello World" program in traditional programming. This example is critical as it introduces users to the core concepts and workflows of the pyGANDALF framework. Through this example, which is of course implemented in both OpenGL and WebGPU, users will learn how to create their first shader and material, and how to add components to entities to render their first triangle (see Figure 4.3 and Listings 4.1, 4.2).

```

1 #version 330 core
2 layout(location = 0) in vec3 a_Pos;
3
4 uniform mat4 u_ModelViewProjection;
5 uniform mat4 u_Model;
6
7 void main()
8 {
9     gl_Position = u_ModelViewProjection * vec4(a_Pos, 1.0);
10 }
11
12 #version 330 core
13 out vec4 FragColor;
14
15 uniform vec3 u_Color = vec3(1.0, 1.0, 1.0);
16
17 void main()
18 {
19     FragColor = vec4(u_Color, 1.0);
20 }

```

Listing 4.1: GLSL Shader Code for for Unlit Geometry with Projections

```

1 struct VertexInput {
2     @location(0) a_Position : vec3<f32>
3 };
4 struct VertexOutput {
5     @builtin(position) v_Position: vec4<f32>
6 };
7
8 struct UniformData {
9     viewMatrix: mat4x4f,

```

```

10     projectionMatrix: mat4x4f,
11     objectColor: vec4f,
12 };
13
14 struct ModelData {
15     modelMatrix: array<mat4x4f, 512>,
16 };
17
18 @group(0) @binding(0) var<uniform> u_UniformData: UniformData;
19 @group(0) @binding(1) var<storage, read> u_ModelData: ModelData;
20
21 @vertex
22 fn vs_main(@builtin(instance_index) ID: u32, in: VertexInput) -> VertexOutput {
23     var out: VertexOutput;
24     var mvp: mat4x4f = u_UniformData.projectionMatrix * u_UniformData.viewMatrix * u_ModelData.
        modelMatrix[ID];
25     out.v_Position = mvp * vec4<f32>(in.a_Position, 1.0);
26     return out;
27 }
28
29 @fragment
30 fn fs_main(in: VertexOutput) -> @location(0) vec4<f32> {
31     let physical_color = pow(u_UniformData.objectColor.rgb, vec3<f32>(2.2));
32     return vec4<f32>(physical_color, u_UniformData.objectColor.a);
33 }

```

Listing 4.2: WGSL Shader Code for Unlit Geometry with Projections

Objective The primary goal of this example is to teach users the basics of rendering a simple geometric shape using the pyGANDALF framework. By completing this example, users will learn: (1) how to create and compile shaders, (2) how to create and assign materials, (3) how to add components to entities, (4) how to render a simple triangle on the screen.

4.2.3 Quad

Building on the principles established in the "Triangle" example, the "Quad" example introduces the fundamental technique of indexed drawing. This method involves using an index buffer to define which vertices in the vertex buffer are used to form triangles, allowing for vertex reuse and optimization. This example, implemented in both APIs, helps students understand how indexed drawing can lead to more efficient memory usage and potentially improved performance. It also showcases the way the users can alter the color of the geometry using the pyGANDALF API (see Figure 4.4 and Listings 4.3, 4.4).

```

1 #version 330 core
2 layout(location = 0) in vec3 a_Pos;
3 layout(location = 1) in vec2 a_TexCoord;
4
5 uniform mat4 u_ModelViewProjection;
6 uniform mat4 u_Model;
7
8 out vec2 v_TexCoord;
9
10 void main()
11 {
12     v_TexCoord = a_TexCoord;
13     gl_Position = u_ModelViewProjection * vec4(a_Pos, 1.0);
14 }
15
16 #version 330 core
17 out vec4 FragColor;
18
19 uniform sampler2D u_AlbedoMap;
20 uniform vec3 u_Color = vec3(1.0, 1.0, 1.0);
21
22 in vec2 v_TexCoord;

```



Figure 4.4: Rendering of a Quad.

```

23
24 void main()
25 {
26     FragColor = texture(u_AlbedoMap, v_TexCoord) * vec4(u_Color, 1.0);
27 }

```

Listing 4.3: GLSL Shader Code with Support for Textures and Projections

```

1 struct VertexInput {
2     @location(0) a_Position : vec3<f32>,
3     @location(1) a_TexCoord: vec2<f32>,
4 };
5 struct VertexOutput {
6     @builtin(position) v_Position: vec4<f32>,
7     @location(0) v_TexCoord : vec2<f32>,
8 };
9
10 struct UniformData {
11     viewMatrix: mat4x4f,
12     projectionMatrix: mat4x4f,
13     objectColor: vec4f,
14 };
15
16 struct ModelData {
17     modelMatrix: array<mat4x4f, 512>,
18 };
19
20 @group(0) @binding(0) var<uniform> u_UniformData: UniformData;
21 @group(0) @binding(1) var<storage, read> u_ModelData: ModelData;
22 @group(1) @binding(0) var u_Texture: texture_2d<f32>;
23 @group(1) @binding(1) var u_Sampler: sampler;
24
25 @vertex
26 fn vs_main(@builtin(instance_index) ID: u32, in: VertexInput) -> VertexOutput {
27     var out: VertexOutput;
28     var mvp: mat4x4f = u_UniformData.projectionMatrix * u_UniformData.viewMatrix * u_ModelData.
29         modelMatrix[ID];
30     out.v_Position = mvp * vec4<f32>(in.a_Position, 1.0);
31     out.v_TexCoord = in.a_TexCoord;
32     return out;
33 }
34
35 @fragment
36 fn fs_main(in: VertexOutput) -> @location(0) vec4<f32> {
37     let physical_color = pow(u_UniformData.objectColor.rgb, vec3<f32>(2.2));
38     return textureSample(u_Texture, u_Sampler, in.v_TexCoord) * vec4<f32>(physical_color,
39         u_UniformData.objectColor.a);

```



Figure 4.5: Rendering of a Quad using a Perspective Camera.

38

Listing 4.4: WGSL Shader Code with Support for Textures and Projections

Objective The primary objective of this example is to teach users how to: (1) create and use an index buffer, (2) understand the benefits of indexed drawing, (3) render a more complex shape (a quad) efficiently by reusing vertices. (4) Alter the color of the geometry.

4.2.4 Camera

The "Camera" example demonstrates how to create and utilize a camera within the pyGANDALF framework. This example, implemented in both APIs, is crucial for understanding how camera settings can dramatically affect the appearance of geometry in a scene. Students will learn how to manipulate various camera parameters, such as the field of view (FOV), near and far planes, and projection types, to achieve different visual effects (see Figure 4.5 and Listings 4.3, 4.4).

Objective The primary objective of this example is to teach users how to: (1) create and configure a camera component, (2) understand and adjust camera settings, (3) observe the impact of different camera settings on scene rendering.

4.2.5 Textured Quad

The "Textured Quad" example in pyGANDALF is designed to teach the fundamentals of texture mapping. This example, implemented in both APIs, helps



Figure 4.6: Rendering of a Textured Quad.

students understand the basics of applying textures to 3D objects, specifically focusing on how texture coordinates work, why they produce specific visual results, and how to set them up within the framework (see Figure 4.6 and Listings 4.3, 4.4).

Objective The main objectives of this example are to: (1) introduce the concept of texture mapping, (2) explain how texture coordinates are used to map a texture onto a 3D surface, (3) demonstrate the setup and application of textures in pyGANDALF.

4.2.6 ECSS Cube

The "ECSS Cube" example in pyGANDALF demonstrates the framework's scene graph capabilities, specifically how to create parent-child hierarchies to form complex geometry. In this example, implemented in both APIs, a cube is constructed using six quads, each representing a face of the cube (see Figure 4.7 and Listings 4.3, 4.4).

Objective The main objectives of this example are to: (1) introduce the concept of scene graphs and hierarchical transformations, (2) show how to create and manage parent-child relationships between entities, (3) demonstrate the construction of a 3D cube from multiple quads.

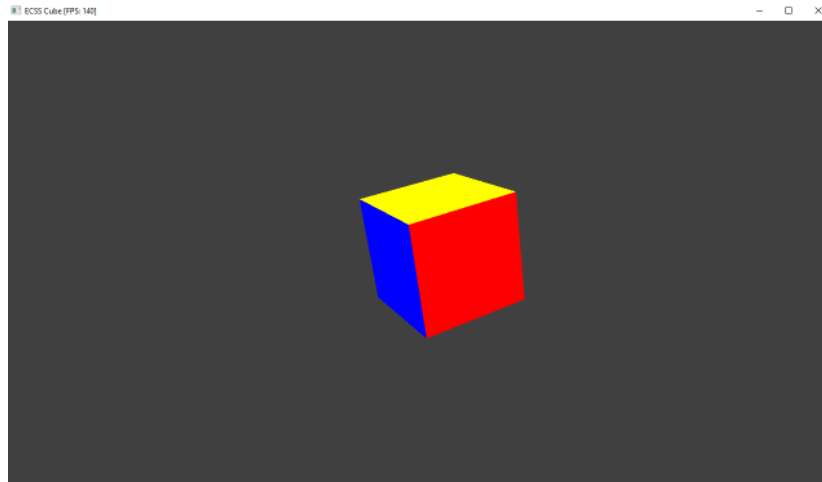


Figure 4.7: A Cube is Constructed Using Six Quads, Each Representing a Face of the Cube.

4.3 Intermediate Examples

4.3.1 Blinn-Phong Model

This example, implemented in both APIs, is the first in the intermediate category and aims to introduce students to the Blinn-Phong shading model, one of the most fundamental and widely used shading techniques in computer graphics. It also demonstrates how to load an OBJ model using the pyGANDALF API (see Figure 4.8 and Listings 4.5, 4.6).

```

1  #version 330 core
2  layout(location = 0) in vec3 a_Position;
3  layout(location = 1) in vec3 a_Normal;
4  layout(location = 2) in vec2 a_TexCoord;
5
6  uniform mat4 u_ModelViewProjection;
7  uniform mat4 u_Model;
8
9  out vec3 v_Position;
10 out vec3 v_Normal;
11 out vec2 v_TexCoord;
12
13 void main()
14 {
15     v_Position = (u_Model * vec4(a_Position, 1.0)).xyz;
16     v_Normal = (transpose(inverse(u_Model)) * vec4(a_Normal, 0.0)).xyz;
17     v_TexCoord = a_TexCoord;
18
19     gl_Position = u_ModelViewProjection * vec4(a_Position, 1.0);
20 }
21
22 #version 330 core
23 out vec4 FragColor;
24
25 uniform sampler2D u_AlbedoMap;
26 uniform vec3 u_Color = vec3(1.0, 1.0, 1.0);
27
28 in vec3 v_Position;
29 in vec3 v_Normal;
30 in vec2 v_TexCoord;
31
32 // Light properties

```



Figure 4.8: Blinn-Phong Model.

```

33 uniform int u_LightCount = 0;
34 uniform vec3 u_LightPositions[16];
35 uniform vec3 u_LightColors[16];
36 uniform float u_LightIntensities[16];
37
38 // Material properties
39 uniform float u_Glossiness = 5.0f;
40 uniform float u_Metallicness = 5.0f;
41
42 // Camera properties
43 uniform vec3 u_ViewPosition = vec3(0.0, 0.0, 10.0);
44
45 void main()
46 {
47     vec3 textureColor;
48
49     textureColor = texture(u_AlbedoMap, v_TexCoord).rgb;
50
51     float ambientCoefficient = 0.1;
52     vec3 normal = normalize(v_Normal);
53     vec3 camDir = normalize(u_ViewPosition - v_Position);
54     vec3 diffuse = vec3(0.0);
55     vec3 specular = vec3(0.0);
56     vec3 ambient = vec3(0.0);
57
58     for (int i = 0; i < u_LightCount; i++)
59     {
60         // diffuse
61         vec3 lightDir = normalize(u_LightPositions[i] - v_Position);
62         float diff = max(dot(lightDir, normal), 0.0);
63         vec3 D = diff * u_LightColors[i] * u_LightIntensities[i];
64
65         // specular
66         vec3 halfwayDir = normalize(lightDir + camDir);
67         float spec = pow(max(dot(normal, halfwayDir), 0.0), 32.0) * u_Glossiness;
68         vec3 S = u_LightColors[i] * spec * u_LightIntensities[i];
69
70         diffuse += D;
71         specular += S;
72         ambient += u_LightColors[i] * u_LightIntensities[i];
73     }
74
75     ambient = ambientCoefficient * ambient;
76
77     vec3 BlinnPhong = ambient + diffuse + specular;
78     vec3 final = textureColor.rgb * BlinnPhong;
79     final = pow(final, vec3(1.0 / 1.2));
80
81     FragColor = vec4(final * u_Color, 1.0);

```

82 }

Listing 4.5: Blinn-Phong GLSL Shader Code

```

1 struct VertexInput {
2     @location(0) a_Position : vec3<f32>,
3     @location(1) a_Normal: vec3<f32>,
4     @location(2) a_TexCoord: vec2<f32>,
5 };
6 struct VertexOutput {
7     @builtin(position) v_Position: vec4<f32>,
8     @location(0) v_Normal : vec3<f32>,
9     @location(1) v_TexCoord : vec2<f32>,
10    @location(2) v_CurrentPosition : vec3<f32>,
11 };
12
13 struct UniformData {
14     viewMatrix: mat4x4f,
15     projectionMatrix: mat4x4f,
16     objectColor: vec4f,
17     viewPosition: vec4<f32>,
18     lightPositions: array<vec4<f32>, 16>,
19     lightColors: array<vec4f, 16>,
20     lightIntensities: array<vec4f, 16>,
21     lightCount: f32,
22 };
23
24 struct ModelData{
25     modelMatrix: array<mat4x4f, 512>,
26 };
27
28 @group(0) @binding(0) var<uniform> u_UniformData: UniformData;
29 @group(0) @binding(1) var<storage, read> u_ModelData: ModelData;
30 @group(1) @binding(0) var u_AlbedoMap: texture_2d<f32>;
31 @group(1) @binding(1) var u_AlbedoSampler: sampler;
32
33 @vertex
34 fn vs_main(@builtin(instance_index) ID: u32, in: VertexInput) -> VertexOutput {
35     var out: VertexOutput;
36     var mvp: mat4x4f = u_UniformData.projectionMatrix * u_UniformData.viewMatrix * u_ModelData.
37         modelMatrix[ID];
38     out.v_Position = mvp * vec4<f32>(in.a_Position, 1.0);
39     out.v_CurrentPosition = (u_ModelData.modelMatrix[ID] * vec4f(in.a_Position, 1.0)).xyz;
40     out.v_Normal = (u_ModelData.modelMatrix[ID] * vec4f(in.a_Normal, 0.0)).xyz;
41     out.v_TexCoord = in.a_TexCoord;
42     return out;
43 }
44
45 @fragment
46 fn fs_main(in: VertexOutput) -> @location(0) vec4<f32> {
47     var textureColor: vec4<f32> = textureSample(u_AlbedoMap, u_AlbedoSampler, in.v_TexCoord);
48
49     var normal: vec3<f32> = normalize(in.v_Normal);
50     var camDir: vec3<f32> = normalize(u_UniformData.viewPosition - vec4<f32>(in.v_CurrentPosition,
51         1.0)).xyz;
52     var diffuse: vec3<f32> = vec3<f32>(0.0);
53     var specular: vec3<f32> = vec3<f32>(0.0);
54     var ambient: vec3<f32> = vec3<f32>(0.0);
55
56     var ambientCoefficient: f32 = 0.1;
57     var u_Glossiness: f32 = 5.0;
58
59     for (var f: f32 = 0.0; f < u_UniformData.lightCount; f = f + 1.0) {
60         var i: i32 = i32(f);
61
62         // ambient
63         ambient = ambient + u_UniformData.lightColors[i].rgb * u_UniformData.lightIntensities[i].x;
64
65         // diffuse
66         var lightDir: vec3<f32> = normalize(u_UniformData.lightPositions[i].xyz - in.
67             v_CurrentPosition);
68         var diff: f32 = max(dot(lightDir, normal), 0.0);
69         var D: vec3<f32> = diff * u_UniformData.lightColors[i].rgb * u_UniformData.lightIntensities[i].x;
70         diffuse = diffuse + D;
71
72         // specular
73         var halfwayDir: vec3<f32> = normalize(lightDir + camDir);
74         var spec: f32 = pow(max(dot(normal, halfwayDir), 0.0), 32.0) * u_Glossiness;
75         var S: vec3<f32> = u_UniformData.lightColors[i].rgb * spec * u_UniformData.lightIntensities[i].x;
76         specular = specular + S;
77     }
78 }

```



Figure 4.9: Blinn-Phong Textured Model.

```

75
76     ambient = ambientCoefficient * ambient;
77
78     var BlinnPhong: vec3<f32> = ambient + diffuse + specular;
79     var finalColor: vec3<f32> = textureColor.rgb * BlinnPhong;
80     finalColor = pow(finalColor, vec3<f32>(1.0 / 1.2));
81
82     // gamma correct
83     let physicalColor = pow(finalColor * u_UniformData.objectColor.rgb, vec3<f32>(2.2));
84
85     return vec4f(physicalColor.r, physicalColor.g, physicalColor.b, textureColor.a);
86 }

```

Listing 4.6: Blinn-Phong WGSL Shader Code

Objective The main objectives of this example are to: (1) explain the principles of the Blinn-Phong shading model, (2) guide students through the implementation of the Blinn-Phong model in shaders, (3) highlight fundamental differences between the shading languages of the two APIs, (4) demonstrate how to load and render an OBJ model using the pyGANDALF framework.

4.3.2 Blinn-Phong Textured Model

The "Blinn-Phong Textured Model" example builds upon the previous Blinn-Phong shading example, demonstrating, in both APIs, how to extend the Blinn-Phong model to render 3D models with textures using the pyGANDALF API. (see Figure 4.9 and Listings 4.5, 4.6).

Objective The main objectives of this example are to: (1) Show how to integrate textures into the Blinn-Phong shading model. (2) Demonstrate the process of loading and rendering textured 3D models using the pyGANDALF framework.



Figure 4.10: Rendering of a Skybox Using Cube Mapping.

4.3.3 Cube Mapping

The "Cube Mapping" example in pyGANDALF, implemented in both APIs, delves into the technique of cube mapping, a pivotal method in computer graphics used to create skyboxes. Skyboxes are essential for enhancing the realism and immersive quality of a scene by simulating a distant environment, such as a sky or landscape, that surrounds the entire scene. Cube mapping achieves this effect by mapping a textures onto the six faces of a cube, creating the illusion of a vast, encompassing environment. Through this example, students will learn how to create a seamless environmental backdrop for their scenes, gaining insights into the practical applications of cube mapping and the optimization techniques necessary for maintaining performance in real-time rendering scenarios (see Figure 4.10 and Listings 4.7, 4.8).

```

1  #version 330 core
2  layout (location = 0) in vec3 a_Position;
3
4  out vec3 v_TexCoords;
5
6  uniform mat4 u_ViewProjection;
7
8  void main()
9  {
10     v_TexCoords = a_Position;
11     vec4 pos = u_ViewProjection * vec4(a_Position, 1.0);
12     gl_Position = pos.xyww;
13 }
14
15 #version 330 core
16 out vec4 FragColor;
17
18 in vec3 v_TexCoords;
19
20 uniform samplerCube u_Skybox;
21
22 void main()
23 {
24     FragColor = texture(u_Skybox, v_TexCoords);

```

25 }

Listing 4.7: Skybox GLSL Shader Code

```

1 struct VertexInput {
2     @location(0) a_Position : vec3<f32>
3 };
4 struct VertexOutput {
5     @builtin(position) v_Position : vec4<f32>,
6     @location(0) v_TextureCoords : vec3<f32>,
7 };
8
9 struct UniformData {
10    viewProjection: mat4x4f,
11 };
12
13 @group(0) @binding(0) var<uniform> u_UniformData: UniformData;
14 @group(1) @binding(0) var u_SkyTexture: texture_cube<f32>;
15 @group(1) @binding(1) var u_SkySampler: sampler;
16
17 @vertex
18 fn vs_main(in: VertexInput) -> VertexOutput {
19     var position: vec4f = u_UniformData.viewProjection * vec4f(in.a_Position, 1.0);
20
21     var output : VertexOutput;
22     output.v_TextureCoords = in.a_Position;
23     output.v_Position = position.xyww;
24     return output;
25 }
26
27 @fragment
28 fn fs_main(in: VertexOutput) -> @location(0) vec4<f32> {
29     return textureSample(u_SkyTexture, u_SkySampler, in.v_TextureCoords);
30 }

```

Listing 4.8: Skybox WGSL Shader Code

Objective The main objectives of this example are to: (1) introduce the concept of cube mapping and its importance in achieving realistic environmental effects, (2) guide students through the step-by-step process of implementing skyboxes using cube maps within the pyGANDALF framework, (3) demonstrate how to efficiently utilize cube maps to minimize performance overhead while maintaining high visual fidelity, (4) illustrate the integration of cube maps into a 3D scene, helping students understand the intricacies of texture coordinates and how they contribute to the final rendered effect.

4.3.4 Environment Mapping

The "Environment Mapping" example in pyGANDALF, implemented in both APIs, builds on the principles demonstrated in the "Cube Mapping" example, extending them to simulate complex interactions between light and different surfaces, providing them with the skills to enhance the visual quality of their 3D scenes significantly. Environment mapping is crucial for simulating the behavior of reflective and refractive materials, such as mirrors and glass, within a 3D scene. This technique uses cube maps to create realistic reflections and refractions, enhancing the visual realism of materials that interact with their surroundings in complex ways (see Figure 4.11 and Listings 4.9, 4.10, 4.11, 4.12).

```

1 #version 330 core
2 layout(location = 0) in vec3 a_Position;
3 layout(location = 1) in vec3 a_Normal;

```

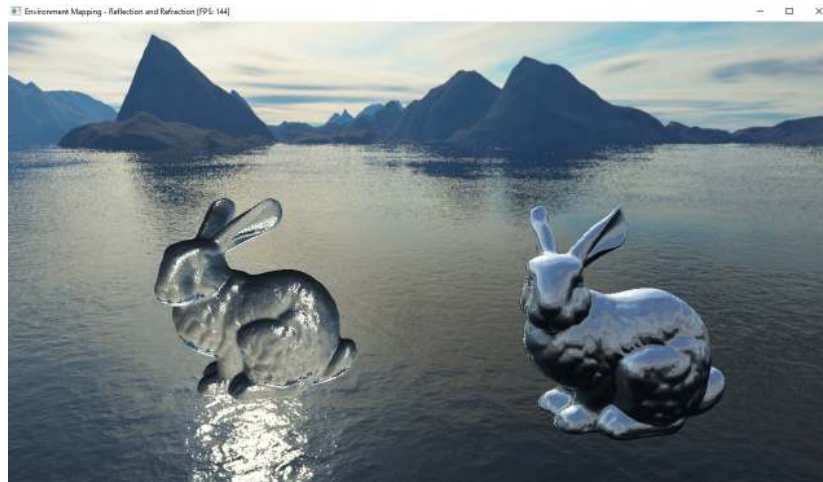


Figure 4.11: Rendering of Meshes Simulating Reflection(Right) and Refrac- tion(Left) by Utilizing Environment Mapping.

```

4 layout(location = 2) in vec2 a_TexCoord;
5
6 out vec3 v_Normal;
7 out vec3 v_Position;
8
9 uniform mat4 u_ModelViewProjection;
10 uniform mat4 u_Model;
11
12 void main()
13 {
14     v_Normal = mat3(transpose(inverse(u_Model))) * a_Normal;
15     v_Position = vec3(u_Model * vec4(a_Position, 1.0));
16     gl_Position = u_ModelViewProjection * vec4(a_Position, 1.0);
17 }
18
19 #version 330 core
20 out vec4 FragColor;
21
22 in vec3 v_Normal;
23 in vec3 v_Position;
24
25 uniform vec3 u_ViewPosition;
26 uniform samplerCube u_Skybox;
27
28 void main()
29 {
30     vec3 I = normalize(v_Position - u_ViewPosition);
31     vec3 R = reflect(I, normalize(v_Normal));
32     FragColor = vec4(texture(u_Skybox, R).rgb, 1.0);
33 }

```

Listing 4.9: Skybox Reflection GLSL Shader Code

```

1 #version 330 core
2 layout(location = 0) in vec3 a_Position;
3 layout(location = 1) in vec3 a_Normal;
4 layout(location = 2) in vec2 a_TexCoord;
5
6 out vec3 v_Normal;
7 out vec3 v_Position;
8
9 uniform mat4 u_ModelViewProjection;
10 uniform mat4 u_Model;
11
12 void main()
13 {
14     v_Normal = mat3(transpose(inverse(u_Model))) * a_Normal;

```

```

15     v_Position = vec3(u_Model * vec4(a_Position, 1.0));
16     gl_Position = u_ModelViewProjection * vec4(a_Position, 1.0);
17 }
18
19 #version 330 core
20 out vec4 FragColor;
21
22 in vec3 v_Normal;
23 in vec3 v_Position;
24
25 uniform vec3 u_ViewPosition;
26 uniform samplerCube u_Skybox;
27
28 void main()
29 {
30     float ratio = 1.00 / 1.52;
31     vec3 I = normalize(v_Position - u_ViewPosition);
32     vec3 R = refract(I, normalize(v_Normal), ratio);
33     FragColor = vec4(texture(u_Skybox, R).rgb, 1.0);
34 }

```

Listing 4.10: Skybox Refraction GLSL Shader Code

```

1 struct VertexInput {
2     @location(0) a_Position : vec3<f32>,
3     @location(1) a_Normal : vec3<f32>,
4     @location(2) a_TexCoord : vec3<f32>,
5 };
6 struct VertexOutput {
7     @builtin(position) v_Position : vec4<f32>,
8     @location(0) v_Normal : vec3<f32>,
9 };
10
11 struct UniformData {
12     viewMatrix: mat4x4f,
13     projectionMatrix: mat4x4f,
14     viewPosition: vec4<f32>,
15 };
16
17 struct ModelData {
18     inverseModelMatrix: array<mat4x4f, 512>,
19     modelMatrix: array<mat4x4f, 512>,
20 };
21
22 @group(0) @binding(0) var<uniform> u_UniformData: UniformData;
23 @group(0) @binding(1) var<storage, read> u_ModelData: ModelData;
24 @group(1) @binding(0) var u_SkyTexture: texture_cube<f32>;
25 @group(1) @binding(1) var u_SkySampler: sampler;
26
27 @vertex
28 fn vs_main(@builtin(instance_index) ID: u32, in: VertexInput) -> VertexOutput {
29     var output : VertexOutput;
30     output.v_Normal = (transpose(u_ModelData.inverseModelMatrix[ID]) * vec4(in.a_Normal, 1.0)).xyz;
31     output.v_Position = u_UniformData.projectionMatrix * u_UniformData.viewMatrix * u_ModelData.
        modelMatrix[ID] * vec4(in.a_Position, 1.0);
32     return output;
33 }
34
35 @fragment
36 fn fs_main(in: VertexOutput) -> @location(0) vec4<f32> {
37     var I : vec3f = normalize(in.v_Position.xyz - u_UniformData.viewPosition.xyz);
38     var R : vec3f = reflect(I, normalize(in.v_Normal));
39     return vec4f(textureSample(u_SkyTexture, u_SkySampler, R).rgb, 1.0);
40 }

```

Listing 4.11: Skybox Reflection WGSL Shader Code

```

1 struct VertexInput {
2     @location(0) a_Position : vec3<f32>,
3     @location(1) a_Normal : vec3<f32>,
4     @location(2) a_TexCoord : vec3<f32>,
5 };
6 struct VertexOutput {
7     @builtin(position) v_Position : vec4<f32>,
8     @location(0) v_Normal : vec3<f32>,
9 };
10
11 struct UniformData {
12     viewMatrix: mat4x4f,
13     projectionMatrix: mat4x4f,
14     viewPosition: vec4<f32>,

```

```

15 };
16
17 struct ModelData {
18     inverseModelMatrix: array<mat4x4f, 512>,
19     modelMatrix: array<mat4x4f, 512>,
20 };
21
22 @group(0) @binding(0) var<uniform> u_UniformData: UniformData;
23 @group(0) @binding(1) var<storage, read> u_ModelData: ModelData;
24 @group(1) @binding(0) var u_SkyTexture: texture_cube<f32>;
25 @group(1) @binding(1) var u_SkySampler: sampler;
26
27 @vertex
28 fn vs_main(@builtin(instance_index) ID: u32, in: VertexInput) -> VertexOutput {
29     var output : VertexOutput;
30     output.v_Normal = (transpose(u_ModelData.inverseModelMatrix[ID]) * vec4(in.a_Normal, 1.0)).xyz;
31     output.v_Position = u_UniformData.projectionMatrix * u_UniformData.viewMatrix * u_ModelData.
32         modelMatrix[ID] * vec4(in.a_Position, 1.0);
33     return output;
34 }
35
36 @fragment
37 fn fs_main(in: VertexOutput) -> @location(0) vec4<f32> {
38     var ratio : f32 = 1.00 / 1.52;
39     var I : vec3f = normalize(in.v_Position.xyz - u_UniformData.viewPosition.xyz);
40     var R : vec3f = refract(I, normalize(in.v_Normal), ratio);
41     return vec4f(textureSample(u_SkyTexture, u_SkySampler, R).rgb, 1.0);
42 }

```

Listing 4.12: Skybox Refraction WGSL Shader Code

Objective The main objectives of this example are to: (1) expand on the foundational knowledge of cube mapping to include more advanced techniques such as reflection and refraction, (2) guide students through the process of simulating reflective surfaces, like mirrors, by mapping environment reflections onto these surfaces, (3) demonstrate how to achieve refraction effects to simulate materials like glass, showing how light interacts with these materials and bends as it passes through, (4) explain the mathematical and shader-based approaches necessary to implement reflection and refraction using cube maps within the pyGANDALF framework.

4.3.5 Basic Compute

The "Basic Compute" example in pyGANDALF introduces students to compute shaders, a powerful addition to the graphics pipeline with applications beyond traditional rendering tasks. Compute shaders leverage the parallel processing capabilities of GPUs to perform general-purpose computations efficiently, enabling tasks that would be impractical or slow on the CPU alone. In this introductory example, students will learn how to pass data to the GPU, process it using compute shaders, and retrieve the results back to the CPU for further use or processing. Compute shaders are only available in the WebGPU API due to the lack of cross platform support for them in the OpenGL API (see Listing 4.13)

```

1 struct Data {
2     data: array<f32, 1269>,
3 };
4
5 @group(0) @binding(0) var<storage, read> inputBuffer: Data;
6 @group(0) @binding(1) var<storage, read_write> outputBuffer: Data;
7
8 // The function to evaluate for each element of the processed buffer
9 fn f(x: f32) -> f32 {

```

```
10     return x * 10.0;
11 }
12
13 @compute @workgroup_size(32, 1, 1)
14 fn computeStuff(@builtin(global_invocation_id) id: vec3<u32>) {
15     // Apply the function f to the buffer element at index id.x:
16     outputBuffer.data[id.x] = f(inputBuffer.data[id.x]);
17 }
```

Listing 4.13: Compute WGSL Shader Code

Objective The main objectives of this example are to: (1) introduce students to the concept and capabilities of compute shaders in modern graphics programming, (2) demonstrate how compute shaders leverage GPU parallelism to accelerate computations, (3) teach students the process of transferring data between the CPU and GPU for computational tasks, (4) provide hands-on experience in writing and executing compute shaders within the pyGANDALF framework.

4.3.6 User Defined System & Component

The "User Defined System & Component" example in pyGANDALF empowers students to delve deeper into the framework's Entity Component System (ECS) API. Here, students will learn how to extend the capabilities of their graphics applications by creating custom Components and Systems. Using the intuitive pyGANDALF API, they will explore the process of defining new components to store specific data and systems to process this data efficiently. This example in particular showcases how can we achieve rigid body animation by rotating entities around their y-axis, which can later be extended to much more complex movements and animations.

Objective The main objectives of this example are to: (1) enable students to understand the structure and benefits of an Entity Component System (ECS), (2) teach students how to create custom Components tailored to their application's needs, (3) demonstrate the implementation of custom Systems to process data associated with custom Components, (4) encourage students to experiment with extending the pyGANDALF framework to add unique functionality to their graphics applications.

4.3.7 Multiple Scenes

The "Multiple Scenes" example in pyGANDALF introduces students to the robust scene management capabilities of the framework. Students will delve into creating and managing multiple scenes within their graphics applications, gaining a deeper understanding of how scene switching and event handling are implemented. This example empowers students to organize their graphics projects efficiently while enhancing their understanding of scene lifecycle management and event-driven programming within pyGANDALF.

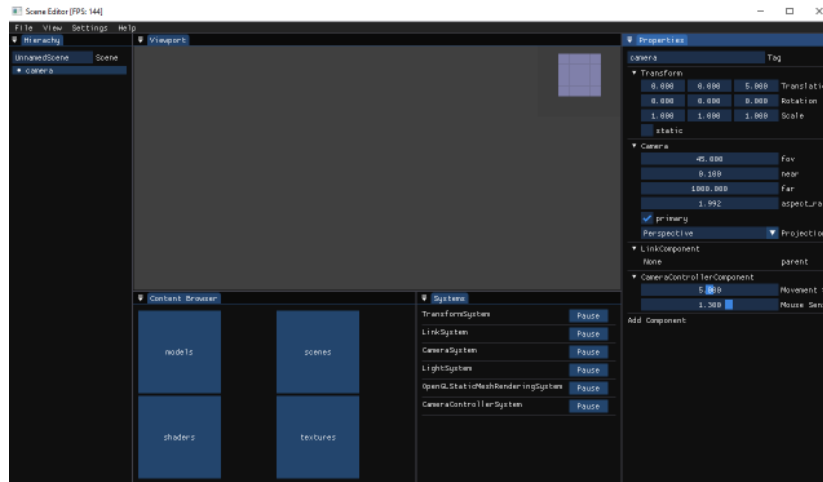


Figure 4.12: The Scene Editor.

Objective The main objectives of this example are to: (1) familiarize students with the concept of scene management in computer graphics applications, (2) guide students in creating multiple scenes using the pyGANDALF framework, (3) demonstrate the process of switching between scenes dynamically during runtime, (4) introduce students to attaching callbacks to the event system to handle scene change events effectively.

4.3.8 The Scene Editor

This example acts like a central hub of the framework. It contains an empty scene with the editor layer attached that is ready for creation. From this example, the students can browse through the available serialized scenes of pyGandalf, using the editor, providing a fast and easy way to access and edit scenes. Educators can also leverage this for seamless transitions between scenes, facilitating the demonstration and explanation of various graphics techniques and features through an editor UI. This accelerates the process compared to manually opening and running code examples. (see Figure 4.12).

Objective The main objectives of this example are to: (1) enable students to browse and interact with serialized scenes within pyGANDALF using the editor interface, (2) facilitate fast and intuitive scene editing, encouraging experimentation and learning in graphics programming, (3) provide educators with a tool to swiftly switch between different serialized scenes, (4) enable educators to demonstrate various graphics techniques and features efficiently through the editor UI.

4.4 Advanced Examples

4.4.1 Tessellation Shaders

The "Tessellation Shaders" example in pyGANDALF, implemented only in OpenGL, introduces students to the tessellation stage of the graphics pipeline, a more advanced and less commonly used stage compared to the vertex and fragment stages. The tessellation stage occurs after the vertex stage and includes two additional shaders: the Tessellation Control Shader (TCS) and the Tessellation Evaluation Shader (TES).

The Tessellation Control Shader's primary task is to determine the level of tessellation. This involves a three-step process: (1) specifying the number of vertices per patch, (2) performing any necessary transformations on each vertex of the patch, (3) specifying the tessellation levels, which control the amount of tessellation to be applied.

Following the TCS, the Tessellation Primitive Generator performs the actual tessellation of the patch, generating intermediate points. Although there is no specific shader code for the Tessellation Primitive Generator, it relies on the output of the TCS and the input of the TES to guide the generation process. The tessellation control shader outputs the tessellation levels, or number of times to subdivide each edge. The tessellation evaluation shader's input then specifies how to space the subdivisions on each edge.

The Tessellation Evaluation Shader's job is to specify the (x,y,z) location of each generated intermediate point. The presence of the TES in the pipeline triggers the tessellation primitive generation, making it crucial for the tessellation process.

In pyGANDALF, students will explore an example that showcases rendering a huge heightmap with dynamic Level of Detail (LOD) to help them better understand the use case of tessellation shaders. This example provides a practical application of tessellation, illustrating its importance in rendering detailed and scalable graphics. (see Figure 4.13 and Listing 4.14).

```

1 // tessellation control
2 #version 410 core
3
4 layout(vertices=4) out;
5
6 uniform mat4 u_Model;
7 uniform mat4 u_View;
8
9 in vec2 v_TexCoord[];
10 out vec2 v_TextureCoord[];
11
12 void main()
13 {
14     // pass attributes through
15     gl_out[gl_InvocationID].gl_Position = gl_in[gl_InvocationID].gl_Position;
16     v_TextureCoord[gl_InvocationID] = v_TexCoord[gl_InvocationID];
17
18     // invocation zero controls tessellation levels for the entire patch
19     if (gl_InvocationID == 0)
20     {
21         // Step 1: define constants to control tessellation parameters
22         const int MIN_TESS_LEVEL = 4;
23         const int MAX_TESS_LEVEL = 64;
24         const float MIN_DISTANCE = 20;
25         const float MAX_DISTANCE = 800;
26
27         // Step 2: transform each vertex into eye space

```

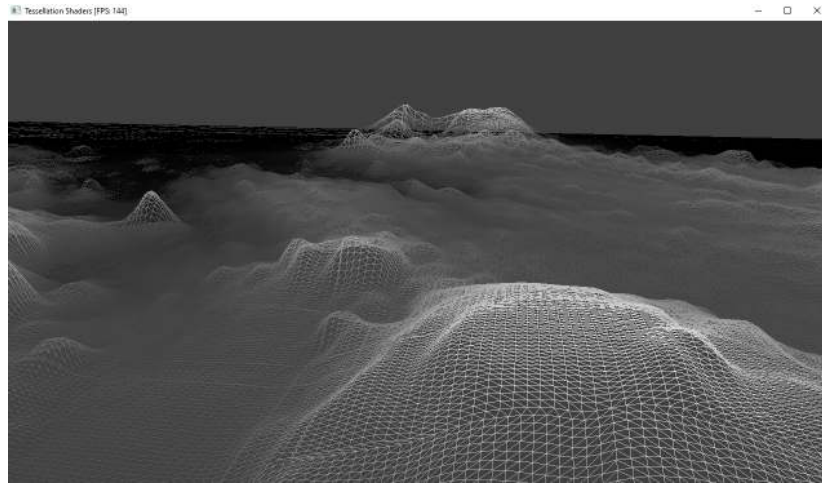


Figure 4.13: Rendering of a Height Map with Dynamic Level of Detail Using Tessellation Shaders.

```

28     vec4 eyeSpacePos00 = u_View * u_Model * gl_in[0].gl_Position;
29     vec4 eyeSpacePos01 = u_View * u_Model * gl_in[1].gl_Position;
30     vec4 eyeSpacePos10 = u_View * u_Model * gl_in[2].gl_Position;
31     vec4 eyeSpacePos11 = u_View * u_Model * gl_in[3].gl_Position;
32
33     // Step 3: "distance" from camera scaled between 0 and 1
34     float distance00 = clamp( (abs(eyeSpacePos00.z) - MIN_DISTANCE) / (MAX_DISTANCE-MIN_DISTANCE)
35     , 0.0, 1.0 );
36     float distance01 = clamp( (abs(eyeSpacePos01.z) - MIN_DISTANCE) / (MAX_DISTANCE-MIN_DISTANCE)
37     , 0.0, 1.0 );
38     float distance10 = clamp( (abs(eyeSpacePos10.z) - MIN_DISTANCE) / (MAX_DISTANCE-MIN_DISTANCE)
39     , 0.0, 1.0 );
40     float distance11 = clamp( (abs(eyeSpacePos11.z) - MIN_DISTANCE) / (MAX_DISTANCE-MIN_DISTANCE)
41     , 0.0, 1.0 );
42
43     // Step 4: interpolate edge tessellation level based on closer vertex
44     float tessLevel0 = mix( MAX_TESS_LEVEL, MIN_TESS_LEVEL, min(distance10, distance00) );
45     float tessLevel1 = mix( MAX_TESS_LEVEL, MIN_TESS_LEVEL, min(distance00, distance01) );
46     float tessLevel2 = mix( MAX_TESS_LEVEL, MIN_TESS_LEVEL, min(distance01, distance11) );
47     float tessLevel3 = mix( MAX_TESS_LEVEL, MIN_TESS_LEVEL, min(distance11, distance10) );
48
49     // Step 5: set the corresponding outer edge tessellation levels
50     gl_TessLevelOuter[0] = tessLevel0;
51     gl_TessLevelOuter[1] = tessLevel1;
52     gl_TessLevelOuter[2] = tessLevel2;
53     gl_TessLevelOuter[3] = tessLevel3;
54
55     // Step 6: set the inner tessellation levels to the max of the two parallel edges
56     gl_TessLevelInner[0] = max(tessLevel1, tessLevel3);
57     gl_TessLevelInner[1] = max(tessLevel0, tessLevel2);
58 }
59
60 // tessellation evaluation shader
61 #version 410 core
62 layout (quads, fractional_odd_spacing, ccw) in;
63
64 uniform sampler2D u_HeightMap; // the texture corresponding to our height map
65 uniform mat4 u_Model; // the model matrix
66 uniform mat4 u_View; // the view matrix
67 uniform mat4 u_Projection; // the projection matrix
68
69 // received from Tessellation Control Shader - all texture coordinates for the patch vertices
70 in vec2 v_TextureCoord[];
71 // send to Fragment Shader for coloring
72 out float v_Height;

```

```

72
73 void main()
74 {
75     // get patch coordinate
76     float u = gl_TessCoord.x;
77     float v = gl_TessCoord.y;
78
79     // retrieve control point texture coordinates
80     vec2 t00 = v_TextureCoord[0];
81     vec2 t01 = v_TextureCoord[1];
82     vec2 t10 = v_TextureCoord[2];
83     vec2 t11 = v_TextureCoord[3];
84
85     // bilinearly interpolate texture coordinate across patch
86     vec2 t0 = (t01 - t00) * u + t00;
87     vec2 t1 = (t11 - t10) * u + t10;
88     vec2 texCoord = (t1 - t0) * v + t0;
89
90     // lookup texel at patch coordinate for height and scale + shift as desired
91     v_Height = texture(u_HeightMap, texCoord).y * 64.0 - 16.0;
92
93     // retrieve control point position coordinates
94     vec4 p00 = gl_in[0].gl_Position;
95     vec4 p01 = gl_in[1].gl_Position;
96     vec4 p10 = gl_in[2].gl_Position;
97     vec4 p11 = gl_in[3].gl_Position;
98
99     // compute patch surface normal
100     vec4 uVec = p01 - p00;
101     vec4 vVec = p10 - p00;
102     vec4 normal = normalize( vec4(cross(vVec.xyz, uVec.xyz), 0) );
103
104     // bilinearly interpolate position coordinate across patch
105     vec4 p0 = (p01 - p00) * u + p00;
106     vec4 p1 = (p11 - p10) * u + p10;
107     vec4 p = (p1 - p0) * v + p0;
108
109     // displace point along normal
110     p += normal * v_Height;
111
112     // output patch point position in clip space
113     gl_Position = u_Projection * u_View * u_Model * p;
114 }

```

Listing 4.14: Tessellation GLSL Shader Code

Objective The main objectives of this example are to: (1) expose students to the tessellation stage of the graphics pipeline, beyond the commonly used vertex and fragment stages, (2) teach students how to implement and utilize the Tessellation Control Shader and Tessellation Evaluation Shader in pyGANDALF, (3) demonstrate the process of specifying the number of vertices per patch, performing transformations, and controlling tessellation levels, (4) enable students to understand the generation and placement of intermediate points through the Tessellation Evaluation Shader.

4.4.2 Geometry Shaders

The "Geometry Shaders" example in pyGANDALF, implemented only in OpenGL, introduces students to the geometry shader, an optional stage in the graphics pipeline that lies between the vertex and fragment shaders. The geometry shader takes as input a set of vertices forming a single primitive, such as a point or a triangle. Unlike other shader stages, the geometry shader has the unique ability to transform these vertices in various ways before passing them to the next stage.

What sets the geometry shader apart is its capability to convert the original primitive into entirely different primitives. It can generate more vertices than

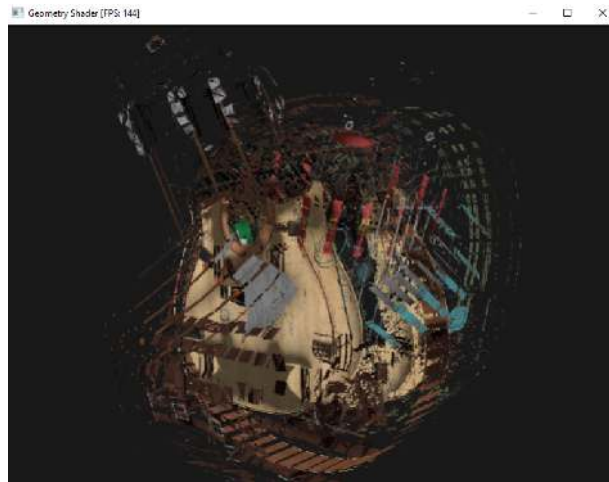


Figure 4.14: Example of an Exploding Mesh Using a Geometry Shader.

initially provided, enabling the creation of complex effects and transformations. By leveraging this stage, students can explore a range of techniques such as exploding objects, optimized particle systems, and other sophisticated visual effects. (see Figure 4.14 and Listing 4.15).

```

1  #version 330 core
2  layout (triangles) in;
3  layout (triangle_strip, max_vertices = 3) out;
4
5  in VS_OUT {
6      vec2 texCoords;
7  } gs_in[];
8
9  out vec2 v_TexCoords;
10
11 uniform float u_Time;
12
13 vec4 explode(vec4 position, vec3 normal)
14 {
15     float magnitude = 2.0;
16     vec3 direction = normal * ((sin(u_Time) + 1.0) / 2.0) * magnitude;
17     return position + vec4(direction, 0.0);
18 }
19
20 vec3 GetNormal()
21 {
22     vec3 a = vec3(gl_in[0].gl_Position) - vec3(gl_in[1].gl_Position);
23     vec3 b = vec3(gl_in[2].gl_Position) - vec3(gl_in[1].gl_Position);
24     return normalize(cross(a, b));
25 }
26
27 void main() {
28     vec3 normal = GetNormal();
29
30     gl_Position = explode(gl_in[0].gl_Position, normal);
31     v_TexCoords = gs_in[0].texCoords;
32     EmitVertex();
33     gl_Position = explode(gl_in[1].gl_Position, normal);
34     v_TexCoords = gs_in[1].texCoords;
35     EmitVertex();
36     gl_Position = explode(gl_in[2].gl_Position, normal);
37     v_TexCoords = gs_in[2].texCoords;
38     EmitVertex();
39     EndPrimitive();
40 }

```

Listing 4.15: Geometry GLSL Shader Code

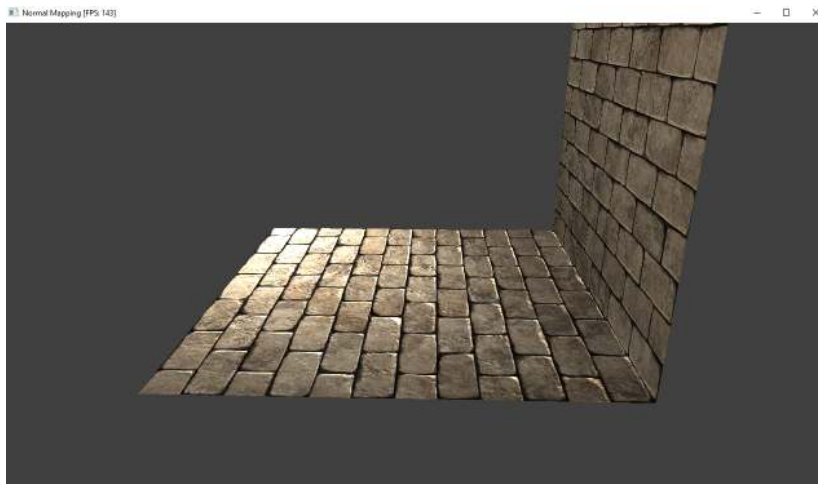


Figure 4.15: Showcase of Normal Mapping in a Brick Textured Wall.

Objective The main objectives of this example are to: (1) introduce students to the geometry shader and its position within the graphics pipeline, (2) demonstrate how the geometry shader processes a set of vertices forming a primitive and can transform them, (3) show how to utilize the geometry shader to convert primitives into different forms and generate additional vertices, (4) provide examples of advanced effects achievable with geometry shaders, including exploding objects and optimized particle systems, (5) enhance students' understanding of advanced shader programming and the potential applications of geometry shaders in real-world graphics scenarios.

4.4.3 Normal Mapping

In this example, implemented in both APIs, students will see in action a fundamental technique in computer graphics that significantly improves the visual fidelity of a scene: normal mapping. Normal mapping utilizes textures to store normal information rather than color information. Each texture stores a 3-dimensional vector, representing the normal vector at each pixel. This method greatly enhances the accuracy of lighting, providing more detailed and realistic surface details compared to traditional per-vertex normals. (see Figure 4.15 and Listings 4.16, 4.17).

```

1 #version 330 core
2 layout(location = 0) in vec3 a_Position;
3 layout(location = 1) in vec3 a_Normal;
4 layout(location = 2) in vec2 a_TexCoord;
5 layout(location = 3) in vec3 a_Tangent;
6 layout(location = 4) in vec3 a_Bitangent;
7
8 uniform mat4 u_ModelViewProjection;
9 uniform mat4 u_Model;
10 uniform vec3 u_LightPositions[16];
11 uniform int u_LightCount = 0;
12 uniform vec3 u_ViewPosition = vec3(0.0, 0.0, 10.0);
13
14 out vec2 v_TexCoord;

```

```

15 out vec3 v_TangentLightPos[16];
16 out vec3 v_TangentViewPos;
17 out vec3 v_TangentFragPos;
18 flat out int v_LightCount;
19
20 void main()
21 {
22     mat3 normalMatrix = transpose(inverse(mat3(u_Model)));
23     vec3 T = normalize(normalMatrix * a_Tangent);
24     vec3 B = normalize(normalMatrix * a_Bitangent);
25     vec3 N = normalize(normalMatrix * a_Normal);
26
27     // Re-orthogonalize T with respect to N
28     T = normalize(T - dot(T, N) * N);
29
30     mat3 TBN = transpose(mat3(T, B, N));
31
32     v_TexCoord = a_TexCoord;
33
34     for (int i = 0; i < u_LightCount; i++)
35     {
36         v_TangentLightPos[i] = TBN * u_LightPositions[i];
37     }
38
39     v_TangentViewPos = TBN * u_ViewPosition;
40     v_TangentFragPos = TBN * (u_Model * vec4(a_Position, 1.0)).xyz;
41
42     v_LightCount = u_LightCount;
43
44     gl_Position = u_ModelViewProjection * vec4(a_Position, 1.0);
45 }
46
47 #version 330 core
48 out vec4 FragColor;
49
50 uniform sampler2D u_AlbedoMap;
51 uniform sampler2D u_NormalMap;
52 uniform vec3 u_Color = vec3(1.0, 1.0, 1.0);
53
54 in vec2 v_TexCoord;
55 in vec3 v_TangentLightPos[16];
56 in vec3 v_TangentViewPos;
57 in vec3 v_TangentFragPos;
58 flat in int v_LightCount;
59
60 // Light properties
61 uniform vec3 u_LightColors[16];
62 uniform float u_LightIntensities[16];
63
64 // Material properties
65 uniform float u_Glossiness = 5.0f;
66 uniform float u_Metallicness = 5.0f;
67
68 void main()
69 {
70     vec3 textureColor;
71     textureColor = texture(u_AlbedoMap, v_TexCoord).rgb;
72
73     vec3 normal;
74     normal = texture(u_NormalMap, v_TexCoord).rgb;
75
76     float ambientCoefficient = 0.1;
77     normal = normalize(normal);
78     vec3 camDir = normalize(v_TangentViewPos - v_TangentFragPos);
79     vec3 diffuse = vec3(0.0);
80     vec3 specular = vec3(0.0);
81     vec3 ambient = vec3(0.0);
82
83     for (int i = 0; i < v_LightCount; i++)
84     {
85         // diffuse
86         vec3 lightDir = normalize(v_TangentLightPos[i] - v_TangentFragPos);
87         float diff = max(dot(lightDir, normal), 0.0);
88         vec3 D = diff * u_LightColors[i] * u_LightIntensities[i];
89
90         // specular
91         vec3 halfwayDir = normalize(lightDir + camDir);
92         float spec = pow(max(dot(normal, halfwayDir), 0.0), 32.0) * u_Glossiness;
93         vec3 S = u_LightColors[i] * spec * u_LightIntensities[i];
94
95         diffuse += D;
96         specular += S;
97         ambient += u_LightColors[i] * u_LightIntensities[i];
98     }

```

```

99
100     ambient = ambientCoefficient * ambient;
101
102     vec3 BlinnPhong = ambient + diffuse + specular;
103     vec3 final = textureColor.rgb * BlinnPhong;
104     final = pow(final, vec3(1.0 / 1.2));
105
106     FragColor = vec4(final * u_Color, 1.0);
107 }

```

Listing 4.16: Normal Mapping GLSL Shader Code

```

1 struct VertexInput {
2     @location(0) a_Position : vec3<f32>,
3     @location(1) a_Normal: vec3<f32>,
4     @location(2) a_TexCoord: vec2<f32>,
5     @location(3) a_Tangent: vec3<f32>,
6     @location(4) a_Bitangent: vec3<f32>,
7 };
8 struct VertexOutput {
9     @builtin(position) v_Position: vec4<f32>,
10    @location(0) v_TexCoord : vec2<f32>,
11    @location(1) v_TangentViewPos : vec3<f32>,
12    @location(2) v_TangentFragPos : vec3<f32>,
13    @location(3) v_TangentLightPos0 : vec3<f32>,
14    @location(4) v_TangentLightPos1 : vec3<f32>,
15    @location(5) v_TangentLightPos2 : vec3<f32>,
16    @location(6) v_TangentLightPos3 : vec3<f32>,
17 };
18
19 struct UniformData {
20     viewMatrix: mat4x4f,
21     projectionMatrix: mat4x4f,
22     objectColor: vec4f,
23     viewPosition: vec4<f32>,
24     lightPositions: array<vec4f, 4>,
25     lightColors: array<vec4f, 4>,
26     lightIntensities: array<vec4f, 4>,
27     lightCount: f32,
28 };
29
30 struct ModelData {
31     modelMatrix: array<mat4x4f, 512>,
32     inverseModelMatrix: array<mat4x4f, 512>,
33 };
34
35 @group(0) @binding(0) var<uniform> u_UniformData: UniformData;
36 @group(0) @binding(1) var<storage, read> u_ModelData: ModelData;
37 @group(1) @binding(0) var u_AlbedoTexture: texture_2d<f32>;
38 @group(1) @binding(1) var u_AlbedoSampler: sampler;
39 @group(1) @binding(2) var u_NormalMap: texture_2d<f32>;
40 @group(1) @binding(3) var u_NormalSampler: sampler;
41
42 @vertex
43 fn vs_main(@builtin(instance_index) ID: u32, in: VertexInput) -> VertexOutput {
44     let inverseModelMat3x3 = mat3x3f(u_ModelData.inverseModelMatrix[ID][0].xyz, u_ModelData.
45         inverseModelMatrix[ID][1].xyz, u_ModelData.inverseModelMatrix[ID][2].xyz);
46     var normalMatrix: mat3x3f = transpose(inverseModelMat3x3);
47
48     var T: vec3f = normalize(normalMatrix * in.a_Tangent);
49     var B: vec3f = normalize(normalMatrix * in.a_Bitangent);
50     var N: vec3f = normalize(normalMatrix * in.a_Normal);
51
52     // Re-orthogonalize T with respect to N
53     T = normalize(T - dot(T, N) * N);
54
55     var TBN: mat3x3f = transpose(mat3x3f(T, B, N));
56
57     var out: VertexOutput;
58     var mvp: mat4x4f = u_UniformData.projectionMatrix * u_UniformData.viewMatrix * u_ModelData.
59         modelMatrix[ID];
60     out.v_Position = mvp * vec4<f32>(in.a_Position, 1.0);
61     out.v_TexCoord = in.a_TexCoord;
62     out.v_TangentViewPos = TBN * u_UniformData.viewPosition.xyz;
63     out.v_TangentFragPos = TBN * (u_ModelData.modelMatrix[ID] * vec4f(in.a_Position, 1.0)).xyz;
64
65     if (u_UniformData.lightCount == 1) {
66         out.v_TangentLightPos0 = TBN * u_UniformData.lightPositions[0].xyz;
67     } else if (u_UniformData.lightCount == 2) {
68         out.v_TangentLightPos0 = TBN * u_UniformData.lightPositions[0].xyz;
69         out.v_TangentLightPos1 = TBN * u_UniformData.lightPositions[1].xyz;
70     } else if (u_UniformData.lightCount == 3) {
71         out.v_TangentLightPos0 = TBN * u_UniformData.lightPositions[0].xyz;
72     }
73 }

```

```

70     out.v_TangentLightPos1 = TBN * u_UniformData.lightPositions[1].xyz;
71     out.v_TangentLightPos2 = TBN * u_UniformData.lightPositions[2].xyz;
72 } else if (u_UniformData.lightCount == 4) {
73     out.v_TangentLightPos0 = TBN * u_UniformData.lightPositions[0].xyz;
74     out.v_TangentLightPos1 = TBN * u_UniformData.lightPositions[1].xyz;
75     out.v_TangentLightPos2 = TBN * u_UniformData.lightPositions[2].xyz;
76     out.v_TangentLightPos3 = TBN * u_UniformData.lightPositions[3].xyz;
77 }
78
79     return out;
80 }
81
82 @fragment
83 fn fs_main(in: VertexOutput) -> @location(0) vec4<f32> {
84     var textureColor: vec4<f32> = textureSample(u_AlbedoTexture, u_AlbedoSampler, in.v_TexCoord);
85
86     var normal: vec3f = textureSample(u_NormalMap, u_NormalSampler, in.v_TexCoord).rgb;
87
88     normal = normalize(normal);
89     var camDir: vec3<f32> = normalize(in.v_TangentViewPos - in.v_TangentFragPos);
90     var diffuse: vec3<f32> = vec3<f32>(0.0);
91     var specular: vec3<f32> = vec3<f32>(0.0);
92     var ambient: vec3<f32> = vec3<f32>(0.0);
93
94     var ambientCoefficient: f32 = 0.1;
95     var u_Glossiness: f32 = 3.0;
96
97     for (var f: f32 = 0.0; f < u_UniformData.lightCount; f = f + 1.0) {
98         var i: i32 = i32(f);
99         var tangentLightPos: vec3f;
100
101         if (i == 0) {
102             tangentLightPos = in.v_TangentLightPos0;
103         } else if (i == 1) {
104             tangentLightPos = in.v_TangentLightPos1;
105         } else if (i == 2) {
106             tangentLightPos = in.v_TangentLightPos2;
107         } else if (i == 3) {
108             tangentLightPos = in.v_TangentLightPos3;
109         }
110
111         // ambient
112         ambient = ambient + u_UniformData.lightColors[i].rgb * u_UniformData.lightIntensities[i].x;
113
114         // diffuse
115         var lightDir: vec3<f32> = normalize(tangentLightPos - in.v_TangentFragPos);
116         var diff: f32 = max(dot(lightDir, normal), 0.0);
117         var D: vec3<f32> = diff * u_UniformData.lightColors[i].rgb * u_UniformData.lightIntensities[i].x;
118         diffuse = diffuse + D;
119
120         // specular
121         var halfwayDir: vec3<f32> = normalize(lightDir + camDir);
122         var spec: f32 = pow(max(dot(normal, halfwayDir), 0.0), 32.0) * u_Glossiness;
123         var S: vec3<f32> = u_UniformData.lightColors[i].rgb * spec * u_UniformData.lightIntensities[i].x;
124         specular = specular + S;
125     }
126
127     ambient = ambientCoefficient * ambient;
128
129     var BlinnPhong: vec3<f32> = ambient + diffuse + specular;
130     var finalColor: vec3<f32> = textureColor.rgb * BlinnPhong;
131     finalColor = pow(finalColor, vec3<f32>(1.0 / 1.2));
132
133     // gamma correct
134     let physicalColor = pow(finalColor * u_UniformData.objectColor.rgb, vec3<f32>(2.2));
135
136     return vec4f(physicalColor.r, physicalColor.g, physicalColor.b, textureColor.a);
137 }

```

Listing 4.17: Normal Mapping WGSL Shader Code

Objective The main objectives of this example are to: (1) introduce students to the concept and importance of normal mapping in computer graphics, (2) explain how normal maps store normal vector information in textures, (3) demonstrate the process of applying normal maps to a 3D model to improve lighting accuracy,

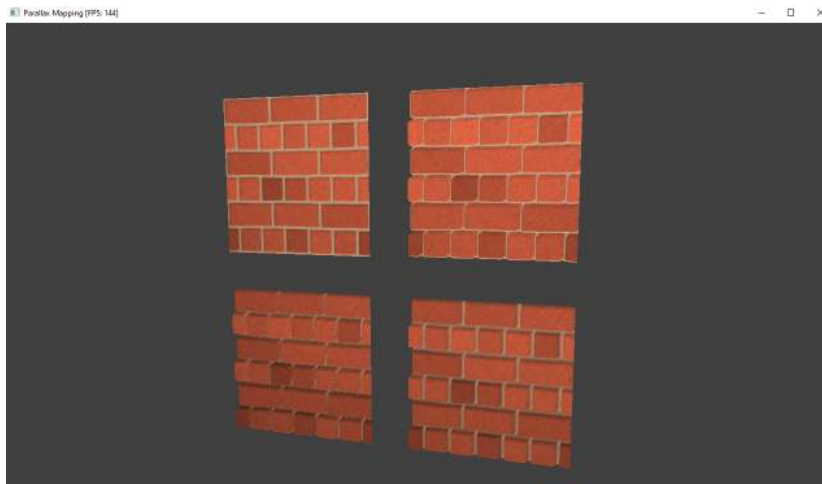


Figure 4.16: Showcase of Different Parallax Mapping Techniques.

(4) show the visual differences between per-vertex normals and normal mapping.

4.4.4 Parallax Mapping

In this example, implemented in both APIs, students will explore parallax mapping, a technique similar to normal mapping but based on different principles. Parallax mapping significantly enhances the detail and depth of a textured surface. While it is still an illusion, parallax mapping more effectively conveys a sense of depth by displacing vertices, especially when combined with normal mapping. This example will allow students to visually compare different techniques of parallax mapping, such as regular, steep parallax, and parallax occlusion culling, and understand how they differ from normal mapping. (see Figure 4.16 and Listings 4.18, 4.19).

```

1  #version 330 core
2  layout(location = 0) in vec3 a_Position;
3  layout(location = 1) in vec3 a_Normal;
4  layout(location = 2) in vec2 a_TexCoord;
5  layout(location = 3) in vec3 a_Tangent;
6  layout(location = 4) in vec3 a_Bitangent;
7
8  uniform mat4 u_ModelViewProjection;
9  uniform mat4 u_Model;
10 uniform vec3 u_LightPositions[16];
11 uniform int u_LightCount = 0;
12 uniform vec3 u_ViewPosition = vec3(0.0, 0.0, 10.0);
13
14 out vec2 v_TexCoord;
15 out vec3 v_TangentLightPos[16];
16 out vec3 v_TangentViewPos;
17 out vec3 v_TangentFragPos;
18 flat out int v_LightCount;
19
20 void main()
21 {
22     vec3 T = normalize(mat3(u_Model) * a_Tangent);
23     vec3 B = normalize(mat3(u_Model) * a_Bitangent);
24     vec3 N = normalize(mat3(u_Model) * a_Normal);
25
26     mat3 TBN = transpose(mat3(T, B, N));

```

```

27     v_TexCoord = a_TexCoord;
28
29
30     for (int i = 0; i < u_LightCount; i++)
31     {
32         v_TangentLightPos[i] = TBN * u_LightPositions[i];
33     }
34
35     v_TangentViewPos = TBN * u_ViewPosition;
36     v_TangentFragPos = TBN * (u_Model * vec4(a_Position, 1.0)).xyz;
37
38     v_LightCount = u_LightCount;
39
40     gl_Position = u_ModelViewProjection * vec4(a_Position, 1.0);
41 }
42
43 #version 330 core
44 out vec4 FragColor;
45
46 uniform sampler2D u_AlbedoMap;
47 uniform sampler2D u_NormalMap;
48 uniform sampler2D u_DisplacementMap;
49
50 uniform vec3 u_Color = vec3(1.0, 1.0, 1.0);
51
52 in vec2 v_TexCoord;
53 in vec3 v_TangentLightPos[16];
54 in vec3 v_TangentViewPos;
55 in vec3 v_TangentFragPos;
56 flat in int v_LightCount;
57
58 // Light properties
59 uniform vec3 u_LightColors[16];
60 uniform float u_LightIntensities[16];
61
62 // Material properties
63 uniform float u_Glossiness = 5.0f;
64 uniform float u_Metallicness = 5.0f;
65
66 float heightScale = 0.1;
67
68 vec2 ParallaxMapping(vec2 texCoords, vec3 viewDir)
69 {
70     float height = texture(u_DisplacementMap, texCoords).r;
71     return texCoords - viewDir.xy * (height * heightScale);
72 }
73
74 void main()
75 {
76     vec3 camDir = normalize(v_TangentViewPos - v_TangentFragPos);
77     vec2 offsetTexCoord = ParallaxMapping(v_TexCoord, camDir);
78
79     if (offsetTexCoord.x > 1.0 || offsetTexCoord.y > 1.0 || offsetTexCoord.x < 0.0 || offsetTexCoord.y
80         < 0.0)
81         discard;
82
83     vec3 textureColor;
84     textureColor = texture(u_AlbedoMap, offsetTexCoord).rgb;
85
86     vec3 normal;
87     normal = texture(u_NormalMap, offsetTexCoord).rgb;
88
89     float ambientCoefficient = 0.1;
90     normal = normalize(normal);
91     vec3 diffuse = vec3(0.0);
92     vec3 specular = vec3(0.0);
93     vec3 ambient = vec3(0.0);
94
95     for (int i = 0; i < v_LightCount; i++)
96     {
97         // diffuse
98         vec3 lightDir = normalize(v_TangentLightPos[i] - v_TangentFragPos);
99         float diff = max(dot(lightDir, normal), 0.0);
100         vec3 D = diff * u_LightColors[i] * u_LightIntensities[i];
101
102         // specular
103         vec3 halfwayDir = normalize(lightDir + camDir);
104         float spec = pow(max(dot(normal, halfwayDir), 0.0), 32.0) * u_Glossiness;
105         vec3 S = u_LightColors[i] * spec * u_LightIntensities[i];
106
107         diffuse += D;
108         specular += S;
109         ambient += u_LightColors[i] * u_LightIntensities[i];
110     }

```

```

110
111     ambient = ambientCoefficient * ambient;
112
113     vec3 BlinnPhong = ambient + diffuse + specular;
114     vec3 final = textureColor.rgb * BlinnPhong;
115     final = pow(final, vec3(1.0 / 1.2));
116
117     FragColor = vec4(final * u_Color, 1.0);
118 }

```

Listing 4.18: Parallax Mapping GLSL Shader Code

```

1 struct VertexInput {
2     @location(0) a_Position : vec3<f32>,
3     @location(1) a_Normal : vec3<f32>,
4     @location(2) a_TexCoord : vec2<f32>,
5     @location(3) a_Tangent : vec3<f32>,
6     @location(4) a_Bitangent : vec3<f32>,
7 };
8 struct VertexOutput {
9     @builtin(position) v_Position : vec4<f32>,
10    @location(0) v_TexCoord : vec2<f32>,
11    @location(1) v_TangentViewPos : vec3<f32>,
12    @location(2) v_TangentFragPos : vec3<f32>,
13    @location(3) v_TangentLightPos0 : vec3<f32>,
14    @location(4) v_TangentLightPos1 : vec3<f32>,
15    @location(5) v_TangentLightPos2 : vec3<f32>,
16    @location(6) v_TangentLightPos3 : vec3<f32>,
17 };
18
19 struct UniformData {
20     viewMatrix : mat4x4f,
21     projectionMatrix : mat4x4f,
22     objectColor : vec4f,
23     viewPosition : vec4<f32>,
24     lightPositions : array<vec4f, 4>,
25     lightColors : array<vec4f, 4>,
26     lightIntensities : array<vec4f, 4>,
27     lightCount : f32,
28 };
29
30 struct ModelData {
31     modelMatrix : array<mat4x4f, 512>,
32 };
33
34 @group(0) @binding(0) var<uniform> u_UniformData : UniformData;
35 @group(0) @binding(1) var<storage, read> u_ModelData : ModelData;
36 @group(1) @binding(0) var u_AlbedoTexture : texture_2d<f32>;
37 @group(1) @binding(1) var u_AlbedoSampler : sampler;
38 @group(1) @binding(2) var u_NormalMap : texture_2d<f32>;
39 @group(1) @binding(3) var u_NormalSampler : sampler;
40 @group(1) @binding(4) var u_DisplacementMap : texture_2d<f32>;
41 @group(1) @binding(5) var u_DisplacementSampler : sampler;
42
43 @vertex
44 fn vs_main(@builtin(instance_index) ID : u32, in : VertexInput) -> VertexOutput {
45     let modelMat3x3 = mat3x3f(u_ModelData.modelMatrix[ID][0].xyz, u_ModelData.modelMatrix[ID][1].xyz,
46     u_ModelData.modelMatrix[ID][2].xyz);
47     var normalMatrix : mat3x3f = transpose(modelMat3x3);
48
49     var T : vec3f = normalize(normalMatrix * in.a_Tangent);
50     var B : vec3f = normalize(normalMatrix * in.a_Bitangent);
51     var N : vec3f = normalize(normalMatrix * in.a_Normal);
52
53     // Re-orthogonalize T with respect to N
54     T = normalize(T - dot(T, N) * N);
55
56     var TBN : mat3x3f = transpose(mat3x3f(T, B, N));
57
58     var out : VertexOutput;
59     var mvp : mat4x4f = u_UniformData.projectionMatrix * u_UniformData.viewMatrix * u_ModelData.
60     modelMatrix[ID];
61     out.v_Position = mvp * vec4<f32>(in.a_Position, 1.0);
62     out.v_TexCoord = in.a_TexCoord;
63     out.v_TangentViewPos = TBN * u_UniformData.viewPosition.xyz;
64     out.v_TangentFragPos = TBN * (u_ModelData.modelMatrix[ID] * vec4f(in.a_Position, 1.0)).xyz;
65
66     if (u_UniformData.lightCount == 1) {
67         out.v_TangentLightPos0 = TBN * u_UniformData.lightPositions[0].xyz;
68     } else if (u_UniformData.lightCount == 2) {
69         out.v_TangentLightPos0 = TBN * u_UniformData.lightPositions[0].xyz;
70         out.v_TangentLightPos1 = TBN * u_UniformData.lightPositions[1].xyz;
71     } else if (u_UniformData.lightCount == 3) {

```

```

70     out.v_TangentLightPos0 = TBN * u_UniformData.lightPositions[0].xyz;
71     out.v_TangentLightPos1 = TBN * u_UniformData.lightPositions[1].xyz;
72     out.v_TangentLightPos2 = TBN * u_UniformData.lightPositions[2].xyz;
73 } else if (u_UniformData.lightCount == 4) {
74     out.v_TangentLightPos0 = TBN * u_UniformData.lightPositions[0].xyz;
75     out.v_TangentLightPos1 = TBN * u_UniformData.lightPositions[1].xyz;
76     out.v_TangentLightPos2 = TBN * u_UniformData.lightPositions[2].xyz;
77     out.v_TangentLightPos3 = TBN * u_UniformData.lightPositions[3].xyz;
78 }
79
80     return out;
81 }
82
83 const heightScale: f32 = 0.1;
84
85 fn ParallaxMapping(texCoords: vec2f, viewDir: vec3f) -> vec2f
86 {
87     let height: f32 = textureSample(u_DisplacementMap, u_DisplacementSampler, texCoords).r;
88     return texCoords - viewDir.xy * (height * heightScale);
89 }
90
91 @fragment
92 fn fs_main(in: VertexOutput) -> @location(0) vec4<f32> {
93     var camDir: vec3<f32> = normalize(in.v_TangentViewPos - in.v_TangentFragPos);
94     var offsetTexCoord: vec2f = ParallaxMapping(in.v_TexCoord, camDir);
95
96     if (offsetTexCoord.x > 1.0 || offsetTexCoord.y > 1.0 || offsetTexCoord.x < 0.0 || offsetTexCoord.
97         y < 0.0) {
98         discard;
99     }
100
101     var textureColor: vec4<f32> = textureSample(u_AlbedoTexture, u_AlbedoSampler, offsetTexCoord);
102
103     var normal: vec3f = textureSample(u_NormalMap, u_NormalSampler, offsetTexCoord).rgb;
104
105     normal = normalize(normal);
106     var diffuse: vec3<f32> = vec3<f32>(0.0);
107     var specular: vec3<f32> = vec3<f32>(0.0);
108     var ambient: vec3<f32> = vec3<f32>(0.0);
109
110     var ambientCoefficient: f32 = 0.1;
111     var u_Glossiness: f32 = 3.0;
112
113     for (var f: f32 = 0.0; f < u_UniformData.lightCount; f = f + 1.0) {
114         var i: i32 = i32(f);
115         var tangentLightPos: vec3f;
116
117         if (i == 0) {
118             tangentLightPos = in.v_TangentLightPos0;
119         } else if (i == 1) {
120             tangentLightPos = in.v_TangentLightPos1;
121         } else if (i == 2) {
122             tangentLightPos = in.v_TangentLightPos2;
123         } else if (i == 3) {
124             tangentLightPos = in.v_TangentLightPos3;
125         }
126
127         // ambient
128         ambient = ambient + u_UniformData.lightColors[i].rgb * u_UniformData.lightIntensities[i].x;
129
130         // diffuse
131         var lightDir: vec3<f32> = normalize(tangentLightPos - in.v_TangentFragPos);
132         var diff: f32 = max(dot(lightDir, normal), 0.0);
133         var D: vec3<f32> = diff * u_UniformData.lightColors[i].rgb * u_UniformData.lightIntensities[i].x;
134         diffuse = diffuse + D;
135
136         // specular
137         var halfwayDir: vec3<f32> = normalize(lightDir + camDir);
138         var spec: f32 = pow(max(dot(normal, halfwayDir), 0.0), 32.0) * u_Glossiness;
139         var S: vec3<f32> = u_UniformData.lightColors[i].rgb * spec * u_UniformData.lightIntensities[i].x;
140         specular = specular + S;
141     }
142
143     ambient = ambientCoefficient * ambient;
144
145     var BlinnPhong: vec3<f32> = ambient + diffuse + specular;
146     var finalColor: vec3<f32> = textureColor.rgb * BlinnPhong;
147     finalColor = pow(finalColor, vec3<f32>(1.0 / 1.2));
148
149     // gamma correct
150     let physicalColor = pow(finalColor * u_UniformData.objectColor.rgb, vec3<f32>(2.2));

```

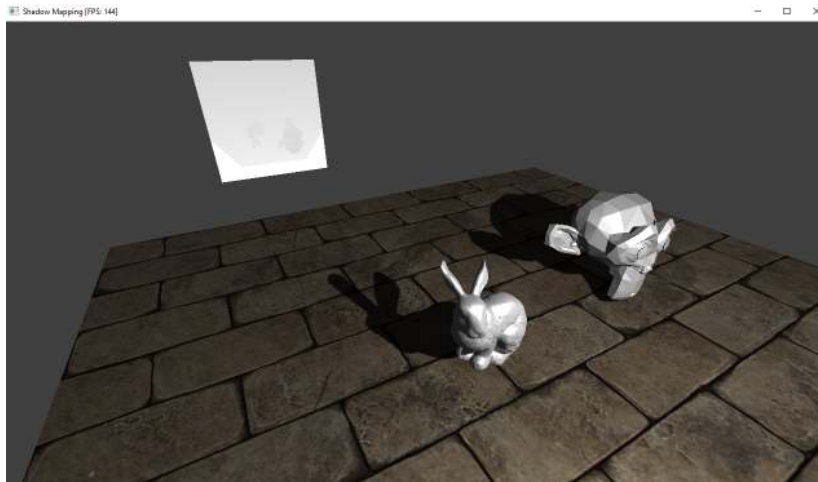


Figure 4.17: Rendering of a Scene Using Shadow Mapping. Also, in the Upper Left, the Depth Only Texture that was Produced During the First Depth Only Pre-Pass is Rendered.

```

151     return vec4f(physicalColor.r, physicalColor.g, physicalColor.b, textureColor.a);
152 }

```

Listing 4.19: Parallax Mapping WGSL Shader Code

Objective The main objectives of this example are to: (1) introduce students to the concept and principles of parallax mapping, (2) demonstrate how parallax mapping enhances surface detail and depth, (3) compare different techniques of parallax mapping: regular, steep parallax, and parallax occlusion culling, (4) show the differences, advantages and disadvantages of parallax mapping over normal mapping.

4.4.5 Shadow Mapping

In this example, implemented in OpenGL and work in progress in WebGPU, students will learn about shadow mapping, a fundamental technique for achieving shadows in real-time graphics. The concept behind shadow mapping is straightforward: render the scene from the light's perspective to determine which areas are lit and which are in shadow. This information is stored in a depth-only texture. Shadow mapping consists of two passes: first, rendering the depth map, and second, rendering the scene normally while using the generated depth map to determine whether fragments are in shadow. (see Figure 4.17 and Listing 4.20).

```

1 #version 330 core
2 layout(location = 0) in vec3 a_Position;
3 layout(location = 1) in vec3 a_Normal;
4 layout(location = 2) in vec2 a_TexCoord;
5

```

```

6 uniform mat4 u_ModelViewProjection;
7 uniform mat4 u_Model;
8
9 uniform mat4 u_LightSpaceMatrix;
10
11 out vec3 v_Position;
12 out vec3 v_Normal;
13 out vec2 v_TexCoord;
14 out vec4 v_FragPosLightSpace;
15
16 void main()
17 {
18     v_Position = (u_Model * vec4(a_Position, 1.0)).xyz;
19     v_Normal = (transpose(inverse(u_Model)) * vec4(a_Normal, 0.0)).xyz;
20     v_TexCoord = a_TexCoord;
21     v_FragPosLightSpace = u_LightSpaceMatrix * vec4(v_Position, 1.0);
22
23     gl_Position = u_ModelViewProjection * vec4(a_Position, 1.0);
24 }
25
26 #version 330 core
27 out vec4 FragColor;
28
29 uniform sampler2D u_AlbedoMap;
30 uniform sampler2D u_ShadowMap;
31 uniform vec3 u_Color = vec3(1.0, 1.0, 1.0);
32
33 in vec3 v_Position;
34 in vec3 v_Normal;
35 in vec2 v_TexCoord;
36 in vec4 v_FragPosLightSpace;
37
38 // Light properties
39 uniform int u_LightCount = 0;
40 uniform vec3 u_LightPositions[16];
41 uniform vec3 u_LightColors[16];
42 uniform float u_LightIntensities[16];
43
44 // Material properties
45 uniform float u_Glossiness = 5.0;
46 uniform float u_Metallicess = 5.0;
47
48 // Camera properties
49 uniform vec3 u_ViewPosition = vec3(0.0, 0.0, 10.0);
50
51 float ShadowCalculation(vec4 fragPosLightSpace)
52 {
53     // perform perspective divide
54     vec3 projCoords = fragPosLightSpace.xyz / fragPosLightSpace.w;
55     // transform to [0,1] range
56     projCoords = projCoords * 0.5 + 0.5;
57     // get closest depth value from light's perspective (using [0,1] range fragPosLight as coords)
58     float closestDepth = texture(u_ShadowMap, projCoords.xy).r;
59     // get depth of current fragment from light's perspective
60     float currentDepth = projCoords.z;
61
62     // calculate bias (based on depth map resolution and slope)
63     vec3 normal = normalize(v_Normal);
64     vec3 lightDir = normalize(u_LightPositions[u_LightCount - 1] - v_Position);
65     float bias = max(0.005 * (1.0 - dot(normal, lightDir)), 0.001);
66
67     // PCF
68     float shadow = 0.0;
69     vec2 texelSize = 1.0 / textureSize(u_ShadowMap, 0);
70     for(int x = -1; x <= 1; ++x)
71     {
72         for(int y = -1; y <= 1; ++y)
73         {
74             float pcfDepth = texture(u_ShadowMap, projCoords.xy + vec2(x, y) * texelSize).r;
75             shadow += currentDepth - bias > pcfDepth ? 1.0 : 0.0;
76         }
77     }
78     shadow /= 9.0;
79
80     if (projCoords.z > 1.0)
81         shadow = 0.0;
82
83     return shadow;
84 }
85
86 void main()
87 {
88     vec3 textureColor;
89

```

```

90 textureColor = texture(u_AlbedoMap, v_TexCoord).rgb;
91
92 float ambientCoefficient = 0.1;
93 vec3 normal = normalize(v_Normal);
94 vec3 camDir = normalize(u_ViewPosition - v_Position);
95 vec3 diffuse = vec3(0.0);
96 vec3 specular = vec3(0.0);
97 vec3 ambient = vec3(0.0);
98
99 for (int i = 0; i < u_LightCount; i++)
100 {
101     // diffuse
102     vec3 lightDir = normalize(u_LightPositions[i] - v_Position);
103     float diff = max(dot(lightDir, normal), 0.0);
104     vec3 D = diff * u_LightColors[i] * u_LightIntensities[i];
105
106     // specular
107     vec3 halfwayDir = normalize(lightDir + camDir);
108     float spec = pow(max(dot(normal, halfwayDir), 0.0), 32.0) * u_Glossiness;
109     vec3 S = u_LightColors[i] * spec * u_LightIntensities[i];
110
111     diffuse += D;
112     specular += S;
113     ambient += u_LightColors[i] * u_LightIntensities[i];
114 }
115
116 ambient = ambientCoefficient * ambient;
117
118 float shadow = ShadowCalculation(v_FragPosLightSpace);
119
120 vec3 BlinnPhong = (ambient + (1.0 - shadow) * (diffuse + specular));
121 vec3 final = textureColor.rgb * BlinnPhong;
122 final = pow(final, vec3(1.0 / 1.2));
123
124 FragColor = vec4(final * u_Color, 1.0);
125 }

```

Listing 4.20: Shadow Mapping GLSL Shader Code

Objective The main objectives of this example are to: (1) introduce students to the principles and workflow of shadow mapping, (2) demonstrate how to render a scene from the light’s perspective to create a depth map, (3) show how to use the depth map in a second rendering pass to determine and apply shadows, (4) explain common issues and solutions related to shadow mapping, such as shadow acne and peter panning.

4.4.6 Physical Based Rendering - Sphere

In this example, implemented in both APIs, students will be introduced to physically based rendering (PBR) [23], an advanced shading technique that produces more realistic lighting. PBR is a collection of rendering techniques that closely match the physical world’s behavior of light. By aiming to mimic light in a physically plausible way, PBR generally looks more realistic compared to traditional lighting algorithms like Phong and Blinn-Phong. Although PBR is an approximation of reality, it is based on principles of physics, thus achieving more believable results.

For a PBR lighting model to be considered physically based, it must satisfy the following conditions: (1) be based on the microfacet surface model, (2) be energy conserving, (3) use a physically based bidirectional reflective distribution function (BRDF).

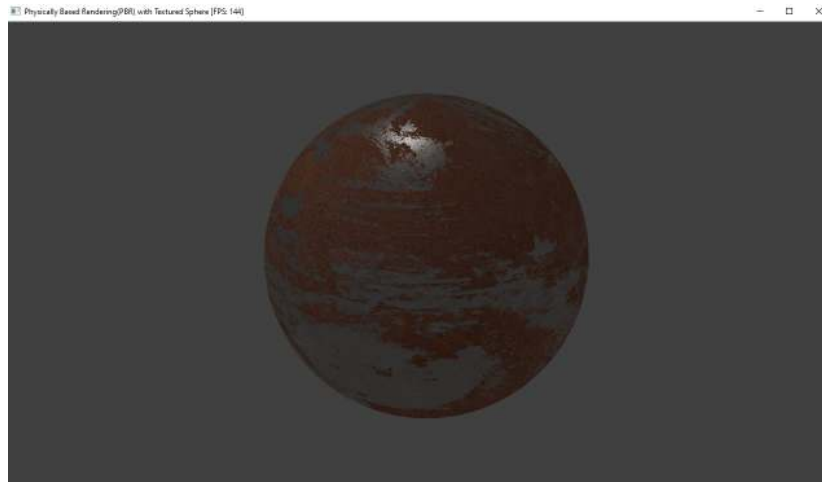


Figure 4.18: Rendering of a Rusted Sphere Using Physical Based Rendering.

In this example, students will learn how to render a sphere with a rusted appearance using PBR techniques. (see Figure 4.18 and Listings 4.21, 4.22).

```

1  #version 330 core
2  layout(location = 0) in vec3 a_Position;
3  layout(location = 1) in vec3 a_Normal;
4  layout(location = 2) in vec2 a_TexCoord;
5
6  uniform mat4 u_ModelViewProjection;
7  uniform mat4 u_Model;
8
9  out vec3 v_Position;
10 out vec3 v_Normal;
11 out vec2 v_TexCoord;
12
13 void main()
14 {
15     v_Position = (u_Model * vec4(a_Position, 1.0)).xyz;
16     v_Normal = (transpose(inverse(u_Model)) * vec4(a_Normal, 0.0)).xyz;
17     v_TexCoord = a_TexCoord;
18
19     gl_Position = u_ModelViewProjection * vec4(a_Position, 1.0);
20 }
21
22 #version 330 core
23 out vec4 FragColor;
24
25 uniform sampler2D u_AlbedoMap;
26 uniform sampler2D u_NormalMap;
27 uniform sampler2D u_MetallicMap;
28 uniform sampler2D u_RoughnessMap;
29
30 uniform vec3 u_Color = vec3(1.0, 1.0, 1.0);
31
32 in vec3 v_Position;
33 in vec3 v_Normal;
34 in vec2 v_TexCoord;
35
36 // Light properties
37 uniform int u_LightCount = 0;
38 uniform float u_LightIntensities[16];
39 uniform vec3 u_LightPositions[16];
40 uniform vec3 u_LightColors[16];
41
42 // Camera properties
43 uniform vec3 u_ViewPosition = vec3(0.0, 0.0, 10.0);
44
45 const float PI = 3.14159265359;
46
47 // Easy trick to get tangent-normals to world-space to keep PBR code simplified.

```

```

48 vec3 getNormalFromMap()
49 {
50     vec3 tangentNormal = texture(u_NormalMap, v_TexCoord).xyz * 2.0 - 1.0;
51
52     vec3 Q1 = dFdx(v_Position);
53     vec3 Q2 = dFdy(v_Position);
54     vec2 st1 = dFdx(v_TexCoord);
55     vec2 st2 = dFdy(v_TexCoord);
56
57     vec3 N = normalize(v_Normal);
58     vec3 T = normalize(Q1*st2.t - Q2*st1.t);
59     vec3 B = -normalize(cross(N, T));
60     mat3 TBN = mat3(T, B, N);
61
62     return normalize(TBN * tangentNormal);
63 }
64
65 float DistributionGGX(vec3 N, vec3 H, float roughness)
66 {
67     float a = roughness*roughness;
68     float a2 = a*a;
69     float NdotH = max(dot(N, H), 0.0);
70     float NdotH2 = NdotH*NdotH;
71
72     float nom = a2;
73     float denom = (NdotH2 * (a2 - 1.0) + 1.0);
74     denom = PI * denom * denom;
75
76     return nom / denom;
77 }
78
79 float GeometrySchlickGGX(float NdotV, float roughness)
80 {
81     float r = (roughness + 1.0);
82     float k = (r*r) / 8.0;
83
84     float nom = NdotV;
85     float denom = NdotV * (1.0 - k) + k;
86
87     return nom / denom;
88 }
89
90 float GeometrySmith(vec3 N, vec3 V, vec3 L, float roughness)
91 {
92     float NdotV = max(dot(N, V), 0.0);
93     float NdotL = max(dot(N, L), 0.0);
94     float ggx2 = GeometrySchlickGGX(NdotV, roughness);
95     float ggx1 = GeometrySchlickGGX(NdotL, roughness);
96
97     return ggx1 * ggx2;
98 }
99
100 vec3 fresnelSchlick(float cosTheta, vec3 F0)
101 {
102     return F0 + (1.0 - F0) * pow(clamp(1.0 - cosTheta, 0.0, 1.0), 5.0);
103 }
104
105 void main()
106 {
107     vec3 albedo = pow(texture(u_AlbedoMap, v_TexCoord).rgb, vec3(2.2));
108     float metallic = texture(u_MetallicMap, v_TexCoord).r;
109     float roughness = texture(u_RoughnessMap, v_TexCoord).r;
110
111     vec3 N = getNormalFromMap();
112     vec3 V = normalize(u_ViewPosition - v_Position);
113
114     // calculate reflectance at normal incidence; if dia-electric (like plastic) use F0
115     // of 0.04 and if it's a metal, use the albedo color as F0 (metallic workflow)
116     vec3 F0 = vec3(0.04);
117     F0 = mix(F0, albedo, metallic);
118
119     // reflectance equation
120     vec3 Lo = vec3(0.0);
121     for(int i = 0; i < u_LightCount; ++i)
122     {
123         // calculate per-light radiance
124         vec3 L = normalize(u_LightPositions[i] - v_Position);
125         vec3 H = normalize(V + L);
126         float distance = length(u_LightPositions[i] - v_Position);
127         float attenuation = 1.0 / (distance * distance);
128         vec3 radiance = u_LightColors[i] * attenuation * u_LightIntensities[i];
129
130         // Cook-Torrance BRDF
131         float NDF = DistributionGGX(N, H, roughness);

```

```

132 float G = GeometrySmith(N, V, L, roughness);
133 vec3 F = fresnelSchlick(max(dot(H, V), 0.0), F0);
134
135 vec3 numerator = NDF * G * F;
136 float denominator = 4.0 * max(dot(N, V), 0.0) * max(dot(N, L), 0.0) + 0.0001; // + 0.0001 to
prevent divide by zero
vec3 specular = numerator / denominator;
137
138 // kS is equal to Fresnel
139 vec3 kS = F;
140 // for energy conservation, the diffuse and specular light can't
141 // be above 1.0 (unless the surface emits light); to preserve this
142 // relationship the diffuse component (kD) should equal 1.0 - kS.
143 vec3 kD = vec3(1.0) - kS;
144 // multiply kD by the inverse metalness such that only non-metals
145 // have diffuse lighting, or a linear blend if partly metal (pure metals
146 // have no diffuse light).
147 kD *= 1.0 - metallic;
148
149 // scale light by NdotL
150 float NdotL = max(dot(N, L), 0.0);
151
152 // add to outgoing radiance Lo
153 Lo += (kD * albedo / PI + specular) * radiance * NdotL; // note that we already multiplied
the BRDF by the Fresnel (kS) so we won't multiply by kS again
154 }
155
156 // ambient lighting (note that the next IBL tutorial will replace
157 // this ambient lighting with environment lighting).
158 vec3 ambient = vec3(0.05) * albedo;
159
160 vec3 color = ambient + Lo;
161
162 // HDR tonemapping
163 color = color / (color + vec3(1.0));
164 // gamma correct
165 color = pow(color, vec3(1.0 / 2.2));
166
167 FragColor = vec4(color * u_Color, 1.0);
168 }
169

```

Listing 4.21: PBR GLSL Shader Code

```

1 struct VertexInput {
2     @location(0) a_Position : vec3<f32>,
3     @location(1) a_Normal : vec3<f32>,
4     @location(2) a_TexCoord: vec2<f32>,
5 };
6 struct VertexOutput {
7     @builtin(position) v_Position: vec4<f32>,
8     @location(0) v_Normal : vec3<f32>,
9     @location(1) v_TexCoord : vec2<f32>,
10    @location(2) v_CurrentPosition : vec3<f32>,
11 };
12
13 struct UniformData {
14     viewMatrix: mat4x4f,
15     projectionMatrix: mat4x4f,
16     objectColor: vec4f,
17     viewPosition: vec4<f32>,
18     lightPositions: array<vec4f, 16>,
19     lightColors: array<vec4f, 16>,
20     lightIntensities: array<vec4f, 16>,
21     lightCount: f32,
22 };
23
24 struct ModelData {
25     modelMatrix: array<mat4x4f, 512>,
26     inverseModelMatrix: array<mat4x4f, 512>,
27 };
28
29 @group(0) @binding(0) var<uniform> u_UniformData: UniformData;
30 @group(0) @binding(1) var<storage, read> u_ModelData: ModelData;
31 @group(1) @binding(0) var u_AlbedoMap: texture_2d<f32>;
32 @group(1) @binding(1) var u_AlbedoSampler: sampler;
33 @group(1) @binding(2) var u_NormalMap: texture_2d<f32>;
34 @group(1) @binding(3) var u_NormalSampler: sampler;
35 @group(1) @binding(4) var u_MetallicMap: texture_2d<f32>;
36 @group(1) @binding(5) var u_MetallicSampler: sampler;
37 @group(1) @binding(6) var u_RoughnessMap: texture_2d<f32>;
38 @group(1) @binding(7) var u_RoughnessSampler: sampler;
39
40 @vertex

```

```

41 fn vs_main(@builtin(instance_index) ID: u32, in: VertexInput) -> VertexOutput {
42     var out: VertexOutput;
43     var mvp: mat4x4f = u_UniformData.projectionMatrix * u_UniformData.viewMatrix * u_ModelData.
        modelMatrix[ID];
44     out.v_Position = mvp * vec4<f32>(in.a_Position, 1.0);
45     out.v_CurrentPosition = (u_ModelData.modelMatrix[ID] * vec4f(in.a_Position, 1.0)).xyz;
46     out.v_Normal = (transpose(u_ModelData.inverseModelMatrix[ID]) * vec4f(in.a_Normal, 0.0)).xyz;
47     out.v_TexCoord = in.a_TexCoord;
48     return out;
49 }
50
51 const PI = 3.14159265359;
52
53 // Easy trick to get tangent-normals to world-space to keep PBR code simplified.
54 fn getNormalFromMap(currentPosition: vec3f, texCoord: vec2f, normal: vec3f) -> vec3f
55 {
56     var tangentNormal: vec3f = textureSample(u_NormalMap, u_NormalSampler, texCoord).xyz * 2.0 - 1.0;
57
58     var Q1: vec3f = dpdx(currentPosition);
59     var Q2: vec3f = dpdy(currentPosition);
60     var st1: vec2f = dpdx(texCoord);
61     var st2: vec2f = dpdy(texCoord);
62
63     var N: vec3f = normalize(normal);
64     var T: vec3f = normalize(Q1 * st2.y - Q2 * st1.y);
65     var B: vec3f = -normalize(cross(N, T));
66     var TBN: mat3x3f = mat3x3f(T, B, N);
67
68     return normalize(TBN * tangentNormal);
69 }
70
71 fn DistributionGGX(N: vec3f, H: vec3f, roughness: f32) -> f32
72 {
73     var a: f32 = roughness * roughness;
74     var a2: f32 = a * a;
75     var NdotH: f32 = max(dot(N, H), 0.0);
76     var NdotH2: f32 = NdotH * NdotH;
77
78     var nom: f32 = a2;
79     var denom: f32 = (NdotH2 * (a2 - 1.0) + 1.0);
80     denom = PI * denom * denom;
81
82     return nom / denom;
83 }
84
85 fn GeometrySchlickGGX(NdotV: f32, roughness: f32) -> f32
86 {
87     var r: f32 = (roughness + 1.0);
88     var k: f32 = (r * r) / 8.0;
89
90     var nom: f32 = NdotV;
91     var denom: f32 = NdotV * (1.0 - k) + k;
92
93     return nom / denom;
94 }
95
96 fn GeometrySmith(N: vec3f, V: vec3f, L: vec3f, roughness: f32) -> f32
97 {
98     var NdotV: f32 = max(dot(N, V), 0.0);
99     var NdotL: f32 = max(dot(N, L), 0.0);
100     var ggx2: f32 = GeometrySchlickGGX(NdotV, roughness);
101     var ggx1: f32 = GeometrySchlickGGX(NdotL, roughness);
102
103     return ggx1 * ggx2;
104 }
105
106 fn fresnelSchlick(cosTheta: f32, F0: vec3f) -> vec3f
107 {
108     return F0 + (1.0 - F0) * pow(clamp(1.0 - cosTheta, 0.0, 1.0), 5.0);
109 }
110
111 @fragment
112 fn fs_main(in: VertexOutput) -> @location(0) vec4<f32> {
113     var albedo: vec3f = pow(textureSample(u_AlbedoMap, u_AlbedoSampler, in.v_TexCoord).rgb, vec3
        (2.2));
114     var metallic: f32 = textureSample(u_MetallicMap, u_MetallicSampler, in.v_TexCoord).r;
115     var roughness: f32 = textureSample(u_RoughnessMap, u_RoughnessSampler, in.v_TexCoord).r;
116
117     var N: vec3f = getNormalFromMap(in.v_CurrentPosition, in.v_TexCoord, in.v_Normal);
118     var V: vec3f = normalize(u_UniformData.viewPosition.xyz - in.v_CurrentPosition);
119
120     // calculate reflectance at normal incidence; if dia-electric (like plastic) use F0
121     // of 0.04 and if it's a metal, use the albedo color as F0 (metallic workflow)
122     var F0: vec3f = vec3f(0.04);

```

```

123 F0 = mix(F0, albedo, metallic);
124
125 // reflectance equation
126 var Lo: vec3f = vec3f(0.0);
127 for (var f: f32 = 0.0; f < u_UniformData.lightCount; f = f + 1.0) {
128     var i: i32 = i32(f);
129
130     // calculate per-light radiance
131     var L: vec3f = normalize(u_UniformData.lightPositions[i].xyz - in.v_CurrentPosition);
132     var H: vec3f = normalize(V + L);
133     var distance: f32 = length(u_UniformData.lightPositions[i].xyz - in.v_CurrentPosition);
134     var attenuation: f32 = 1.0 / (distance * distance);
135     var radiance: vec3f = u_UniformData.lightColors[i].rgb * attenuation * u_UniformData.
lightIntensities[i].x;
136
137     // Cook-Torrance BRDF
138     var NDF: f32 = DistributionGGX(N, H, roughness);
139     var G: f32 = GeometrySmith(N, V, L, roughness);
140     var F: vec3f = fresnelSchlick(max(dot(H, V), 0.0), F0);
141
142     var numerator: vec3f = NDF * G * F;
143     var denominator: f32 = 4.0 * max(dot(N, V), 0.0) * max(dot(N, L), 0.0) + 0.0001; // + 0.0001
to prevent divide by zero
144     var specular: vec3f = numerator / denominator;
145
146     // kS is equal to Fresnel
147     var kS: vec3f = F;
148     // for energy conservation, the diffuse and specular light can't
149     // be above 1.0 (unless the surface emits light); to preserve this
150     // relationship the diffuse component (kD) should equal 1.0 - kS.
151     var kD: vec3f = vec3(1.0) - kS;
152     // multiply kD by the inverse metalness such that only non-metals
153     // have diffuse lighting, or a linear blend if partly metal (pure metals
154     // have no diffuse light).
155     kD *= 1.0 - metallic;
156
157     // scale light by NdotL
158     var NdotL: f32 = max(dot(N, L), 0.0);
159
160     // add to outgoing radiance Lo
161     Lo += (kD * albedo / PI + specular) * radiance * NdotL; // note that we already multiplied
the BRDF by the Fresnel (kS) so we won't multiply by kS again
162 }
163
164 // ambient lighting (note that the next IBL tutorial will replace
165 // this ambient lighting with environment lighting).
166 var ambient: vec3f = vec3(0.05) * albedo;
167
168 var color: vec3f = ambient + Lo;
169
170 // HDR tonemapping
171 color = color / (color + vec3f(1.0));
172 // gamma correct
173 color = pow(color, vec3f(1.0 / 1.5));
174
175 return vec4f(color * u_UniformData.objectColor.xyz, 1.0);
176 }

```

Listing 4.22: PBR WGSL Shader Code

Objective The main objectives of this example are to: (1) introduce the principles and theory behind physically based rendering, (2) demonstrate the micro-facet surface model and its application in rendering, (3) show how to implement energy-conserving lighting models, (4) explain the use of physically based BRDFs in shading.

4.4.7 Physical Based Rendering - Model

In this example, implemented in both APIs, students will expand upon the previous physically based rendering (PBR) [23] tutorial. While the prior example focused on rendering a sphere, this one will delve into the process of rendering a



Figure 4.19: Rendering of a 3D Model Using Physical Based Rendering.

more complex 3D model using PBR techniques. This example aims to show the necessary workflow and materials required to achieve realistic results with PBR in more intricate scenarios. (see Figure 4.19 and Listings 4.21, 4.22).

Objective The main objectives of this example are to: (1) extend knowledge from basic PBR concepts to more complex 3D models, (2) demonstrate how to prepare and use various material textures in PBR workflows, such as albedo, normal, roughness, and metallic maps, (3) showcase the step-by-step process of setting up a 3D model for PBR rendering, (4) help students understand how to achieve realistic results with PBR by carefully managing materials and lighting.

4.4.8 Advanced Compute

In this example, implemented in the WebGPU API due to lack of cross platform support for them in OpenGL, students will expand on their knowledge of compute shaders by exploring a more advanced real-world application. They will learn how to use textures in compute shaders to apply filters and manage different settings via a uniform buffer to produce varied results. Additionally, students will familiarize themselves with the concept of asynchronous operations on the GPU, as compute shaders are dispatched and invoked asynchronously to prevent blocking the main thread while waiting for results from the GPU (see Figure 4.20 and Listing 4.23).

```

1 struct Uniforms {
2     reset: f32,
3 }
4
5 @group(0) @binding(0) var inputTexture: texture_2d<f32>;
6 @group(0) @binding(1) var outputTexture: texture_storage_2d<rgba8unorm, write>;
7 @group(0) @binding(2) var<uniform> u_Uniforms: Uniforms;
8
9 @compute @workgroup_size(8, 8, 1)
10 fn computeSobelX(@builtin(global_invocation_id) id: vec3<u32>) {

```



Figure 4.20: Applying Filters to Images Using Compute Shaders.

```

11  var color = vec3f(1.0);
12
13  if (u_Uniforms.reset == 0.0)
14  {
15      color = abs(
16          1 * textureLoad(inputTexture, vec2<u32>(id.x - 1, id.y - 1), 0).rgb
17          + 2 * textureLoad(inputTexture, vec2<u32>(id.x - 1, id.y + 0), 0).rgb
18          + 1 * textureLoad(inputTexture, vec2<u32>(id.x - 1, id.y + 1), 0).rgb
19          - 1 * textureLoad(inputTexture, vec2<u32>(id.x + 1, id.y - 1), 0).rgb
20          - 2 * textureLoad(inputTexture, vec2<u32>(id.x + 1, id.y + 0), 0).rgb
21          - 1 * textureLoad(inputTexture, vec2<u32>(id.x + 1, id.y + 1), 0).rgb
22      );
23  }
24
25  textureStore(outputTexture, id.xy, vec4f(color, 1.0));
26 }

```

Listing 4.23: Advanced Compute WGSL Shader Code

Objective The main objectives of this example are to: (1) extend basic compute shader knowledge to more complex, real-world applications, (2) demonstrate how to use textures in compute shaders for applying image filters, (3) introduce asynchronous GPU operations to prevent blocking the main thread, (4) provide practical experience in handling advanced compute shader tasks and optimizing performance.

4.5 Summary

Most of the examples are implemented in both OpenGL and WebGPU. This dual implementation provides valuable insights into the differences between modern low-level APIs and more high-level legacy ones. By comparing the two, students will:

- Gain a deeper understanding of the trade-offs between different API designs.

- Learn about performance considerations and why certain features in modern APIs lead to better performance.
- Understand the detailed control offered by low-level APIs and how it contrasts with the abstractions in legacy APIs.

The educational examples provided in the framework are designed to gradually build students' expertise in computer graphics. By progressing from introductory to advanced topics and implementing examples in both OpenGL and WebGPU, students gain a comprehensive understanding of graphics programming. This structured approach not only equips them with practical skills but also deepens their theoretical knowledge, enabling them to appreciate the nuances and performance implications of different API designs. This educational journey, from basic to advanced topics, ensures that students are well-prepared to tackle real-world graphics programming challenges and adapt to evolving technologies in the field.

All code from the examples described in this section are available in the github repository link: <https://github.com/papagiannakis/pyGandalf>

Chapter 5

Evaluation

In this chapter, we present the evaluation methods conducted alongside their results, accompanied by a brief discussion and analysis. The evaluation is divided into two main parts: the performance evaluation of the framework and the educational evaluation. We will explore both aspects in detail in the following sections.

5.1 Performance evaluation

The performance evaluation focuses on assessing the efficiency, scalability, and robustness of the pyGANDALF framework with both the editor attached and detached. This involves measuring various metrics such as frame rates and memory usage of CPU and GPU on various use cases.

5.1.1 Experimental Setup

We evaluated the performance of the pyGANDALF framework on a PC running Windows 10. The computer was equipped with an Intel Core i9 9900K Processor (8 cores/16 threads, 12MB Cache, 4.9 GHz max boost), an NVIDIA GeForce RTX 2080 GPU with 8GB GDDR4 Video RAM, and 16GB of DDR4 RAM at 3600MHz.

We designed a series of benchmark scenarios that represent common use cases in graphics applications, including simple rendering tasks, complex scenes with multiple objects, and dynamic interactions. In particular, we evaluated the performance of the framework in these scenarios:

- Scene 1: A simple scene that consists of a model and a skybox.
- Scene 2: A more complex scene that consists of 10 models and a skybox.
- Scene 3: A much more complex scene that consists of 50 models and a skybox.
- Scene 4: The same scene setup as Scene 3, but all the 50 model are operated by a system that rotates them around every frame.

- Scene 5: A much more complex scene that consists of 100 models and a skybox, also all the 100 model are operated by a system that rotates them around every frame.

The following evaluation setup was used to evaluate both graphics API implementations (OpenGL and WebGPU) and when the editor layer is attached (only available for OpenGL).

5.1.2 Results & Discussion

5.1.2.1 Frames Per Second

In Table 5.1, we can see the results of the performance evaluation regarding frames per second (FPS). Let's first focus on the OpenGL performance, comparing scenarios with and without the editor layer attached.

We can observe that when the editor layer is attached, there is a 20% - 25% reduction in frames per second (FPS). This reduction is expected because the editor utilizes both the GPU and CPU. The GPU is used to render the user interface through the Dear ImGui Bundle package, over which we have limited control regarding performance optimization. Simultaneously, the CPU executes various systems of the editor to maintain its functionality and purpose. Consequently, the performance decrease is justifiable and does not significantly impair the framework's performance, allowing it to continue operating smoothly and in real time. Even in the most demanding scene, with 100 objects—a scenario unlikely to be required in an educational environment—the editor maintains a frame rate above 60 FPS on our mid-range machine.

Pivoting away from the editor layer, let's examine the performance differences between OpenGL and WebGPU. In the simplest scene, Scene 1, we see that the OpenGL renderer outperforms the WebGPU renderer. This discrepancy is likely due to driver and OS restrictions beyond our control, given the high frame rates achieved. However, when observing the results for the more complex scenes, we can see that the WebGPU renderer consistently outperforms its OpenGL counterpart and scales much better with an increasing number of rendered objects.

This result can be attributed to the low-level nature of WebGPU, which allowed us to fine-tune the renderer with minimal effort and without complex abstractions that could detract from the educational focus of the framework. Additionally, the WebGPU renderer is set up to utilize instanced drawing, significantly contributing to the performance gains when rendering multiple instances of the same object. This efficiency demonstrates the advantages of WebGPU in handling more demanding scenes while maintaining higher performance, making it an excellent choice for educational purposes.

Scene	OpenGL	WebGPU	OpenGL + Editor
Scene 1	1200 fps	850 fps	910 fps
Scene 2	450 fps	815 fps	350 fps
Scene 3	105 fps	545 fps	85 fps
Scene 4	103 fps	541 fps	82 fps
Scene 5	72 fps	355 fps	61 fps

Table 5.1: Frames Per Second (Higher is Better).

5.1.2.2 Memory Usage

CPU Memory Usage In Table 5.2, we can see the results of the CPU memory usage for the different test cases. The OpenGL implementation shows minimal difference in CPU memory usage whether the editor layer is attached or detached, with consumption being almost identical in both scenarios. On the other hand, the WebGPU implementation consumes significantly more CPU memory, about 200MB to 300MB more, due to the instanced drawing technique employed. This technique allocates fixed-sized CPU-side buffers to accommodate the maximum number of instances that can be rendered, leading to higher memory usage.

Scene	OpenGL	WebGPU	OpenGL + Editor
Scene 1	185MB	385MB	185MB
Scene 2	200MB	400MB	185MB
Scene 3	195MB	480MB	210MB
Scene 4	180MB	460MB	200MB
Scene 5	180MB	535MB	200MB

Table 5.2: A comparison of CPU Memory Usage between OpenGL, OpenGL + Editor and WebGPU (Lower is Better).

GPU Memory Usage In Table 5.3, a pattern similar to CPU memory usage emerges. Specifically, both the OpenGL implementation—whether with the editor layer attached or detached—shows negligible difference in GPU usage. On the other hand, the WebGPU implementation exhibits significantly higher GPU memory consumption, about 200MB more. This can be attributed to the necessity of allocating large GPU-side buffers to support instanced drawing efficiently across numerous instances.

Scene	OpenGL	WebGPU	OpenGL + Editor
Scene 1	180MB	200MB	200MB
Scene 2	200MB	400MB	200MB
Scene 3	400MB	600MB	400MB
Scene 4	400MB	600MB	400MB
Scene 5	600MB	800MB	600MB

Table 5.3: A comparison of GPU Memory Usage between OpenGL, OpenGL + Editor and WebGPU (Lower is Better).

5.1.3 Summary

In the performance evaluation of the framework, we observe that pyGANDALF performs adequately across a variety of test scenes, from simple, lightweight scenarios to those involving the rendering of hundreds of objects with attached behaviors, maintaining a frame rate above 60 fps. The editor layer proves to be a lightweight addition, reducing the frame rate by only 20% to 25% without significantly impacting memory consumption, while providing a powerful and easy-to-use environment for creating and editing scenes.

Moreover, the performance comparison between the two supported graphics APIs, OpenGL and WebGPU, reveals that WebGPU implementations consistently outperform their OpenGL counterparts. This is primarily due to the low-level nature of WebGPU, which allows for fine-tuning performance-critical details, and the use of instance drawing techniques that greatly enhance rendering performance. However, this increased performance in frames per second comes at the cost of higher memory consumption in both the CPU and GPU. Despite this, the trade-off between memory usage and performance favors WebGPU, as the performance gains are significant and justify the additional memory usage.

5.2 Educational evaluation

The educational evaluation aims to assess the effectiveness of the pyGANDALF framework as a teaching tool for computer graphics concepts. This involves gathering feedback from domain experts who used the framework and are capable of providing feedback for the educational impact of the framework.

5.2.1 Experimental Setup

An expert-based evaluation [26] [27] was conducted to assess the effectiveness of the pyGANDALF framework. Seven (7) computer graphics experts participated in the evaluation that are also former students of the Computer Graphics CS358 and Interactive Computer Graphics CS553 courses of the Computer Science Department, University of Crete, Greece. The evaluators were divided into two groups:

(1) the first group used the pyGANDALF framework to implement a scenario in both OpenGL and WebGPU, (2) the second group used Python and the OpenGL and WebGPU APIs without any additional framework. The evaluators were tasked with implementing a typical scenario that would be assigned in a classroom setting. This scenario aimed to assess the framework’s ability to enhance student success and learning outcomes [28]. The evaluation was guided by a set of questions that the evaluators answered at each step of the implementation scenario. In particular, the evaluation of the pyGANDALF framework consisted of the following questions [29] [30]:

1. How clearly the tools indicate the availability of functionalities?
2. How intuitively the tools guide you to achieve the desired effects?
3. How effectively the tools help you understand which API feature corresponds to the desired functionality?
4. How well the tools help you associate API features with their desired outcomes?
5. Rate the sufficiency of feedback when a functionality has been performed.
6. How mentally demanding was the task?
7. How successful were you in accomplishing what you were asked to do?
8. How hard did you have to work to accomplish your level of performance?
9. How discouraged, irritated, stressed, and annoyed were you?

Each question was graded on a scale from 1 to 5, where 1 indicated a very small chance of success and 5 indicated a very good chance of success. The scenario was the following: “The concept involves creating a house composed of a cube serving as the base and a pyramid as the roof. The house will rotate around its y-axis in each frame. The cube will feature a brick texture, while the pyramid will have an orange color. Additionally, a perspective camera should be established to properly view the house.”

Before each evaluation session the evaluator was introduced by the facilitator to the aims and objectives of the pyGANDALF framework as well as who is the target audience. Furthermore, a collection of fundamental examples were given to both groups that showcased core concepts and functionalities to better guide and help evaluators become acquainted with the task at hand.

To give the evaluation a better structure and a more straightforward way of evaluating the effectiveness of the framework, the scenario that was given to the evaluators was separated into 5 discrete steps / tasks:

- Task 1: Attributes - Definition of vertex data and layout

- Task 2: Textures - Load and use textures
- Task 3: Shaders - Load, compile and use shaders
- Task 4: Camera - Set up camera and projection
- Task 5: Uniforms - Add interactivity to the scenario

The evaluators conducted the expert-based evaluation in separate sessions, during which their implementation times were recorded. The resulting code and a video of the final output were submitted to the facilitator for further examination and analysis.

5.2.2 Results & Discussion

To gain a better understanding and overview of the evaluation results, we will first examine the overall results and then we will take a look at the per step results. In both analysis, we will examine quantitative outcomes, presenting various statistics that were gathered, while also analyzing the results qualitatively, which will provide deeper insights given the small but expert-based nature of the evaluation.

5.2.2.1 Overall Results

Quantitative In Figure 5.1, we can see the overall results of answers per question as percentages for the group that used the pyGANDALF framework. Each question had a scale from 1 to 5 for the evaluators to rate their experience. The first five questions assessed the intuitiveness and clarity of the pyGANDALF framework, with 95% of the answers in the 4 and 5 scales, indicating that the framework excelled in these areas. Questions 6 to 9 focused on the effort required to implement the scenario and any negative feelings experienced by the evaluators. Here, 95% of the answers fell between the 1 and 2 scales, suggesting that the effort required and negative feelings were very low, and the experience was smooth and free from frustrations.

In Figure 5.2, we can see the overall results of answers per question as percentages for the group that used no framework. For the first five questions, which focus on the intuitiveness and clarity of the tools and resources provided, about 25% of respondents indicated a lack of clarification and intuitiveness, around 55% fell in the middle, feeling that the intuitiveness and clarification were neither good nor bad, and the remaining 20% thought it was good enough. For questions 6 to 9, which address the effort required to implement the scenario and any negative feelings experienced during the implementation, the majority of evaluators reported experiencing negative emotions and felt they had to work hard to achieve the required results.

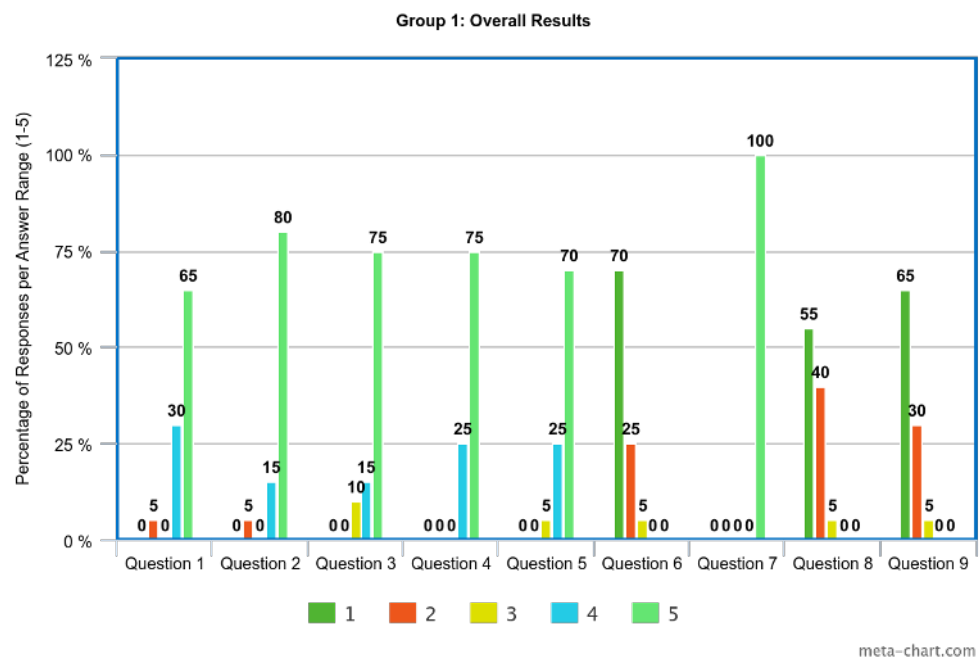


Figure 5.1: Overall Results for Group 1 that used pyGANDAL, showing the percentage of responses (y-axis) in the 1-5 range per question (x-axis).

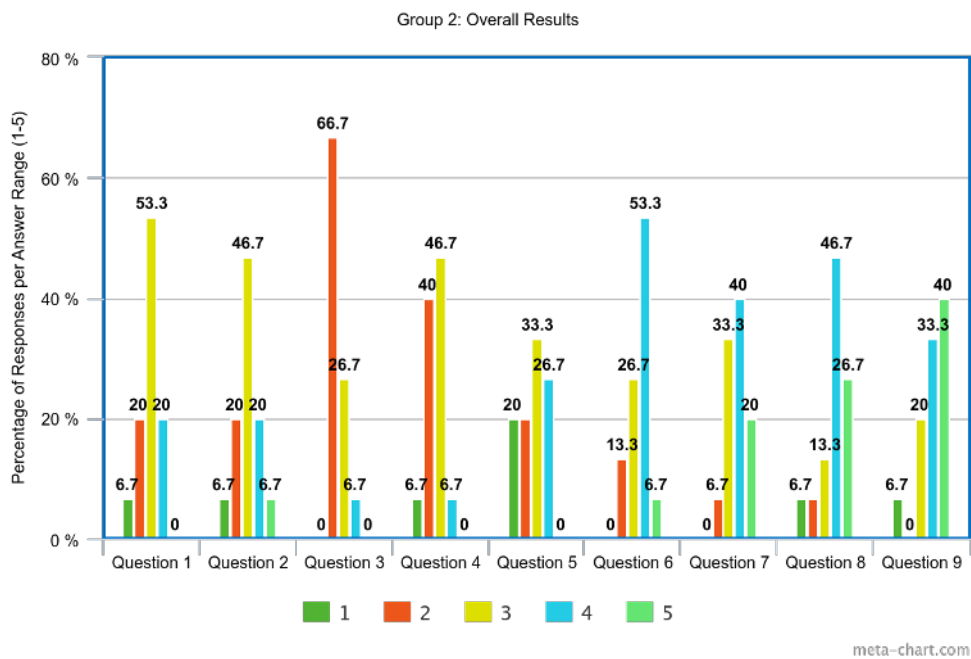


Figure 5.2: Overall Results for Group 2 that used no framework, showing the percentage of responses (y-axis) in the 1-5 range per question (x-axis).

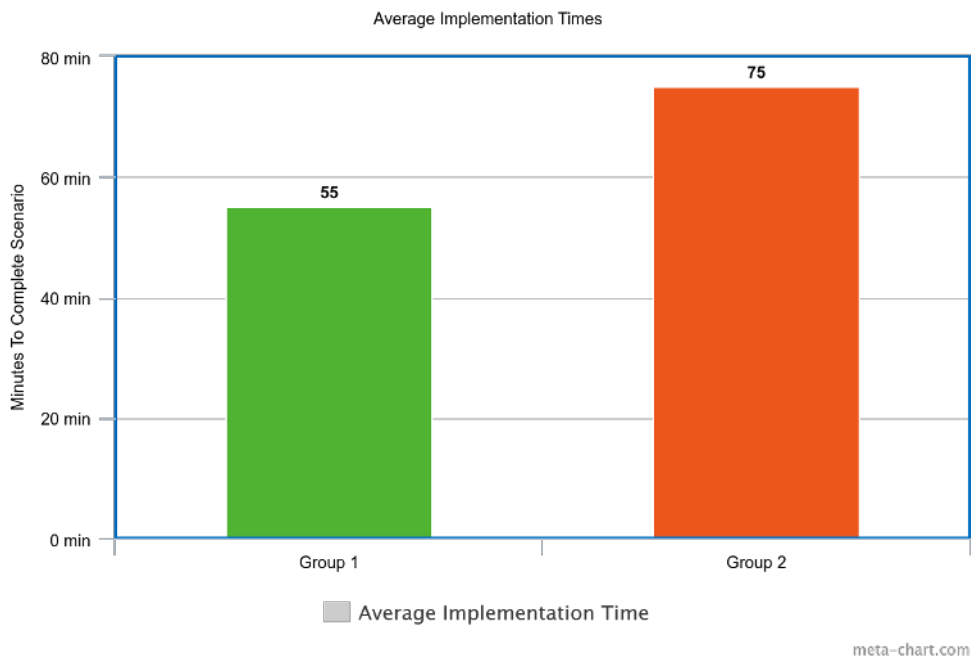


Figure 5.3: Average Implementation Times for Both Groups.

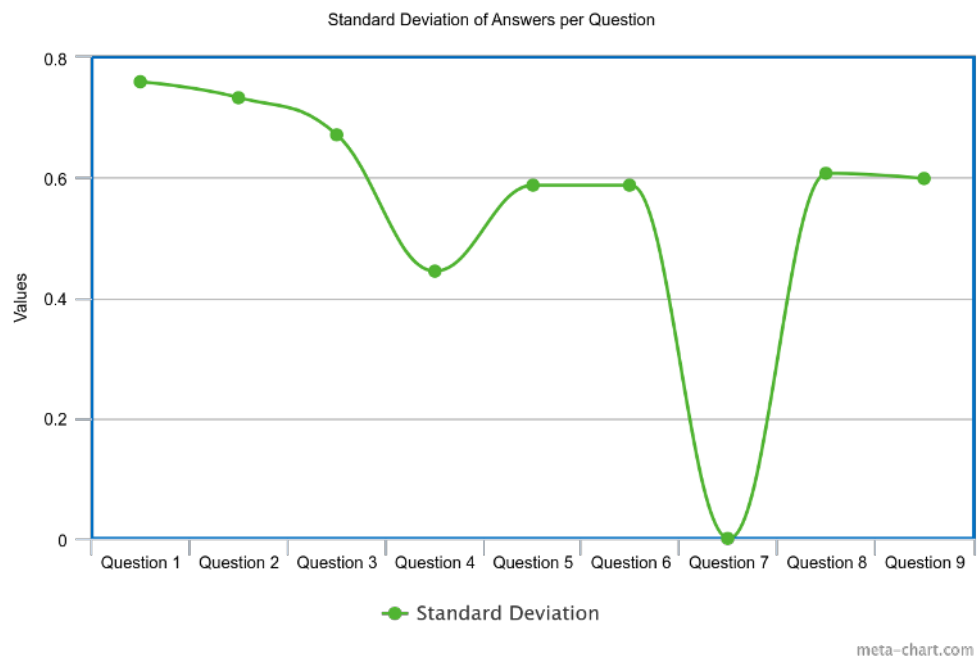


Figure 5.4: Standard Deviation Results for Group 1 that used pyGANDALF.

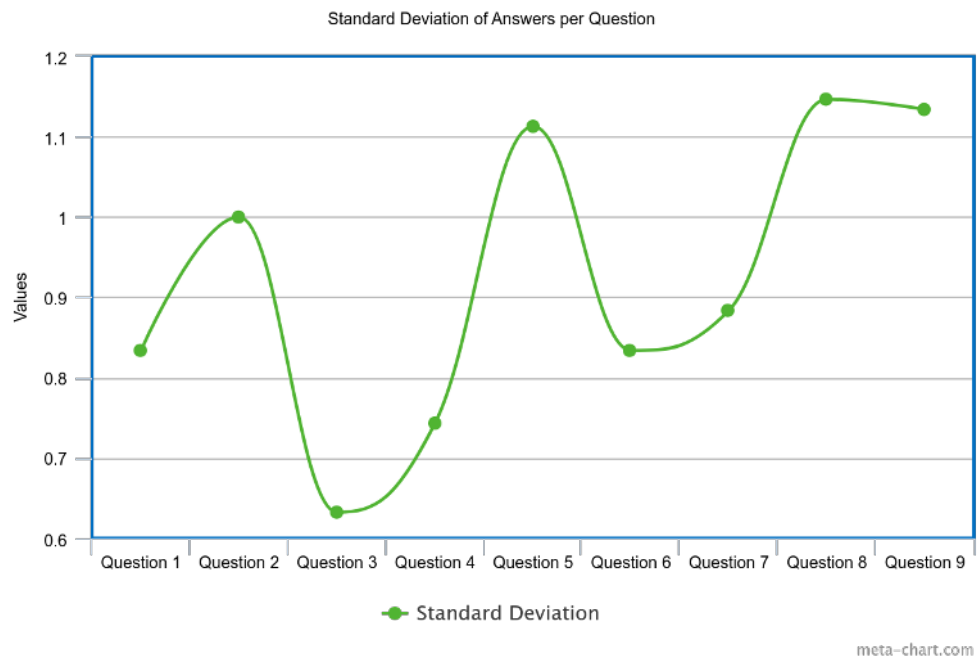


Figure 5.5: Standard Deviation Results for Group 2 that used pyGANDALF.

Qualitative In a broader view, we can observe that the evaluators found the pyGANDALF framework to be significantly more intuitive and clearer compared to using no framework. This suggests that in an educational environment, pyGANDALF would enhance productivity and understanding, as students would not need to struggle with the framework itself but could instead focus on the tasks at hand without distractions and technical difficulties. Furthermore, evaluators reported that when using pyGANDALF, they did not have to work as hard to accomplish their goals and experienced minimal negative feelings, indicating a positive user experience. This enables students to tackle problems and assignments that would otherwise be quite complex and time-consuming without the framework.

Additionally, Figures 5.4 and 5.5 illustrate the standard deviation of responses for the two groups. We can observe that the standard deviation is lower when using the pyGANDALF framework, suggesting strong agreement among the evaluators regarding their ratings. This further supports the consistency and reliability of the positive feedback for pyGANDALF.

In Figure 5.3, we can see the average implementation times for the two groups. The first group, which used pyGANDALF, took an average of 55 minutes to complete the scenario, while the second group, which did not use the framework, took an average of 75 minutes. This reinforces the earlier results regarding the effort required, indicating that the evaluators felt they employed less effort and experienced fewer frustrations when using pyGANDALF.

5.2.2.2 Per Task Results

Quantitative & Qualitative In Figure 5.6, we can see the average response per task on a scale of 1 to 5. We observe that all tasks score an average of 4 out of 5 and above on questions regarding intuitiveness and clarity, while scoring 2 out of 5 and below on questions about effort required and frustration experienced. This consistency across tasks indicates that the framework excels in providing a clear and intuitive experience without significant shortcomings in any key areas when developing a computer graphics application.

Upon closer inspection, we see that the tasks “Shaders - Load, compile and use shaders” and “Textures - Create and use textures” have near-perfect scores. This reassures our commitment to offering a straightforward and intuitive way to handle resources within the framework, while maintaining an educational approach that doesn’t abstract too much away from the student.

On the other hand, in Figure 5.8, we see that the group using no framework scored on average 3.5 out of 5 and below on intuitiveness and clarity, and 2.5 out of 5 and above on effort required and negative feelings experienced. These results are significantly below the levels achieved by the pyGANDALF framework, highlighting its superiority in providing a user-friendly and efficient educational environment.

After a closer look at the per task standard deviation in responses revealed by Figure 5.7, we observe that the task “Uniforms - Add interactivity to the scenario”

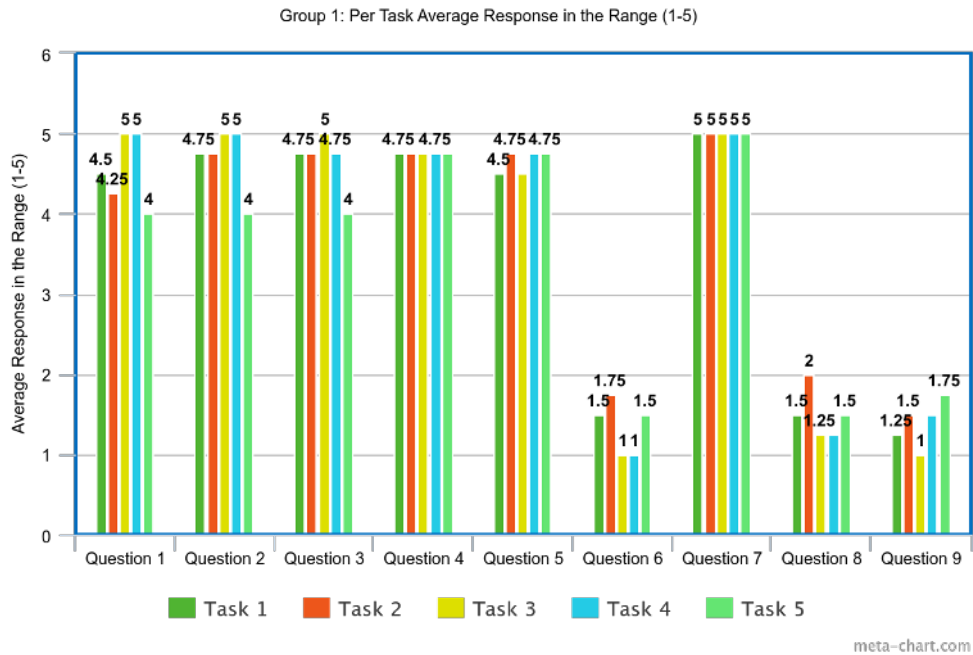


Figure 5.6: Group 1: Per Task Average Response in the Range (1-5).

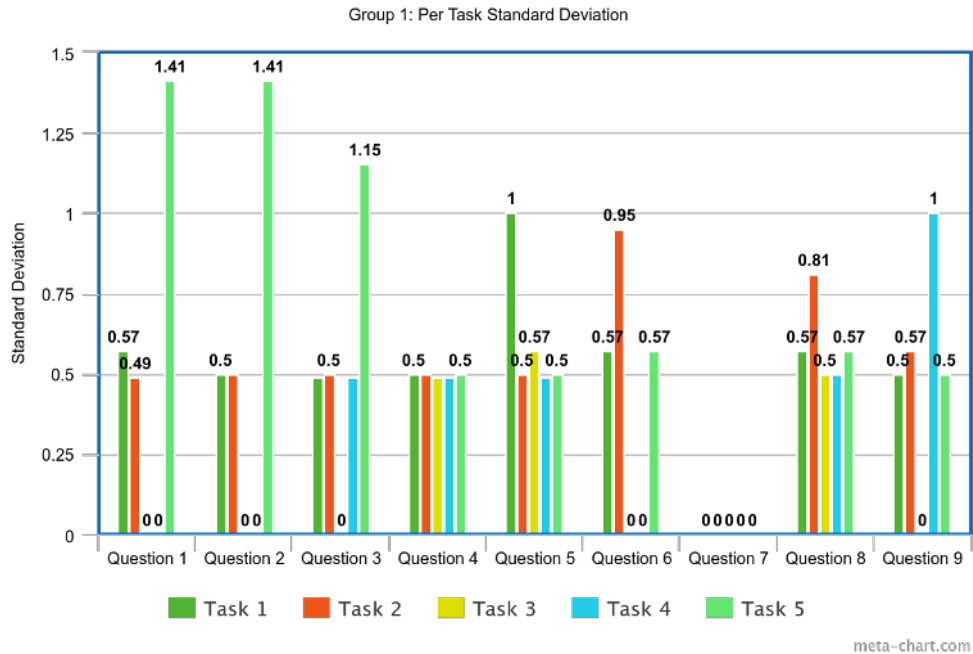


Figure 5.7: Group 1: Per Task Standard Deviation.

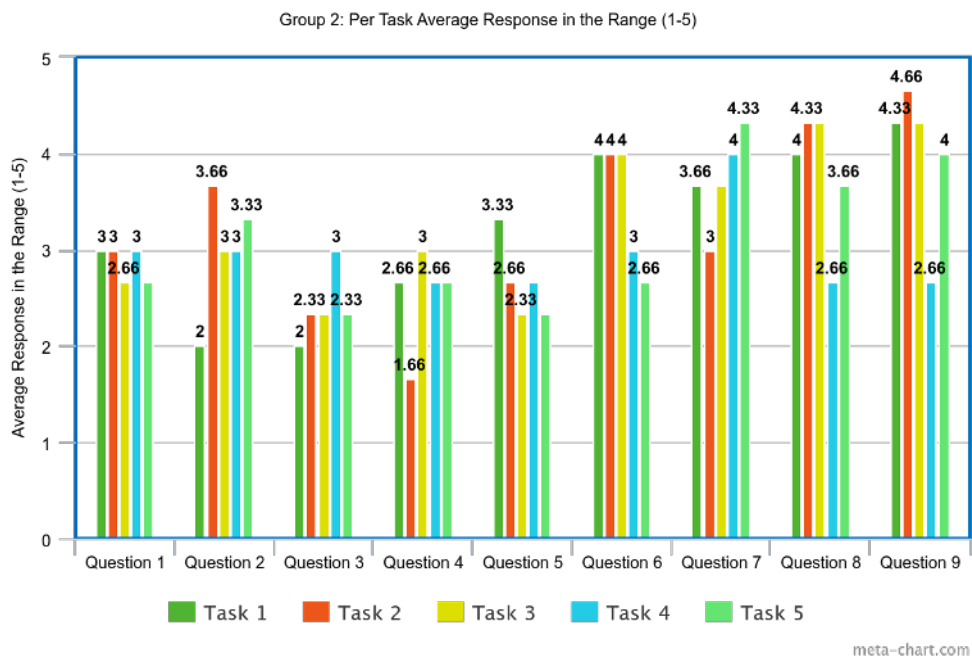


Figure 5.8: Group 2: Per Task Average Response in the Range (1-5).

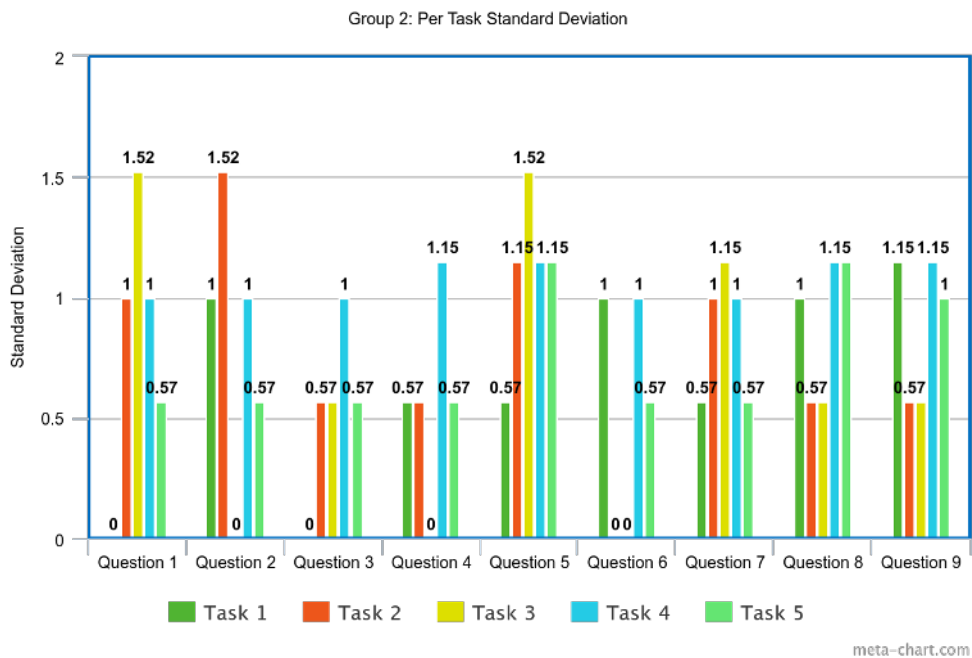


Figure 5.9: Group 2: Per Task Standard Deviation.

has the highest deviation in responses. This suggests that there could be room for improvement in clarity, although the task still received strong overall ratings.

This variability in responses could be attributed to the use of the less widely adopted programming paradigm of entity component systems (ECS). The concept that data is encapsulated in components while systems provide behavior and functionality might be unfamiliar to many users. However, this presents an opportunity to introduce students to this new programming paradigm and expand their expertise and knowledge in software development.

5.2.3 Summary

The framework has performed quite well in the evaluation. The tools are rated highly for clarity and intuitiveness, suggesting that they are easy to understand and use. The effort required is low, and users report minimal negative feelings, indicating a positive user experience. The low standard deviations suggest that there is strong agreement among the respondents about these ratings.

Across all tasks of the scenario, the framework is perceived as highly intuitive, requiring low effort, and causing minimal negative feelings. The high ratings indicate strong satisfaction and usability in all major areas of graphics programming. This detailed breakdown shows that the framework's tools are effective and user-friendly across different aspects, with some minor areas for improvement in clarity for specific steps.

Chapter 6

Conclusion

In this chapter, we will recap the thesis goals, contributions, and novelties in the field of computer graphics education. We will also examine the limitations of our work alongside the future work and improvements that can be implemented.

6.1 Summary

The primary goal of this thesis was to develop a robust and versatile educational framework for computer graphics, specifically targeting the needs of students and educators.

The framework aimed to address gaps in existing educational tools by consolidating all the pioneering ideas identified in the related work research into a single framework, creating a comprehensive solution for teaching computer graphics while also integrating some new ideas into educational frameworks such as Physically Based Rendering (PBR).

The framework incorporates support for the modern low-level API, WebGPU, to enhance students' understanding of GPU hardware. This exposure helps students grasp low-level details and concepts often hidden by legacy high-level APIs. Additionally, maintaining support for OpenGL facilitates easier transitions and comparisons between the two APIs.

One of the main goals of the framework was to ensure simplicity and accessibility. This was achieved by choosing Python as the programming language, avoiding the complexities of low-level languages that could hinder the learning process.

The framework employs an Entity Component System (ECS) architecture, which introduces students to a new programming paradigm, Data Oriented Programming (DOP). This paradigm promotes separation of data and functionality, while utilizing useful, performance-focused programming practices and a modular approach to organizing code, as opposed to traditional Object-Oriented Programming (OOP). The ECS architecture also enabled the integration of a flexible editor layer, providing a fast and user-friendly UI for creating and editing scenes. This feature is designed to empower both students and educators.

Leveraging these features, the framework includes a wide range of educational examples implemented in both APIs. These examples cover fundamental concepts as well as advanced and state-of-the-art graphics techniques and features, while also enabling students to compare the differences in both performance and implementation between the two APIs (OpenGL and WebGPU).

6.2 Comparison of Educational Frameworks

In this section, we will compare the features provided by other state-of-the-art educational frameworks with those offered by our framework, pyGANDALF.

In Table 6.1, you can see the comparison between different features supported by other frameworks and by pyGANDALF.

	ShaderLab Framework	Vulkan all the way	Elements	pyGANDALF
Editor Layer Support	Yes	No	No	Yes
Low level Graphics API Support	No	Yes	No	Yes
Accessible High Level Language Support	No	No	Yes	Yes
Entity Component System Architecture	No	No	Yes	Yes

Table 6.1: Comparison of different educational frameworks and their provided features.

6.3 Limitations & Future Work

6.3.1 Renderer

While the renderer in both APIs performs adequately, as we mentioned before the OpenGL renderer does not support instanced drawing, which would help improve the performance when rendering multiple entities. In the future, we aim to add support for this feature to enhance even more the capabilities of the framework.

6.3.2 Editor Layer

While the editor layer feature provides a powerful way to create and edit scenes through its responsive and user-friendly interface, one of the main limitations is

that it is only available when using the OpenGL API. This limitation is due to the reliance on the Dear ImGui Bundle package for the editor UI, which currently does not support WebGPU. In the future, we could explore ways to add bindings for WebGPU rendering if the package has not added support by then. Supporting the WebGPU API for the editor would significantly enhance the impact and usability of the editor layer feature.

The editor layer has almost infinite ways that can be improved and extended. For example, in the future, we could explore the possibility of adding a node-based system to visualize the scene hierarchy, providing an alternative way of viewing and editing entity parent-child relationships and better understanding the scene structure. Additionally, we could enhance the visual appearance of the editor with icons, better fonts, and improved layouts. The possibilities are endless.

6.3.3 Educational Examples

While the educational examples cover topics ranging from fundamental concepts to examples that encompass the entire graphics pipeline and advanced techniques and effects, there are still areas that have not been explored. These include skeletal animation, post-processing effects and deferred rendering. So, in the future the framework could be expanded to include them as well.

Bibliography

- [1] Thomas Suselo, Burkhard Wünsche, and Andrew Luxton-Reilly. The journey to improve teaching computer graphics: A systematic review. 12 2017.
- [2] D. G. Balreira, M. Walter, and D. W. Fellner. What we are teaching in introduction to computer graphics. In *Proceedings of the European Association for Computer Graphics: Education Papers*, EG '17, page 1–7, Goslar, DEU, 2017. Eurographics Association. doi: 10.2312/eged.20171019. URL <https://doi.org/10.2312/eged.20171019>.
- [3] Learnopengl - your number 1 resource for opengl, 2024. URL <https://learnopengl.com/>.
- [4] Apple inc. metal. accelerating graphics and much more, 2022. URL <https://developer.apple.com/metal>.
- [5] Microsoft corporation. directx graphics and gaming, 2022. URL <https://learn.microsoft.com/en-us/windows/win32/directx>.
- [6] The khronos® group inc. vulkan — cross platform 3d graphics, 2022. URL <https://www.vulkan.org/>.
- [7] Webgpu — unlocking modern gpu access in the browser, 2023. URL <https://gpuweb.github.io/gpuweb/>.
- [8] Introduction to computer graphics and ray-tracing using the webgpu api. In *SIGGRAPH Asia 2022 Courses*, SA '22, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450394741. doi: 10.1145/3550495.3558218. URL <https://doi.org/10.1145/3550495.3558218>.
- [9] Benjamin Kenwright. Web programming using the webgpu api. In *ACM SIGGRAPH 2023 Courses*, SIGGRAPH '23, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400701450. doi: 10.1145/3587423.3595543. URL <https://doi.org/10.1145/3587423.3595543>.
- [10] Learnwebgpu - walkthrough of the use of the webgpu graphics api to create native 3d applications in c++ from scratch, for windows, linux and macos., 2024. URL <https://eliemichel.github.io/LearnWebGPU/>.

- [11] Johannes Unterguggenberger, Bernhard Kerbl, and Michael Wimmer. Vulkan all the way: Transitioning to a modern low-level graphics api in academia. *Computers & Graphics*, 111:155–165, 2023. ISSN 0097-8493. doi: <https://doi.org/10.1016/j.cag.2023.02.001>. URL <https://www.sciencedirect.com/science/article/pii/S0097849323000249>.
- [12] George Papagiannakis, Manos Kamarianakis, Antonis Protopsaltis, Dimitris Angelis, and Paul Zikas. Project Elements: A Computational Entity-component-system in a Scene-graph Pythonic Framework, for a Neural, Geometric Computer Graphics Curriculum. In Alejandra Magana and Jiri Zara, editors, *Eurographics 2023 - Education Papers*. The Eurographics Association, 2023. ISBN 978-3-03868-210-3. doi: 10.2312/eged.20231015.
- [13] Thibault Raffallac and Stéphane Huot. Polyphony: Programming interfaces and interactions with the entity-component-system model. *Proceedings of the ACM on Human-Computer Interaction*, 3(EICS):1–22, 2019.
- [14] Robert Nystrom. *Game programming patterns*. Genever Benning, 2014.
- [15] Brian Danchilla. *Three.js Framework*, pages 173–203. Apress, Berkeley, CA, 2012. ISBN 978-1-4302-3997-0. doi: 10.1007/978-1-4302-3997-0_7. URL https://doi.org/10.1007/978-1-4302-3997-0_7.
- [16] John K Haas. A history of the unity game engine. 2014.
- [17] George Papagiannakis, Petros Papanikolaou, Elisavet Greassidou, and Panos Trahanias. glGA: an OpenGL Geometric Application Framework for a Modern, Shader-based Computer Graphics Curriculum. In Jean-Jacques Bourdin, Joaquim Jorge, and Eike Anderson, editors, *Eurographics 2014 - Education Papers*. The Eurographics Association, 2014. doi: /10.2312/eged.20141026.
- [18] Antoine Toisoul, Daniel Rueckert, and Bernhard Kainz. Accessible glsl shader programming. In *Proceedings of the European Association for Computer Graphics: Education papers*, pages 35–42. Eurographics Association, 2017.
- [19] Burkhard C. Wünsche, Edward Huang, Lindsay Shaw, Thomas Suselo, Kai-Cheung Leung, Davis Dimalen, Wannes van der Mark, Andrew Luxton-Reilly, and Richard Lobb. Coderunnergl - an interactive web-based tool for computer graphics teaching and assessment. In *2019 International Conference on Electronics, Information, and Communication (ICEIC)*, pages 1–7, 2019. doi: 10.23919/ELINFOCOM.2019.8706402.
- [20] Johannes Unterguggenberger, Bernhard Kerbl, and Michael Wimmer. The road to vulkan: Teaching modern low-level apis in introductory graphics courses. In *Eurographics 2022 - Education Papers*, page 9. The Eurographics Association, April 2022. ISBN 978-3-03868-170-0. doi: 10.2312/eged.20221043. URL <https://www.cg.tuwien.ac.at/research/publications/2022/unterguggenberger-2022-vulkan/>.

- [21] Burkhard C. Wünsche, Zhen Chen, Lindsay Shaw, Thomas Suselo, Kai-Cheung Leung, Davis Dimalen, Wannes van der Mark, Andrew Luxton-Reilly, and Richard Lobb. Automatic assessment of opengl computer graphics assignments. In *Proceedings of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education*, ITiCSE 2018, page 81–86, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450357074. doi: 10.1145/3197091.3197112. URL <https://doi.org/10.1145/3197091.3197112>.
- [22] Carlos Andujar, Antonio Chica, Marta Fairén, and Alvar Vinacua. GL-Socket: A CG Plugin-based Framework for Teaching and Assessment. In Frits Post and Jirí Žára, editors, *EG 2018 - Education Papers*. The Eurographics Association, 2018. doi: 10.2312/eged.20181003.
- [23] Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 3rd edition, 2016. ISBN 0128006455.
- [24] Paul Zikas, Antonis Protopsaltis, Nick Lydatakis, Mike Kentros, Stratos Geronikolakis, Steve Kateros, Manos Kamarianakis, Giannis Evangelou, Achilleas Filippidis, Eleni Grigoriou, Dimitris Angelis, Michail Tamiolakis, Michael Dodis, George Kokiadis, John Petropoulos, Maria Pateraki, and George Papagiannakis. Mages 4.0: Accelerating the world’s transition to vr training and democratizing the authoring of the medical metaverse. *IEEE Computer Graphics and Applications*, 43(2):43–56, 2023. doi: 10.1109/MCG.2023.3242686.
- [25] Dear imgui bundle: an extensive set of ready-to-use widgets and libraries, based on imgui, 2023. URL https://pthom.github.io/imgui_bundle/index.html.
- [26] Evaluate interface learnability with cognitive walkthroughs, . URL <https://www.nngroup.com/articles/cognitive-walkthroughs/>.
- [27] How to conduct a cognitive walkthrough, . URL <https://www.interaction-design.org/literature/article/how-to-conduct-a-cognitive-walkthrough>.
- [28] Thomas Mahatody, Mouldi Sagar, and Christophe Kolski. State of the art on the cognitive walkthrough method, its variants and evolutions. *Intl. Journal of Human-Computer Interaction*, 26(8):741–785, 2010.
- [29] Lars-Ola Bligård and Anna-Lisa Osvalder. An analytical approach for predicting and identifying use error and usability problem. In *HCI and Usability for Medicine and Health Care: Third Symposium of the Workgroup Human-Computer Interaction and Usability Engineering of the Austrian Computer*

- Society, USAB 2007 Graz, Austria, November, 22, 2007. Proceedings 3*, pages 427–440. Springer, 2007.
- [30] Sandra G. Hart and Lowell E. Staveland. Development of nasa-tlx (task load index): Results of empirical and theoretical research. In Peter A. Hancock and Najmedin Meshkati, editors, *Human Mental Workload*, volume 52 of *Advances in Psychology*, pages 139–183. North-Holland, 1988. doi: [https://doi.org/10.1016/S0166-4115\(08\)62386-9](https://doi.org/10.1016/S0166-4115(08)62386-9). URL <https://www.sciencedirect.com/science/article/pii/S0166411508623869>.

Appendices

Appendix A

Core Framework Code

```
1 from pyGandalf.scene.scene_manager import SceneManager
2 from pyGandalf.scene.editor_manager import EditorManager
3 from pyGandalf.renderer.base_renderer import BaseRenderer
4 from pyGandalf.renderer.opengl_renderer import OpenGLRenderer
5 from pyGandalf.renderer.webgpu_renderer import WebGPURenderer
6 from pyGandalf.renderer.imgui_renderer import ImGuiRenderer, ImGuiTheme
7 from pyGandalf.core.base_window import BaseWindow
8 from pyGandalf.core.input_manager import InputManager
9 from pyGandalf.core.event_manager import EventManager
10
11 import glfw
12
13 class Application(object):
14     def __new__(cls):
15         if not hasattr(cls, 'instance'):
16             cls.instance = super(Application, cls).__new__(cls)
17             cls.instance.renderer = None
18             cls.instance.window = None
19             cls.instance.is_application_running = False
20             cls.instance.delta_time = 0.0
21             cls.instance.last_time = 0.0
22             cls.instance.timer = 0.0
23             cls.instance.frames = 0.0
24             cls.instance.is_imgui_attached = False
25             cls.instance.is_editor_attached = False
26         return cls.instance
27
28     def get_window(cls) -> BaseWindow:
29         return cls.instance.window
30
31     def get_renderer(cls) -> BaseRenderer:
32         return cls.instance.renderer()
33
34     def is_running(cls) -> bool:
35         return cls.instance.is_application_running
36
37     def set_is_running(cls, is_running):
38         cls.instance.is_application_running = is_running
39
40     def create(cls, window : BaseWindow, renderer : BaseRenderer, attach_imgui = False, attach_editor
41               = False):
42         cls.instance.window = window
43         cls.instance.renderer = renderer
44         cls.instance.is_imgui_attached = attach_imgui
45         cls.instance.is_editor_attached = attach_editor
46         cls.instance.window.create()
47         InputManager().initialize(cls.instance.window.get_handle())
48         EventManager().initialize(cls.instance.window.get_handle(), renderer=cls.instance.renderer)
49
50         if type(renderer()) == OpenGLRenderer:
51             renderer().initialize(attach_editor)
52         elif type(renderer()) == WebGPURenderer:
53             renderer().initialize(cls.instance.window, "high-performance")
54
55         if cls.instance.is_imgui_attached:
56             ImGuiRenderer().initialize(cls.instance.window.get_handle(), ImGuiTheme.DARK,
57                                       attach_editor)
58
59     def start(cls):
```

```

58     SceneManager().on_create()
59
60     if cls.instance.is_editor_attached:
61         EditorManager().on_create()
62
63     def main_loop():
64         cls.instance.begin_frame()
65         cls.instance.renderer().begin_frame()
66         SceneManager().on_update(cls.instance.delta_time)
67         cls.instance.renderer().end_frame()
68         if cls.instance.is_imgui_attached:
69             ImGuiRenderer().begin_frame()
70             if cls.instance.is_editor_attached:
71                 EditorManager().on_gui_update(cls.instance.delta_time)
72                 SceneManager().on_gui_update(cls.instance.delta_time)
73             ImGuiRenderer().end_frame()
74         cls.instance.end_frame()
75         EventManager().process()
76
77     cls.instance.window.dispatch_main_loop(main_loop)
78
79     cls.instance.clean()
80
81     def begin_frame(cls):
82         time = glfw.get_time()
83         cls.instance.delta_time = time - cls.instance.last_time
84         cls.instance.last_time = time
85
86     def end_frame(cls):
87         cls.instance.frames += 1
88         if glfw.get_time() - cls.instance.timer > 1.0:
89             title = cls.instance.window.get_title() + f' [FPS: {cls.instance.frames}]'
90             glfw.set_window_title(cls.instance.window.get_handle(), title)
91             cls.instance.timer += 1
92             cls.instance.frames = 0
93
94     def quit(cls):
95         cls.instance.is_application_running = False
96         cls.instance.window.close()
97
98     def clean(cls):
99         if cls.instance.is_imgui_attached:
100             ImGuiRenderer().clean()
101         if (cls.instance.renderer is not None):
102             cls.instance.renderer().clean()
103         if (cls.instance.window is not None):
104             cls.instance.window.destroy()
105         SceneManager().clean()

```

Listing A.1: Application Class

```

1  from pyGandalf.scene.entity import Entity
2  from pyGandalf.scene.components import Component
3
4  from pyGandalf.core.events import Event, PushEvent, EventType
5  from pyGandalf.utilities.logger import logger
6
7  from enum import Enum
8
9  class SystemState(Enum):
10     NONE = 0
11     PLAY = 1
12     PAUSE = 2
13
14  class System:
15     """A class that represents a system that operates on entities that have all the specified
16     components.
17     """
18     def __init__(self, filters: list[type]):
19         self.filters = filters
20         self.filtered_components = []
21         self.filtered_entities = []
22         self.state = SystemState.PLAY
23
24     def set_state(self, state: SystemState):
25         """Sets the state of the system.
26
27         Args:
28             state (SystemState): The state of the system.
29         """
30         self.state = state
31         self.record_system_state_event(type(self), self.state)

```

```

32 def get_state(self) -> SystemState:
33     """Returns the state of the system.
34
35     Returns:
36         SystemState: The state of the system.
37     """
38     return self.state
39
40 def filter(self, scene):
41     """Filters from the given scene the entities and their component that the system operates on
42     and caches them.
43
44     Args:
45         scene (Scene): The scene to filter entities from.
46     """
47     components_array = scene.get_components_array()
48
49     for entity in scene.get_entities():
50         entity_components = scene.get_entity_component_references(entity)
51         self.filter_entity_components(entity, entity_components, components_array)
52
53 def filter_entity_components(self, entity, entity_components, components_array):
54     """Stores the given entity and its components if the entity has all the components that the
55     system operates on.
56
57     Args:
58         entity (Entity): The entity to process.
59         entity_components (_type_): The components of the entity.
60         components_array (_type_): The dictionary holding all the component arrays.
61     """
62     if (entity in self.filtered_entities):
63         return
64
65     i, j = 0, 0
66     for filter in self.filters:
67         j += 1
68         if filter in entity_components.keys():
69             i += 1
70         if (j == i):
71             self.filtered_components.append(tuple(components_array[filter][entity_components[filter]]
72             for filter in self.filters))
73             self.filtered_entities.append(entity)
74
75     return self.filtered_components[-1]
76
77 return None
78
79 def remove_entity_components(self, entity, component):
80     """Removes the entity and its component from the cached arrays.
81
82     Args:
83         entity (Entity): The entity to remove.
84         component (_type_): The component to remove.
85
86     Returns:
87         bool: 'True' if the entity and the component was successfully removed from the system
88         or 'False' if it was not present in the cached arrays of the system.
89     """
90     if entity not in self.filtered_entities:
91         return False
92
93     index = self.filtered_entities.index(entity)
94
95     if not tuple(filter(lambda i: (type(i) == component), self.filtered_components[index])):
96         return False
97
98     self.filtered_entities.remove(entity)
99     self.filtered_components.pop(index)
100
101     return True
102
103 def on_create_base(self):
104     """Calls the 'on_create_entity' and 'on_create_system' method of the system for each entity
105     and its filtered components.
106
107     """
108     if self.state is SystemState.PLAY:
109         self.on_create_system()
110
111     if self.state is SystemState.PLAY:
112         for entity, components in zip(self.filtered_entities, self.filtered_components):
113             if (len(components) == 1):
114                 self.on_create_entity(entity, components[0])
115             else:
116                 self.on_create_entity(entity, components)

```

```

111 def on_update_base(self, ts: float):
112     """Calls the 'on_update_entity' and 'on_update_system' methods of the system for each entity
113     and its filtered components.
114     """
115     if self.state is SystemState.PLAY:
116         self.on_update_system(ts)
117
118     if self.state is SystemState.PLAY:
119         for entity, components in zip(self.filtered_entities, self.filtered_components):
120             if (len(components) == 1):
121                 self.on_update_entity(ts, entity, components[0])
122             else:
123                 self.on_update_entity(ts, entity, components)
124
125 def on_gui_update_base(self, ts: float):
126     """Calls the 'on_gui_update_entity' and 'on_gui_update_system' methods of the system for each
127     entity and its filtered components.
128     """
129     if self.state is SystemState.PLAY:
130         self.on_gui_update_system(ts)
131
132     if self.state is SystemState.PLAY:
133         for entity, components in zip(self.filtered_entities, self.filtered_components):
134             if (len(components) == 1):
135                 self.on_gui_update_entity(ts, entity, components[0])
136             else:
137                 self.on_gui_update_entity(ts, entity, components)
138
139 def on_create_entity(self, entity: Entity, components: Component | tuple[Component]):
140     """Gets called once in the first frame for each entity that the system operates on.
141
142     Args:
143         entity (Entity): The current entity to process.
144         components (Component | tuple[Component]): The component(s) of the current entity that
145         are manipulated by the system.
146     """
147     raise NotImplementedError()
148
149 def on_update_entity(self, ts: float, entity: Entity, components: Component | tuple[Component]):
150     """Gets called every frame for each entity that the system operates on.
151
152     Args:
153         ts (float): The application time step (delta time).
154         entity (Entity): The current entity to process.
155         components (Component | tuple[Component]): The component(s) of the current entity that
156         are manipulated by the system.
157     """
158     pass
159
160 def on_gui_update_entity(self, ts: float, entity: Entity, components: Component | tuple[Component]):
161     """Gets called every frame for each entity that the system operates on and responsible for
162     drawing gui elements.
163
164     Args:
165         ts (float): The application time step (delta time).
166         entity (Entity): The current entity to process.
167         components (Component | tuple[Component]): The component(s) of the current entity that
168         are manipulated by the system.
169     """
170     pass
171
172 def on_create_system(self):
173     """Gets called once in the first frame and initializes the system (filtered entities and
174     their components).
175     """
176     pass
177
178 def on_update_system(self, ts: float):
179     """Gets called every frame to update the system (filtered entities and their components).
180
181     Args:
182         ts (float): The application time step (delta time).
183     """
184     pass
185
186 def on_gui_update_system(self, ts: float):
187     """Gets called every frame to update the system gui (gui for filtered entities and their
188     components).
189
190     Args:
191         ts (float): The application time step (delta time).
192     """
193     pass

```

```

186         pass
187
188     def filtered_data(self):
189         """Returns the entities and components that the system operates on, zipped.
190
191         Returns:
192             _type_: The entities and components that the system operates on, zipped.
193         """
194         return zip(self.filtered_entities, self.filtered_components)
195
196     def get_filtered_entities(self):
197         """Returns the entities that the system operates on.
198
199         Returns:
200             list[Entity]: The entities that the system operates on.
201         """
202         return self.filtered_entities
203
204     def get_filtered_components(self):
205         """Returns the components that the system operates on.
206
207         Returns:
208             list[tuple]: The components that the system operates on.
209         """
210         return self.filtered_components
211
212     def record_system_state_event(cls, type: type, state: SystemState):
213         """Pushes an EventType.SYSTEM_STATE event when the system state changes.
214
215         Args:
216             type (type): The type of the System
217             state (SystemState): The new system state.
218         """
219         ev = {
220             "type": type,
221             "state": state
222         }
223
224         event = Event()
225         event.type = EventType.SYSTEM_STATE
226         event.data = ev
227         PushEvent(event)

```

Listing A.2: Base System Class

```

1 from pyGandalf.scene.entity import Entity
2 from pyGandalf.systems.system import System
3 from pyGandalf.utilities.logger import logger
4 from pyGandalf.scene.components import LinkComponent
5 from pyGandalf.scene.editor_components import EditorVisibleComponent
6
7 class Scene():
8     """A class that represents a scene which contains all the entities, their components and the
9     registered systems.
10
11     def __init__(self, name = 'UnnamedScene'):
12         self.entity_components = {}
13         self.component_arrays = {}
14         self.entities = []
15         self.systems = []
16         self.name = name
17
18     def enroll_entity(self) -> Entity:
19         """Creates and enrolls a new entity to the scene.
20
21         Returns:
22             Entity: The newly created entity.
23         """
24         entity = Entity()
25         self.entities.append(entity)
26         self.add_component(entity, EditorVisibleComponent())
27         return entity
28
29     def enroll_entity_with_uuid(self, uuid) -> Entity:
30         """Creates and enrolls a new entity to the scene with the given uuid as id.
31
32         Args:
33             uuid (_type_): The uuid to set to the entity's id.
34
35         Returns:
36             Entity: The newly created entity.
37         """
38         entity = Entity()

```

```

38     entity.id = uuid
39     self.entities.append(entity)
40     self.add_component(entity, EditorVisibleComponent())
41     return entity
42
43 def destroy_entity(self, entity: Entity):
44     """Destroy the given entity from the scene. If the entity has children, recursively destroys
45     them all.
46
47     Args:
48         entity (Entity): The entity to destroy.
49
50     entity_component_references = self.get_entity_component_references(entity)
51
52     component_types = []
53     for type in entity_component_references.keys():
54         component_types.append(type)
55
56     # Recursively destroy all children of entity.
57     link = self.get_component(entity, LinkComponent)
58     if link != None:
59         for child in link.children:
60             self.destroy_entity(child)
61
62     # Remove all components from entity.
63     for component_type in component_types:
64         self.remove_component(entity, component_type)
65
66     # Remove entity from list.
67     if entity in self.entities:
68         self.entities.remove(entity)
69
70 def get_entities(self) -> list[Entity]:
71     """Returns a list containing all the enrolled entities of the scene.
72
73     Returns:
74         list[Entity]: A list containing all the enrolled entities of the scene.
75
76     """
77     return self.entities
78
79 def add_component(self, entity: Entity, component):
80     """Adds to the specified entity the given component, if the entity does not already have a
81     component of that type. Also adds the component and entity to the systems that operate on the
82     component type.
83
84     Args:
85         entity (Entity): The entity to add the given component to.
86         component (type(component)): The component to add to the given entity.
87
88     Returns:
89         type(component): The component that was added or the already existing one if the entity
90         already has a component of that type.
91
92     """
93     # Create new component reference array for entity if it does not exist
94     if entity.id not in self.entity_components:
95         self.entity_components[entity.id] = {}
96
97     # Retrieve type from component
98     component_type = type(component)
99
100     # Check if already has component
101     if self.has_component(entity, component_type) is True:
102         logger.info(f'Entity with id: {entity} already has component of type: {component_type},
103         returning that.')
104         return self.get_component(entity, component_type)
105
106     # Create new component array if it does not exist yet
107     if component_type not in self.component_arrays:
108         self.component_arrays[component_type] = []
109
110     # Find the entities component reference
111     component_array = self.component_arrays[component_type]
112     component_index = len(component_array)
113
114     # Update entity's component reference
115     self.entity_components[entity.id][component_type] = component_index
116
117     # Add the new component to the appropriate component array
118     component_array.append(component)
119
120     # Update existing systems that operate on this component. (Usefull in runtime addition of
121     components)
122     for system in self.systems:
123         entity_components = self.get_entity_component_references(entity)

```

```

116         components = system.filter_entity_components(entity, entity_components, self.
117             component_arrays)
118
119         from pyGandalf.core.application import Application
120         if (Application().is_running()):
121             if components != None:
122                 if (len(components) == 1):
123                     system.on_create_entity(entity, components[0])
124                 else:
125                     system.on_create_entity(entity, components)
126
127         return component
128
129 def remove_component(self, entity: Entity, component_type: type):
130     """Removes from the specified entity the component of the given type (if the entity has a
131     component of that type) and from all the systems that operate on the component type.
132
133     Args:
134         entity (Entity): The entity to remove the component from.
135         component_type (type): The type of the component to remove from the entity.
136     """
137     if self.has_component(entity, component_type) is False:
138         return
139
140     # Update existing systems that operate on this component. (Usefull in runtime deletion of
141     # components)
142     for system in self.systems:
143         system.remove_entity_components(entity, component_type)
144
145     # Update component arrays and entity components references
146     self.component_arrays[component_type][self.entity_components[entity.id][component_type]] =
147     None
148     self.entity_components[entity.id][component_type] = None
149
150 def has_component(self, entity: Entity, component_type: type) -> bool:
151     """Returns whether or not the specified entity has a component of the given type.
152
153     Args:
154         entity (Entity): The entity to check.
155         component_type (type): The type of component to check.
156
157     Returns:
158         bool: "True" if the entity has the component or "False" otherwise.
159     """
160     return (entity.id in self.entity_components and
161             component_type in self.entity_components.get(entity.id) and
162             self.entity_components[entity.id][component_type] != None and
163             self.component_arrays[component_type][self.entity_components[entity.id][
164             component_type]] != None)
165
166 def get_component(self, entity: Entity, component_type: type):
167     """Returns a component of the specified type from the given entity or "None" if the
168     entity does not have a component of the specified type.
169
170     Args:
171         entity (Entity): The entity to get the component from.
172         component_type (type): The type of component to get from the given entity.
173
174     Returns:
175         component: The component of the specified type from the given entity or "None" if the
176         entity does not have a component of the specified type.
177     """
178     if self.has_component(entity, component_type) == False:
179         return None
180
181     component_index = self.entity_components[entity.id][component_type]
182     return self.component_arrays[component_type][component_index]
183
184 def get_entity_component_references(self, entity: Entity):
185     """Returns a dictionary with keys the component type and values the indices to the component
186     arrays for the specified entity.
187
188     Args:
189         entity (Entity): The entity to get the component references for.
190
191     Returns:
192         dict[type, list[int]]: A dictionary with keys the component type and values the indices
193         to the component arrays for the specified entity.
194     """
195     return self.entity_components.get(entity.id, {})
196
197 def get_components_array(self):
198     """Returns a dictionary with keys the compoennt type and values the component arrays.
199
200     """

```

```

191     Returns:
192         dict[type, list[component]]: A dictionary with keys the component type and values the
193         component arrays.
194     """
195     return self.component_arrays
196
197 def register_system(self, system: System):
198     """Registers a system to the scene. If called at runtime it will also start the system.
199
200     Args:
201         system (System): The system to register to the scene.
202     """
203     self.systems.append(system)
204
205     system.filter(self)
206
207     from pyGandalf.core.application import Application
208
209     if (Application().is_running()):
210         system.on_create_base()
211
212 def get_systems(self) -> list[System]:
213     """Return a list containing all the systems of the scene.
214
215     Returns:
216         list[System]: A list containing all the systems of the scene.
217     """
218     return self.systems
219
220 def get_system(self, system_type: type) -> (System | None):
221     """Returns a system from the scene of the specified type.
222
223     Args:
224         system_type (type): The type of the system to get.
225
226     Returns:
227         System: A system from the scene of the specified type or "None" if the scene does not
228         have a system with the specified type.
229     """
230     for system in self.systems:
231         if type(system) is system_type:
232             return system
233     return None
234
235 def on_create(self):
236     """Called on the first frame and calls all the on_create_base method from all the registered
237     systems of the scene.
238     """
239     from pyGandalf.core.application import Application
240
241     Application().set_is_running(True)
242
243     for system in self.systems:
244         system.on_create_base()
245
246 def on_update(self, ts: float):
247     """Called every frame and calls all the on_update_base method from all the registered systems
248     of the scene.
249
250     Args:
251         ts (float): The application timestep.
252     """
253     for system in self.systems:
254         system.on_update_base(ts)
255
256 def on_gui_update(self, ts: float):
257     """Called every frame and calls all the on_update_gui_base method from all the registered
258     systems of the scene.
259
260     Args:
261         ts (float): The application timestep.
262     """
263     for system in self.systems:
264         system.on_gui_update_base(ts)

```

Listing A.3: Scene Class

```

1 from pyGandalf.scene.scene import Scene
2 from pyGandalf.utilities.logger import logger
3 from pyGandalf.core.events import Event, PushEvent, EventType
4
5 class SceneManager(object):
6     def __new__(cls):

```

```

7         if not hasattr(cls, 'instance'):
8             cls.instance = super(SceneManager, cls).__new__(cls)
9             cls.instance.scenes = []
10            cls.instance.active_scene = None
11            cls.instance.new_scene_to_loaded = None
12            cls.instance.active_scene_index = 0
13            cls.instance.scene_change_requested = False
14            cls.instance.main_camera = None
15            cls.instance.main_camera_entity = None
16            return cls.instance
17
18    def add_scene(cls, scene: Scene):
19        if scene is None:
20            logger.error('Scene is null, not appending it.')
21            return
22
23        cls.instance.scenes.append(scene)
24
25        if len(cls.instance.scenes) == 1:
26            cls.instance.active_scene = cls.instance.scenes[0]
27
28    def on_create(cls):
29        if len(cls.instance.scenes) == 0:
30            logger.critical('No scenes added, exiting...')
31            exit(-1)
32
33        cls.instance.active_scene_index = 0
34        cls.instance.active_scene = cls.instance.scenes[cls.instance.active_scene_index]
35        cls.instance.active_scene.on_create()
36
37    def on_update(cls, ts):
38        if cls.instance.active_scene is None:
39            logger.critical('There is no active scene currently')
40            return
41
42        cls.instance.active_scene.on_update(ts)
43
44        if cls.instance.scene_change_requested:
45            cls.instance.change_scene_deferred()
46
47    def on_gui_update(cls, ts):
48        if cls.instance.active_scene is None:
49            logger.critical('There is no active scene currently')
50            return
51
52        cls.instance.active_scene.on_gui_update(ts)
53
54    def change_scene(cls, scene: Scene = None):
55        if scene is None:
56            cls.instance.active_scene_index += 1
57        else:
58            cls.instance.new_scene_to_loaded = scene
59            cls.instance.active_scene_index = cls.instance.scenes.index(scene)
60
61        if len(cls.instance.scenes) <= cls.instance.active_scene_index:
62            logger.error(f'Out of bounds scene index: {cls.instance.active_scene_index} for scene
63            change')
64            return
65
66        cls.instance.record_scene_change_event(cls.instance.active_scene, cls.instance.scenes[cls.
67        instance.active_scene_index])
68
69        cls.instance.scene_change_requested = True
70
71    def open_external_scene(cls, scene: Scene):
72        cls.instance.new_scene_to_loaded = scene
73        cls.instance.record_scene_change_event(cls.instance.active_scene, cls.instance.
74        new_scene_to_loaded)
75        cls.instance.scene_change_requested = True
76
77    def change_scene_deferred(cls):
78        if cls.instance.new_scene_to_loaded is None:
79            if len(cls.instance.scenes) <= cls.instance.active_scene_index:
80                logger.error(f'Out of bounds scene index: {cls.instance.active_scene_index} for scene
81                change')
82                cls.instance.scene_change_requested = False
83                return
84
85            cls.instance.active_scene = cls.instance.scenes[cls.instance.active_scene_index]
86        else:
87            cls.instance.active_scene = cls.instance.new_scene_to_loaded
88
89        if cls.instance.active_scene is None:
90            logger.critical(f'New scene is none, exiting.')

```

```

87         exit(-1)
88
89         cls.instance.main_camera = None
90         cls.instance.main_camera_entity = None
91
92         cls.instance.active_scene.on_create()
93         cls.instance.new_scene_to_loaded = None
94         cls.instance.scene_change_requested = False
95
96     def get_active_scene(cls) -> Scene:
97         return cls.instance.active_scene
98
99     def get_main_camera(cls):
100         return cls.instance.main_camera
101
102     def get_main_camera_entity(cls):
103         return cls.instance.main_camera_entity
104
105     def set_main_camera(cls, main_camera_entity, main_camera):
106         cls.instance.main_camera = main_camera
107         cls.instance.main_camera_entity = main_camera_entity
108
109     def clean(cls):
110         cls.instance.scenes.clear()
111         cls.instance.active_scene = None
112         cls.instance.new_scene_to_loaded = None
113         cls.instance.active_scene_index = 0
114         cls.instance.scene_change_requested = False
115         cls.instance.main_camera = None
116         cls.instance.main_camera_entity = None
117
118     def record_scene_change_event(cls, previous, next):
119         ev = {
120             "previous": previous,
121             "next": next
122         }
123
124         event = Event()
125         event.type = EventType.SCENE_CHANGE
126         event.data = ev
127         PushEvent(event)

```

Listing A.4: SceneManager Class

Appendix B

Core Renderer Code

```
1 from pyGandalf.renderer.base_renderer import BaseRenderer
2 from pyGandalf.utilities.opengl_texture_lib import OpenGLTextureLib
3 from pyGandalf.utilities.opengl_material_lib import OpenGLMaterialLib
4
5 from pyGandalf.utilities.logger import logger
6
7 import OpenGL.GL as gl
8 import numpy as np
9 import glm
10
11 import ctypes
12
13 class OpenGLRenderer(BaseRenderer):
14     def initialize(cls, *kargs):
15         # Initialize OpenGL
16
17         cls.instance.use_framebuffer = kargs[0]
18         cls.instance.framebuffer_id = 0
19         cls.instance.color_attachment = 0
20         cls.instance.depth_attachment = 0
21         cls.instance.framebuffer_width = 0
22         cls.instance.framebuffer_height = 0
23         cls.instance.shadows_enabled = False
24
25         cls.instance.clear_color = glm.vec4(0.25, 0.25, 0.25, 1.0)
26
27         if cls.instance.use_framebuffer:
28             cls.instance.invalidate_framebuffer(1280, 720)
29
30     def resize(cls, width, height):
31         gl.glViewport(0, 0, width, height)
32
33     def add_batch(cls, render_data, material):
34         if material.instance == None:
35             logger.error('Material instance is None')
36             return -1
37
38         if material.instance.descriptor.blend_enabled:
39             gl.glEnable(gl.GL_BLEND)
40             gl.glBlendFunc(gl.GL_SRC_ALPHA, gl.GL_ONE_MINUS_SRC_ALPHA)
41             gl.glBlendEquation(gl.GL_FUNC_ADD)
42
43         if material.instance.descriptor.cull_enabled:
44             gl.glEnable(gl.GL_CULL_FACE)
45             gl.glFrontFace(gl.GL_CCW)
46
47         if material.instance.descriptor.depth_enabled:
48             gl.glEnable(gl.GL_DEPTH_TEST)
49
50         if material.instance.descriptor.primitive == gl.GL_PATCHES:
51             gl.glPatchParameteri(gl.GL_PATCH_VERTICES, material.instance.descriptor.
vertices_per_patch)
52
53         # Vertex Array Object (VAO)
54         render_data.render_pipeline = gl.glGenVertexArrays(1)
55         gl.glBindVertexArray(render_data.render_pipeline)
56
57         # Filter out None from attributes
58         render_data.attributes = list(filter(lambda x: x is not None, render_data.attributes))
```

```

59     for index, attribute in enumerate(render_data.attributes):
60         # Get a pointer to the NumPy array data
61         attribute_pointer = attribute.ctypes.data_as(ctypes.POINTER(gl.GLfloat))
62
63         # Vertex Buffer Object (VBO)
64         render_data.buffers.append(gl.glGenBuffers(1))
65         gl.glBindBuffer(gl.GL_ARRAY_BUFFER, render_data.buffers[-1])
66         gl.glBufferData(gl.GL_ARRAY_BUFFER, len(attribute) * len(attribute[0]) * 4,
67             attribute_pointer, gl.GL_STATIC_DRAW)
68
69         gl.glEnableVertexAttribArray(index)
70         gl.glVertexAttribPointer(index, len(attribute[0]), gl.GL_FLOAT, gl.GL_FALSE, len(
71             attribute[0]) * 4, ctypes.c_void_p(0))
72
73     if render_data.indices is not None:
74         # Get a pointer to the NumPy array data
75         indices_pointer = render_data.indices.ctypes.data_as(ctypes.POINTER(gl.GLuint))
76
77         # Element Buffer Object (EBO)
78         render_data.index_buffer = gl.glGenBuffers(1)
79         gl.glBindBuffer(gl.GL_ELEMENT_ARRAY_BUFFER, render_data.index_buffer)
80
81         if material.instance.descriptor.primitive == gl.GL_TRIANGLE_STRIP:
82             gl.glBufferData(gl.GL_ELEMENT_ARRAY_BUFFER, len(render_data.indices) * 4,
83                 indices_pointer, gl.GL_STATIC_DRAW)
84         else:
85             gl.glBufferData(gl.GL_ELEMENT_ARRAY_BUFFER, len(render_data.indices) * 3 * 4,
86                 indices_pointer, gl.GL_STATIC_DRAW)
87
88         # Use the shader program
89         gl.glUseProgram(material.instance.shader_program)
90
91     return 0
92
93 def begin_frame(cls):
94     if cls.instance.use_framebuffer:
95         gl.glBindFramebuffer(gl.GL_FRAMEBUFFER, cls.instance.framebuffer_id)
96         gl.glViewport(0, 0, int(cls.instance.framebuffer_width), int(cls.instance.
97             framebuffer_height))
98
99         gl.glClearColor(cls.instance.clear_color.r, cls.instance.clear_color.g, cls.instance.
100             clear_color.b, cls.instance.clear_color.a)
101         gl.glClear(gl.GL_COLOR_BUFFER_BIT | gl.GL_DEPTH_BUFFER_BIT)
102
103 def end_frame(cls):
104     if cls.instance.use_framebuffer:
105         gl.glBindFramebuffer(gl.GL_FRAMEBUFFER, 0)
106
107 def begin_render_pass(cls):
108     pass
109
110 def end_render_pass(cls):
111     pass
112
113 def set_pipeline(cls, render_data):
114     # Bind vao
115     gl.glBindVertexArray(render_data.render_pipeline)
116
117 def set_buffers(cls, render_data):
118     # Bind ebo
119     if render_data.indices is not None:
120         gl.glBindBuffer(gl.GL_ELEMENT_ARRAY_BUFFER, render_data.index_buffer)
121
122     # Bind vertex buffer and their layout
123     for index, attribute in enumerate(render_data.attributes):
124         # Vertex Buffer Object (VBO)
125         gl.glBindBuffer(gl.GL_ARRAY_BUFFER, render_data.buffers[index])
126
127         # Set vertex buffer layout
128         gl.glEnableVertexAttribArray(index)
129         gl.glVertexAttribPointer(index, len(attribute[0]), gl.GL_FLOAT, gl.GL_FALSE, len(
130             attribute[0]) * 4, ctypes.c_void_p(0))
131
132 def set_bind_groups(cls, material):
133     if material.instance.descriptor.depth_mask == gl.GL_FALSE:
134         gl.glDepthMask(gl.GL_FALSE)
135
136     if material.instance.descriptor.cull_enabled:
137         gl.glCullFace(material.instance.descriptor.cull_face)
138
139     if material.instance.descriptor.depth_enabled:
140         gl.glDepthFunc(material.instance.descriptor.depth_func)

```

```

136     # Bind shader program
137     gl.glUseProgram(material.instance.shader_program)
138
139     # Get uniform textures
140     textures = OpenGLMaterialLib().get_textures(material.instance.name)
141
142     # Bind textures
143     for index, texture_name in enumerate(material.instance.data.textures):
144         OpenGLTextureLib().bind(texture_name)
145         if material.instance.has_uniform(textures[index]):
146             slot = OpenGLTextureLib().get_slot(texture_name)
147             if slot == None:
148                 return
149             material.instance.set_uniform(textures[index], int(slot))
150
151 def draw(cls, render_data, material):
152     if material.instance.descriptor.primitive == gl.GL_PATCHES:
153         gl.glDrawArrays(gl.GL_PATCHES, 0, material.instance.descriptor.vertices_per_patch *
154             material.instance.descriptor.patch_resolution * material.instance.descriptor.patch_resolution)
155     else:
156         gl.glDrawArrays(material.instance.descriptor.primitive, 0, render_data.attributes[0].size
157     )
158
159     # Unbind vao
160     gl.glBindVertexArray(0)
161
162     # Unbind textures
163     for texture_name in material.instance.data.textures:
164         OpenGLTextureLib().unbind(texture_name)
165
166     # Unbind shader program
167     gl.glUseProgram(0)
168
169     # Reset depth mask
170     if material.instance.descriptor.depth_mask == gl.GL_FALSE:
171         gl.glDepthMask(gl.GL_TRUE)
172
173 def draw_indexed(cls, render_data, material):
174     gl.glDrawElements(material.instance.descriptor.primitive, render_data.indices.size, gl.
175     GL_UNSIGNED_INT, None)
176
177     # Unbind vao
178     gl.glBindVertexArray(0)
179     gl.glBindBuffer(gl.GL_ELEMENT_ARRAY_BUFFER, 0)
180
181     # Unbind textures
182     for texture_name in material.instance.data.textures:
183         OpenGLTextureLib().unbind(texture_name)
184
185     # Unbind shader program
186     gl.glUseProgram(0)
187
188     # Reset depth mask if it was set to false
189     if material.instance.descriptor.depth_mask == gl.GL_FALSE:
190         gl.glDepthMask(gl.GL_TRUE)
191
192 def clean(cls):
193     pass
194
195 def invalidate_framebuffer(cls, width, height):
196     cls.instance.framebuffer_width = width
197     cls.instance.framebuffer_height = height
198
199     if cls.instance.framebuffer_id:
200         gl.glDeleteFramebuffers(1, np.array(cls.instance.framebuffer_id, dtype=np.uint))
201         gl.glDeleteTextures(1, np.array(cls.instance.color_attachment, dtype=np.uint))
202         gl.glDeleteTextures(1, np.array(cls.instance.depth_attachment, dtype=np.uint))
203
204     # Build the texture that will serve as the color attachment for the framebuffer.
205     cls.instance.color_attachment = gl.glGenTextures(1)
206     gl.glBindTexture(gl.GL_TEXTURE_2D, cls.instance.color_attachment)
207
208     gl.glTexParameterf(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_MIN_FILTER, gl.GL_LINEAR)
209     gl.glTexParameterf(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_MAG_FILTER, gl.GL_LINEAR)
210     gl.glTexParameterf(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_WRAP_S, gl.GL_CLAMP_TO_BORDER)
211     gl.glTexParameterf(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_WRAP_T, gl.GL_CLAMP_TO_BORDER)
212     gl.glTexImage2D(gl.GL_TEXTURE_2D, 0, gl.GL_RGBA, int(width), int(height), 0, gl.GL_RGBA, gl.
213     GL_UNSIGNED_BYTE, None)
214
215     gl.glBindTexture(gl.GL_TEXTURE_2D, 0)
216
217     # Build the texture that will serve as the depth attachment for the framebuffer.
218     cls.instance.depth_attachment = gl.glGenTextures(1)
219     gl.glBindTexture(gl.GL_TEXTURE_2D, cls.instance.depth_attachment)

```

```

216     gl.glTexParameteri(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_MIN_FILTER, gl.GL_NEAREST)
217     gl.glTexParameteri(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_MAG_FILTER, gl.GL_NEAREST)
218     gl.glTexParameteri(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_WRAP_S, gl.GL_CLAMP_TO_EDGE)
219     gl.glTexParameteri(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_WRAP_T, gl.GL_CLAMP_TO_EDGE)
220     gl.glTexImage2D(gl.GL_TEXTURE_2D, 0, gl.GL_DEPTH_COMPONENT24, int(width), int(height), 0, gl.
GL_DEPTH_COMPONENT, gl.GL_UNSIGNED_INT, None)
221
222     gl.glBindTexture(gl.GL_TEXTURE_2D, 0)
223
224     # Build the framebuffer.
225     cls.instance.framebuffer_id = gl.glGenFramebuffers(1)
226     gl.glBindFramebuffer(gl.GL_FRAMEBUFFER, cls.instance.framebuffer_id)
227     gl.glFramebufferTexture2D(gl.GL_FRAMEBUFFER, gl.GL_COLOR_ATTACHMENT0, gl.GL_TEXTURE_2D, cls.
instance.color_attachment, 0)
228     gl.glFramebufferTexture2D(gl.GL_FRAMEBUFFER, gl.GL_DEPTH_ATTACHMENT, gl.GL_TEXTURE_2D, cls.
instance.depth_attachment, 0)
229
230     status: gl.GLenum = gl.glCheckFramebufferStatus(gl.GL_FRAMEBUFFER)
231     if status != gl.GL_FRAMEBUFFER_COMPLETE:
232         logger.error(status)
233         assert(False);
234
235     gl.glBindFramebuffer(gl.GL_FRAMEBUFFER, 0)
236
237     def get_color_attachment(cls):
238         return cls.instance.color_attachment
239
240     def set_fill_mode(cls, mode):
241         gl.glPolygonMode(gl.GL_FRONT_AND_BACK, mode)
242
243     def set_clear_color(cls, clear_color: glm.vec4):
244         cls.instance.clear_color = clear_color
245
246     def set_shadows_enabled(cls, shadows_enabled: bool):
247         cls.instance.shadows_enabled = shadows_enabled
248
249     def get_shadows_enabled(cls):
250         return cls.instance.shadows_enabled

```

Listing B.1: OpenGLRenderer Class

```

1  from pyGandalf.renderer.base_renderer import BaseRenderer
2  from pyGandalf.utilities.logger import logger
3
4  import glm
5  import wgpu
6
7  from dataclasses import dataclass, field
8
9  @dataclass
10 class ColorAttachmentDescription:
11     view: wgpu.GPUTextureView = None
12     resolve_target: wgpu.GPUTextureView = None
13     clear_value: tuple = (0.8, 0.5, 0.3, 1.0)
14     color_load_op: wgpu.LoadOp = wgpu.LoadOp.clear
15     color_store_op: wgpu.StoreOp = wgpu.StoreOp.store
16
17 @dataclass
18 class RenderPassDescription:
19     color_attachment: bool = True
20     depth_stencil_attachment: bool = True
21
22     color_attachments: list[ColorAttachmentDescription] = field(default_factory=list[
ColorAttachmentDescription])
23
24     depth_texture_view: wgpu.GPUTextureView = None
25     depth_clear_value: float = 1.0
26     depth_load_op: wgpu.LoadOp = wgpu.LoadOp.clear
27     depth_store_op: wgpu.StoreOp = wgpu.StoreOp.store
28     depth_read_only: bool = False
29
30     stencil_clear_value: float = 0.0
31     stencil_load_op: wgpu.LoadOp = wgpu.LoadOp.clear
32     stencil_store_op: wgpu.StoreOp = wgpu.StoreOp.store
33     stencil_read_only: bool = True
34
35 @dataclass
36 class RenderPipelineDescription:
37     render_data = None
38     material_instance = None
39
40 class WebGPURenderer(BaseRenderer):
41     def initialize(cls, *kargs):

```

```

42     cls.instance.canvas = kargs[0]
43     power_preference: str = kargs[1]
44     cls.instance.adapter = wgpu.gpu.request_adapter(power_preference=power_preference)
45     cls.instance.device = cls.instance.adapter.request_device(required_features=['
VertexWritableStorage'], required_limits={})
46
47     cls.instance.present_context = cls.instance.canvas.get_context()
48     cls.instance.render_texture_format = cls.instance.present_context.get_preferred_format(cls.
instance.device.adapter)
49     cls.instance.present_context.configure(device=cls.instance.device, format=cls.instance.
render_texture_format)
50
51     cls.instance.command_queue = []
52     cls.instance.command_encoder: wgpu.GPUCommandEncoder = None # type: ignore
53     cls.instance.current_render_pass: wgpu.GPURenderPassEncoder = None # type: ignore
54     cls.instance.current_texture = None
55     cls.instance.depth_texture_view = None
56     cls.instance.clear_color = glm.vec4(0.8, 0.5, 0.3, 1.0)
57     cls.instance.shadows_enabled = False
58
59 def begin_frame(cls):
60     cls.instance.current_texture = cls.instance.present_context.get_current_texture()
61
62 def end_frame(cls):
63     if cls.instance.command_encoder != None:
64         cls.instance.device.queue.submit([cls.instance.command_encoder.finish()])
65     cls.instance.canvas.request_draw()
66
67 def resize(cls, width, height):
68     pass
69
70 def clean(cls):
71     pass
72
73 def create_buffers(cls, render_data):
74     # Filter out None from attributes
75     render_data.attributes = list(filter(lambda x: x is not None, render_data.attributes))
76
77     for attribute in render_data.attributes:
78         buffer : wgpu.GPUBuffer = cls.instance.device.create_buffer_with_data(
79             data=attribute,
80             usage=wgpu.BufferUsage.VERTEX
81         )
82
83         render_data.buffers.append(buffer)
84
85     if render_data.indices is not None:
86         index_buffer : wgpu.GPUBuffer = cls.instance.device.create_buffer_with_data(
87             data=render_data.indices,
88             usage=wgpu.BufferUsage.INDEX
89         )
90         render_data.index_buffer = index_buffer
91
92 def create_render_pipeline(cls, render_pipeline_desc: RenderPipelineDescription):
93     buffers = []
94     render_pipeline_desc.render_data.attributes = list(filter(lambda x: x is not None,
render_pipeline_desc.render_data.attributes))
95     for index, attribute in enumerate(render_pipeline_desc.render_data.attributes):
96         attribute_format = wgpu.VertexFormat.float32
97         vertex_length = len(attribute[index])
98         match vertex_length:
99             case 1:
100                 attribute_format = wgpu.VertexFormat.float32
101             case 2:
102                 attribute_format = wgpu.VertexFormat.float32x2
103             case 3:
104                 attribute_format = wgpu.VertexFormat.float32x3
105             case 4:
106                 attribute_format = wgpu.VertexFormat.float32x4
107
108     buffers.append({
109         "array_stride": 4 * vertex_length,
110         "step_mode": wgpu.VertexStepMode.vertex,
111         "attributes": [
112             {
113                 "format": attribute_format,
114                 "offset": 0,
115                 "shader_location": index,
116             }
117         ],
118     })
119
120     depth_stencil = None
121     if render_pipeline_desc.material_instance.descriptor.depth_enabled:

```

```

122         depth_stencil={
123             "format": render_pipeline_desc.material_instance.descriptor.depth_format,
124             "depth_write_enabled": render_pipeline_desc.material_instance.descriptor.
depth_write_enabled,
125             "depth_compare": render_pipeline_desc.material_instance.descriptor.depth_compare,
126             "stencil_front": {
127                 "compare": wgpu.CompareFunction.always,
128                 "fail_op": wgpu.StencilOperation.keep,
129                 "depth_fail_op": wgpu.StencilOperation.keep,
130                 "pass_op": wgpu.StencilOperation.keep,
131             },
132             "stencil_back": {
133                 "compare": wgpu.CompareFunction.always,
134                 "fail_op": wgpu.StencilOperation.keep,
135                 "depth_fail_op": wgpu.StencilOperation.keep,
136                 "pass_op": wgpu.StencilOperation.keep,
137             },
138             "stencil_read_mask": 0,
139             "stencil_write_mask": 0,
140             "depth_bias": 0,
141             "depth_bias_slope_scale": 0.0,
142             "depth_bias_clamp": 0.0,
143         }
144
145     render_pipeline : wgpu.GPURenderPipeline = cls.instance.device.create_render_pipeline(
146         layout=render_pipeline_desc.material_instance.pipeline_layout,
147         vertex={
148             "module": render_pipeline_desc.material_instance.shader_module,
149             "entry_point": "vs_main",
150             "buffers": buffers
151         },
152         primitive={
153             "topology": render_pipeline_desc.material_instance.descriptor.primitive,
154             "front_face": render_pipeline_desc.material_instance.descriptor.front_face,
155             "cull_mode": render_pipeline_desc.material_instance.descriptor.cull_mode,
156         },
157         depth_stencil=depth_stencil,
158         multisample=None,
159         fragment={
160             "module": render_pipeline_desc.material_instance.shader_module,
161             "entry_point": "fs_main",
162             "targets": [
163                 {
164                     "format": cls.instance.render_texture_format,
165                     "blend": {
166                         "color": (
167                             wgpu.BlendFactor.one,
168                             wgpu.BlendFactor.zero,
169                             wgpu.BlendOperation.add,
170                         ),
171                         "alpha": (
172                             wgpu.BlendFactor.one,
173                             wgpu.BlendFactor.zero,
174                             wgpu.BlendOperation.add,
175                         ),
176                     },
177                 },
178             ],
179         },
180     )
181     render_pipeline_desc.render_data.render_pipeline = render_pipeline
182
183     def begin_render_pass(cls, render_pass_desc: RenderPassDescription):
184         assert cls.instance.current_render_pass == None, 'Previous render pass not ended yet, call
end_render_pass() first before starting a new one.'
185         cls.instance.command_encoder = cls.instance.device.create_command_encoder()
186
187         color_attachments = []
188
189         if render_pass_desc.color_attachment:
190             for color_attachment in render_pass_desc.color_attachments:
191                 color_attachments.append({
192                     "view": color_attachment.view,
193                     "resolve_target": color_attachment.resolve_target,
194                     "clear_value": color_attachment.clear_value,
195                     "load_op": color_attachment.color_load_op,
196                     "store_op": color_attachment.color_store_op,
197                 })
198
199         depth_stencil_attachment = None
200
201         if render_pass_desc.depth_stencil_attachment:
202             if render_pass_desc.depth_texture_view == None:
203                 depth_texture : wgpu.GPUTexture = cls.instance.device.create_texture(

```

```

204         label="depth_texture",
205         size=[cls.instance.current_texture.width, cls.instance.current_texture.height,
206
207         mip_level_count=1,
208         sample_count=1,
209         dimension="2d",
210         format=wgpu.TextureFormat.depth24plus,
211         usage=wgpu.TextureUsage.RENDER_ATTACHMENT
212     )
213
214     cls.instance.depth_texture_view = depth_texture.create_view(
215         label="depth_texture_view",
216         format=wgpu.TextureFormat.depth24plus,
217         dimension="2d",
218         aspect=wgpu.TextureAspect.depth_only,
219         base_mip_level=0,
220         mip_level_count=1,
221         base_array_layer=0,
222         array_layer_count=1,
223     )
224     render_pass_desc.depth_texture_view = cls.instance.depth_texture_view
225
226     depth_stencil_attachment = {
227         "view": render_pass_desc.depth_texture_view,
228         "depth_clear_value": render_pass_desc.depth_clear_value,
229         "depth_load_op": render_pass_desc.depth_load_op,
230         "depth_store_op": render_pass_desc.depth_store_op,
231         "depth_read_only": render_pass_desc.depth_read_only,
232         "stencil_clear_value": render_pass_desc.stencil_clear_value,
233         "stencil_load_op": render_pass_desc.stencil_load_op,
234         "stencil_store_op": render_pass_desc.stencil_store_op,
235         "stencil_read_only": render_pass_desc.stencil_read_only,
236     }
237
238     cls.instance.current_render_pass = cls.instance.command_encoder.begin_render_pass(
239         color_attachments=color_attachments,
240         depth_stencil_attachment=depth_stencil_attachment,
241     )
242
243     def end_render_pass(cls):
244         assert cls.instance.current_render_pass != None, 'Submitting commands to None render pass,
245         call begin_render_pass() first.'
246         cls.instance.current_render_pass.end()
247         cls.instance.current_render_pass = None
248
249     def set_pipeline(cls, render_data):
250         assert cls.instance.current_render_pass != None, 'Submitting commands to None render pass,
251         call begin_render_pass() first.'
252         cls.instance.current_render_pass.set_pipeline(render_data.render_pipeline)
253
254     def set_buffers(cls, render_data):
255         assert cls.instance.current_render_pass != None, 'Submitting commands to None render pass,
256         call begin_render_pass() first.'
257         if render_data.index_buffer != None:
258             cls.instance.current_render_pass.set_index_buffer(render_data.index_buffer, wgpu.
259             IndexFormat.uint32)
260
261         for index, buffer in enumerate(render_data.buffers):
262             cls.instance.current_render_pass.set_vertex_buffer(index, buffer)
263
264     def set_bind_groups(cls, material):
265         assert cls.instance.current_render_pass != None, 'Submitting commands to None render pass,
266         call begin_render_pass() first.'
267         for index, bind_group in enumerate(material.instance.bind_groups):
268             cls.instance.current_render_pass.set_bind_group(index, bind_group, [], 0, 1)
269
270     def write_buffer(cls, buffer, uniform_data, size=0):
271         cls.instance.device.queue.write_buffer(buffer, 0, uniform_data, 0, size)
272
273     def write_texture(cls, uniform_data):
274         cls.instance.device.queue.write_texture(
275             {
276                 "texture": uniform_data.texture,
277                 "mip_level": 0,
278                 "origin": (0, 0, 0)
279             },
280             uniform_data.data.image_bytes,
281             {
282                 "offset": 0,
283                 "bytes_per_row": uniform_data.data.width * 4,
284                 "rows_per_image": uniform_data.data.height
285             },
286             [uniform_data.data.width, uniform_data.data.height, uniform_data.descriptor.
287             array_layer_count]

```

```

281         )
282
283     def draw(cls, render_data, instance_count=1, first_instance=0):
284         assert cls.instance.current_render_pass != None, 'Submitting commands to None render pass,
           call begin_render_pass() first.'
285         cls.instance.current_render_pass.draw(len(render_data.attributes[0]), instance_count, 0,
           first_instance)
286
287     def draw_indexed(cls, render_data, instance_count=1, first_instance=0):
288         assert cls.instance.current_render_pass != None, 'Submitting commands to None render pass,
           call begin_render_pass() first.'
289         cls.instance.current_render_pass.draw_indexed(render_data.indices.size, instance_count, 0, 0,
           first_instance)
290
291     def get_device(cls) -> wgpu.GPUDevice:
292         return cls.instance.device
293
294     def get_command_encoder(cls) -> wgpu.GPUCommandEncoder:
295         return cls.instance.command_encoder
296
297     def get_current_texture(cls) -> wgpu.GPUTexture:
298         return cls.instance.current_texture
299
300     def get_render_texture_format(cls) -> wgpu.TextureFormat:
301         return cls.instance.render_texture_format
302
303     def get_depth_texture_view(cls) -> wgpu.GPUTextureView:
304         return cls.instance.depth_texture_view
305
306     def set_clear_color(cls, clear_color: glm.vec4):
307         cls.instance.clear_color = clear_color
308
309     def set_shadows_enabled(cls, shadows_enabled: bool):
310         cls.instance.shadows_enabled = shadows_enabled
311
312     def get_shadows_enabled(cls):
313         return cls.instance.shadows_enabled

```

Listing B.2: WebGPURenderer Class

Appendix C

Resource Handling Code

```
1 from pyGandalf.utilities.logger import logger
2 from pyGandalf.utilities.definitions import TEXTURES_PATH
3
4 import OpenGL.GL as gl
5 from PIL import Image
6
7 import os
8 from enum import Enum
9 from pathlib import Path
10 from dataclasses import dataclass
11
12 class TextureDimension(Enum):
13     D2 = 0
14     D3 = 1
15     CUBE = 2
16
17 @dataclass
18 class TextureDescriptor:
19     flip: bool = False
20     dimation: TextureDimension = TextureDimension.D2
21     internal_format: gl.Constant = gl.GL_RGBA8
22     format: gl.Constant = gl.GL_RGBA
23     type: gl.Constant = gl.GL_UNSIGNED_BYTE
24     wrap_s: gl.Constant = gl.GL_REPEAT
25     wrap_t: gl.Constant = gl.GL_REPEAT
26     min_filter: gl.Constant = gl.GL_LINEAR
27     max_filter: gl.Constant = gl.GL_LINEAR
28
29 @dataclass
30 class TextureData:
31     path: Path | list[Path] = None
32     image_bytes: bytes = None
33     width: int = 0
34     height: int = 0
35
36 class TextureInstance:
37     def __init__(self, id, slot, name: str, data: TextureData, descriptor: TextureDescriptor):
38         self.id = id
39         self.slot = slot
40         self.name = name
41         self.data = data
42         self.descriptor = descriptor
43
44 class OpenGLTextureLib(object):
45     """A class that is used to build textures and get texture data.
46     """
47     def __new__(cls):
48         if not hasattr(cls, 'instance'):
49             cls.instance = super(OpenGLTextureLib, cls).__new__(cls)
50             cls.instance.textures = {}
51             cls.instance.slots = {}
52             cls.instance.current_slot = 0
53         return cls.instance
54
55     def build(cls, name: str, data: TextureData, descriptor: TextureDescriptor = TextureDescriptor()):
56         :
57         """Builds a new texture (if one does not already exists with that name) and returns its slot.
```

```

58     Args:
59         name (str): The name of the texture.
60         data (TextureData): The data of the texture. You can either give a path (or list of paths
        if cubemap) or the byte data to use when creating the texture and the width and height.
61         descriptor (TextureDescriptor, optional): The description of the texture, which consists
        of various options and flags.
62
63     Returns:
64         int: The texture slot.
65     """
66     if cls.instance.textures.get(name) != None:
67         return cls.instance.textures[name].slot
68
69     img_bytes = data.image_bytes
70     img = None
71
72     if data.path is None or type(data.path) is not list:
73         assert descriptor.dimension == TextureDimension.D2 or descriptor.dimension ==
        TextureDimension.D3, "Single texture path only supported for 2d or 3d textures dimensions"
74
75         if data.path is not None:
76             img = Image.open(data.path)
77             if descriptor.flip:
78                 img = img.transpose(Image.FLIP_TOP_BOTTOM)
79             img_bytes = img.convert("RGBA").tobytes("raw", "RGBA", 0, -1)
80
81         texture_id = gl.glGenTextures(1)
82         gl.glBindTexture(gl.GL_TEXTURE_2D, texture_id)
83
84         gl.glTexParameterf(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_WRAP_S, descriptor.wrap_s)
85         gl.glTexParameterf(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_WRAP_T, descriptor.wrap_t)
86         gl.glTexParameterf(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_MIN_FILTER, descriptor.min_filter)
87         gl.glTexParameterf(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_MAG_FILTER, descriptor.min_filter)
88
89         gl.glTexParameterfv(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_BORDER_COLOR, [1.0, 1.0, 1.0, 1.0])
90
91         gl.glTexImage2D(
92             gl.GL_TEXTURE_2D,
93             0,
94             descriptor.internal_format,
95             img.width if img is not None else data.width, # Width
96             img.height if img is not None else data.height, # Height
97             0,
98             descriptor.format,
99             descriptor.type,
100             img_bytes
101         )
102
103         gl.glGenerateMipmap(gl.GL_TEXTURE_2D)
104
105         gl.glBindTexture(gl.GL_TEXTURE_2D, 0)
106
107         if data.path is not None:
108             data.path = Path(os.path.realpath(data.path, TEXTURES_PATH))
109
110         if img is not None:
111             data.width = img.width
112             data.height = img.height
113
114         texture_instance : TextureInstance = TextureInstance(texture_id, cls.instance.
        current_slot, name, data, descriptor)
115         cls.instance.textures[name] = texture_instance
116
117         cls.instance.current_slot += 1
118
119         return texture_instance.slot
120     elif type(data.path) is list:
121         assert descriptor.dimension == TextureDimension.CUBE, "Multiple texture paths are only
        supported for cube dimension type"
122
123         texture_id = gl.glGenTextures(1)
124         gl.glBindTexture(gl.GL_TEXTURE_CUBE_MAP, texture_id)
125
126         for i, p in enumerate(data.path):
127             img = Image.open(p)
128             if descriptor.flip:
129                 img = img.transpose(Image.FLIP_TOP_BOTTOM)
130             img_bytes = img.convert("RGBA").tobytes("raw", "RGBA", 0, -1)
131
132             gl.glTexImage2D(
133                 gl.GL_TEXTURE_CUBE_MAP_POSITIVE_X + i,
134                 0, descriptor.internal_format, img.width, img.height, 0, descriptor.format,
        descriptor.type, img_bytes
135             )

```

```

136         if data.path is not None:
137             data.path[i] = Path(os.path.relpath(p, TEXTURES_PATH))
138
139
140         gl.glTexParameterf(gl.GL_TEXTURE_CUBE_MAP, gl.GL_TEXTURE_MAG_FILTER, gl.GL_LINEAR)
141         gl.glTexParameterf(gl.GL_TEXTURE_CUBE_MAP, gl.GL_TEXTURE_MIN_FILTER, gl.GL_LINEAR)
142         gl.glTexParameterf(gl.GL_TEXTURE_CUBE_MAP, gl.GL_TEXTURE_WRAP_S, gl.GL_CLAMP_TO_EDGE)
143         gl.glTexParameterf(gl.GL_TEXTURE_CUBE_MAP, gl.GL_TEXTURE_WRAP_T, gl.GL_CLAMP_TO_EDGE)
144         gl.glTexParameterf(gl.GL_TEXTURE_CUBE_MAP, gl.GL_TEXTURE_WRAP_R, gl.GL_CLAMP_TO_EDGE)
145
146         gl.glBindTexture(gl.GL_TEXTURE_CUBE_MAP, 0)
147
148         texture_instance : TextureInstance = TextureInstance(texture_id, cls.instance.
current_slot, name, data, descriptor)
149         cls.instance.textures[name] = texture_instance
150
151         cls.instance.current_slot += 1
152
153         return texture_instance.slot
154
155 def get_id(cls, name: str):
156     """Returns the renderer id of texture with the given name.
157
158     Args:
159         name (str): The name of the texture.
160
161     Returns:
162         _type_: The renderer id.
163     """
164     return cls.instance.textures.get(name).id
165
166 def get_slot(cls, name: str):
167     """Returns the slot of texture with the given name.
168
169     Args:
170         name (str): The name of the texture.
171
172     Returns:
173         float: The texture slot.
174     """
175     texture: TextureInstance = cls.instance.textures.get(name)
176
177     if texture == None:
178         logger.error(f"No such texture exists: '{name}'")
179         return
180
181     return float(texture.slot)
182
183 def bind(cls, name: str):
184     """Binds the texture with the given name.
185
186     Args:
187         name (str): The name of the texture to bind.
188     """
189     texture: TextureInstance = cls.instance.textures.get(name)
190     target = None
191
192     if texture == None:
193         logger.error(f"No such texture exists: '{name}'")
194         return
195
196     match texture.descriptor.dimension:
197         case TextureDimension.D2:
198             target = gl.GL_TEXTURE_2D
199         case TextureDimension.CUBE:
200             target = gl.GL_TEXTURE_CUBE_MAP
201
202     gl.glActiveTexture(texture.slot + gl.GL_TEXTURE0)
203     gl.glBindTexture(target, cls.instance.get_id(name))
204
205 def unbind(cls, name: str):
206     """Unbinds the texture with the given name.
207
208     Args:
209         name (str): The name of the texture to unbind.
210     """
211     texture: TextureInstance = cls.instance.textures.get(name)
212     target = None
213
214     if texture == None:
215         logger.error(f"No such texture exists: '{name}'")
216         return
217
218     match texture.descriptor.dimension:

```

```

219         case TextureDimension.D2:
220             target = gl.GL_TEXTURE_2D
221         case TextureDimension.CUBE:
222             target = gl.GL_TEXTURE_CUBE_MAP
223
224     gl.glActiveTexture(texture.slot + gl.GL_TEXTURE0)
225     gl.glBindTexture(target, 0)
226
227     def bind_textures(cls):
228         """Binds all the available textures.
229         """
230         for texture in cls.instance.textures.values():
231             target = None
232             match texture.descriptor.dimension:
233                 case TextureDimension.D2:
234                     target = gl.GL_TEXTURE_2D
235                 case TextureDimension.CUBE:
236                     target = gl.GL_TEXTURE_CUBE_MAP
237             gl.glActiveTexture(texture.slot + gl.GL_TEXTURE0)
238             gl.glBindTexture(target, texture.id)
239
240     def unbind_textures(cls):
241         """Unbinds all the available textures.
242         """
243         for texture in cls.instance.textures.values():
244             target = None
245             match texture.descriptor.dimension:
246                 case TextureDimension.D2:
247                     target = gl.GL_TEXTURE_2D
248                 case TextureDimension.CUBE:
249                     target = gl.GL_TEXTURE_CUBE_MAP
250             gl.glActiveTexture(texture.slot + gl.GL_TEXTURE0)
251             gl.glBindTexture(target, 0)
252
253     def get_textures(cls) -> dict[str, TextureInstance]:
254         """Returns a dictionary the holds all the textures. As the key is the name of the texture, as
255         the value is the texture data.
256
257         Returns:
258             dict[str, TextureInstance]: A dictionary the holds all the textures.
259         """
260         return cls.instance.textures

```

Listing C.1: OpenGLTextureLib Class

```

1  from pyGandalf.utilities.definitions import SHADERS_PATH
2
3  import OpenGL.GL as gl
4
5  import os
6  import re
7  from pathlib import Path
8
9  class ShaderData:
10     def __init__(self, shader_program, name: str, vs_path: Path, fs_path: Path, gs_path: Path,
11                 tcs_path: Path, tes_path: Path, vs_code: str, fs_code: str, gs_code: str, tcs_code: str,
12                 tes_code: str):
13         self.name = name
14         self.vs_path = vs_path
15         self.fs_path = fs_path
16         self.gs_path = gs_path
17         self.tcs_path = tcs_path
18         self.tes_path = tes_path
19         self.shader_program = shader_program
20         self.vs_code = vs_code
21         self.fs_code = fs_code
22         self.gs_code = gs_code
23         self.tcs_code = tcs_code
24         self.tes_code = tes_code
25
26     class OpenGLShaderLib(object):
27         def __new__(cls):
28             if not hasattr(cls, 'instance'):
29                 cls.instance = super(OpenGLShaderLib, cls).__new__(cls)
30                 cls.instance.shaders : dict[str, ShaderData] = {} # type: ignore
31             return cls.instance
32
33         def compile_shader(cls, source, shader_type):
34             shader = gl.glCreateShader(shader_type)
35             gl.glShaderSource(shader, source)
36             gl.glCompileShader(shader)
37
38             if not gl.glGetShaderiv(shader, gl.GL_COMPILE_STATUS):

```

```

37         raise RuntimeError(gl.glGetShaderInfoLog(shader).decode('utf-8'))
38
39     return shader
40
41 def create_shader_program(cls, vertex_shader_code, fragment_shader_code, geometry_shader_code=
42     None, tessellation_control_shader_code=None, tessellation_evaluation_shader_code=None):
43     vertex_shader = cls.instance.compile_shader(vertex_shader_code, gl.GL_VERTEX_SHADER)
44     fragment_shader = cls.instance.compile_shader(fragment_shader_code, gl.GL_FRAGMENT_SHADER)
45
46     if geometry_shader_code != None:
47         geometry_shader = cls.instance.compile_shader(geometry_shader_code, gl.GL_GEOMETRY_SHADER)
48
49     if tessellation_control_shader_code != None:
50         tessellation_control_shader = cls.instance.compile_shader(
51             tessellation_control_shader_code, gl.GL_TESS_CONTROL_SHADER)
52     if tessellation_evaluation_shader_code != None:
53         tessellation_evaluation_shader = cls.instance.compile_shader(
54             tessellation_evaluation_shader_code, gl.GL_TESS_EVALUATION_SHADER)
55
56     shader_program = gl.glCreateProgram()
57     gl.glAttachShader(shader_program, vertex_shader)
58     gl.glAttachShader(shader_program, fragment_shader)
59     if geometry_shader_code != None:
60         gl.glAttachShader(shader_program, geometry_shader)
61     if tessellation_control_shader_code != None:
62         gl.glAttachShader(shader_program, tessellation_control_shader)
63     if tessellation_evaluation_shader_code != None:
64         gl.glAttachShader(shader_program, tessellation_evaluation_shader)
65     gl.glLinkProgram(shader_program)
66
67     if not gl.glGetProgramiv(shader_program, gl.GL_LINK_STATUS):
68         raise RuntimeError(gl.glGetProgramInfoLog(shader_program).decode('utf-8'))
69
70     gl.glDeleteShader(vertex_shader)
71     gl.glDeleteShader(fragment_shader)
72     if geometry_shader_code != None:
73         gl.glDeleteShader(geometry_shader)
74     if tessellation_control_shader_code != None:
75         gl.glDeleteShader(tessellation_control_shader)
76     if tessellation_evaluation_shader_code != None:
77         gl.glDeleteShader(tessellation_evaluation_shader)
78
79     return shader_program
80
81 def build(cls, name: str, vs_path: Path, fs_path: Path, gs_path: Path=None, tcs_path: Path=None,
82     tes_path: Path=None) -> int:
83     """Builds a new shader (if one does not already exists with that name) and returns its shader
84     program.
85
86     Args:
87         name (str): The name of the shader
88         vs_path (Path): The path to the vertex shader code
89         fs_path (Path): The path to the fragment shader code
90         gs_path (Path): The path to the geometry shader code
91         tcs_path (Path): The path to the tessellation control shader code
92         tes_path (Path): The path to the tessellation evaluation shader code
93
94     Returns:
95         int: The shader program
96     """
97     if cls.instance.shaders.get(name) != None:
98         return cls.instance.shaders.get(name).shader_program
99
100     vs_code = cls.instance.load_from_file(vs_path)
101     fs_code = cls.instance.load_from_file(fs_path)
102     gs_code = None
103     tcs_code = None
104     tes_code = None
105
106     if gs_path != None:
107         gs_code = cls.instance.load_from_file(gs_path)
108
109     if tcs_path != None:
110         tcs_code = cls.instance.load_from_file(tcs_path)
111
112     if tes_path != None:
113         tes_code = cls.instance.load_from_file(tes_path)
114
115     vs_rel_path = Path(os.path.relpath(vs_path, SHADERS_PATH))
116     fs_rel_path = Path(os.path.relpath(fs_path, SHADERS_PATH))
117     gs_rel_path = None
118     tcs_rel_path = None
119     tes_rel_path = None
120
121     if gs_path != None:

```

```

115         gs_rel_path = Path(os.path.relpath(gs_path, SHADERS_PATH))
116         if tcs_path != None:
117             tcs_rel_path = Path(os.path.relpath(tcs_path, SHADERS_PATH))
118         if tcs_path != None:
119             tes_rel_path = Path(os.path.relpath(tes_path, SHADERS_PATH))
120
121         shader_program = cls.instance.create_shader_program(vs_code, fs_code, gs_code, tcs_code,
122         tes_code)
123         cls.instance.shaders[name] = ShaderData(shader_program, name, vs_rel_path, fs_rel_path,
124         gs_rel_path, tcs_rel_path, tes_rel_path, vs_code, fs_code, gs_code, tcs_code, tes_code)
125
126         return shader_program
127
128 def parse(cls, shader_code: str) -> dict:
129     """Parses the provided shader code and identifies all the uniforms along with their types.
130
131     Args:
132         shader_code (str): The source code of the shader to parse.
133
134     Returns:
135         dict: A dictionary holding the uniform name as a key and the uniform type as a value
136     """
137     uniform_pattern = re.compile(r'uniform\s+(\w+)\s+(\w+)\s*')
138     uniform_buffer_pattern = re.compile(r'layout\s*(\s*std140\s*)\s*uniform\s+(\w+)\s*{([~]**)\s*};')
139     uniform_array_pattern = re.compile(r'uniform\s+(\w+)\s+(\w+)\s*\s*{([~]**)\s*(\d+)\s*\}\s*')
140
141     uniforms = {}
142     matches = uniform_pattern.findall(shader_code)
143     for match in matches:
144         uniforms[match[1]] = match[2]
145
146     array_matches = uniform_array_pattern.findall(shader_code)
147     for match in array_matches:
148         array_type = match[0]
149         array_name = match[1]
150         array_size = int(match[2])
151         uniforms[array_name] = f'{array_type}[{array_size}]'
152
153     matches = uniform_buffer_pattern.findall(shader_code)
154     for match in matches:
155         buffer_name = match[0]
156         buffer_content = match[1]
157         buffer_uniforms = {}
158         buffer_matches = uniform_pattern.findall(buffer_content)
159         for buffer_match in buffer_matches:
160             buffer_uniforms[buffer_match[1]] = buffer_match[2]
161         uniforms[buffer_name] = buffer_uniforms
162
163     return uniforms
164
165 def get(cls, name: str) -> ShaderData:
166     """Gets the shader data of the given shader name.
167
168     Args:
169         name (str): The name of the shader to get its data.
170
171     Returns:
172         ShaderData: the shader data of the given shader name.
173     """
174     return cls.instance.shaders.get(name)
175
176 def get_shaders(cls) -> dict[str, ShaderData]:
177     """Returns the dictionary that holds all the shader data
178
179     Returns:
180         dict[str, ShaderData]: the dictionary that holds all the shader data
181     """
182     return cls.instance.shaders
183
184 def load_from_file(cls, path_to_source):
185     """
186     Returns the file contents as a string.
187     """
188     with open(path_to_source) as file:
189         return file.read()
190
191 def clean(cls):
192     cls.instance.shaders.clear()

```

Listing C.2: OpenGLShaderLib Class

```

1 from pyGandalf.utilities.opengl_shader_lib import OpenGLShaderLib
2 from pyGandalf.utilities.logger import logger
3
4 import glm
5 import numpy as np
6 import OpenGL.GL as gl
7
8 import re
9 from dataclasses import dataclass
10
11 class MaterialInstance:
12     def __init__(self, name, data, descriptor, shader_program, shader_params = []):
13         self.name = name
14         self.data = data
15         self.descriptor = descriptor
16         self.shader_program = shader_program
17         self.shader_params = shader_params
18
19     def has_uniform(self, uniform_name: str) -> bool:
20         """Returns True if the material has the uniform with the given name, otherwise False.
21
22         Args:
23             uniform_name (str): The name of the uniform.
24
25         Returns:
26             bool: True if the material has the uniform with the given name, otherwise False.
27         """
28         return gl.glGetUniformLocation(self.shader_program, uniform_name) != -1
29
30     def set_uniform(self, uniform_name: str, uniform_data):
31         """Sets the uniform with the provided name (if valid), with the provided data.
32
33         Args:
34             uniform_name (str): The name of the uniform to set.
35             uniform_data (Any): The new data for the uniform.
36         """
37         uniform_location = gl.glGetUniformLocation(self.shader_program, uniform_name)
38         if uniform_location != -1:
39             self.update_uniform(uniform_location, uniform_name, uniform_data)
40         else:
41             self.uniform_not_found(uniform_name)
42
43     def update_uniform(self, uniform_location: int, uniform_name: str, uniform_data):
44         """Updates the uniform at specified location and the given name with the given data.
45
46         Args:
47             uniform_location (int): The uniform location.
48             uniform_name (str): The uniform name.
49             uniform_data (Any): The new uniform data.
50         """
51         uniform_type = self.shader_params[uniform_name]
52         match uniform_type:
53             case 'float':
54                 assert isinstance(uniform_data, float), f"Uniform type with name: {uniform_name} is
not a floating point number"
55                 gl.glUniform1f(uniform_location, uniform_data)
56                 return
57             case 'int':
58                 assert isinstance(uniform_data, int), f"Uniform type with name: {uniform_name} is not
an integer number"
59                 gl.glUniform1i(uniform_location, uniform_data)
60                 return
61             case 'sampler2D':
62                 assert isinstance(uniform_data, int), f"Uniform type with name: {uniform_name} is not
an integer number"
63                 gl.glUniform1i(uniform_location, uniform_data)
64                 return
65             case 'samplerCube':
66                 assert isinstance(uniform_data, int), f"Uniform type with name: {uniform_name} is not
an integer number"
67                 gl.glUniform1i(uniform_location, uniform_data)
68                 return
69             case 'vec2':
70                 assert isinstance(uniform_data, glm.vec2), f"Uniform type with name: {uniform_name}
is not a 2x1 glm array of float32 type"
71                 gl.glUniform2f(uniform_location, uniform_data.x, uniform_data.y)
72                 return
73             case 'vec3':
74                 assert isinstance(uniform_data, glm.vec3), f"Uniform type with name: {uniform_name}
is not a 3x1 glm array of float32 type"
75                 gl.glUniform3f(uniform_location, uniform_data.x, uniform_data.y, uniform_data.z)
76                 return
77             case 'vec4':

```

```

78         assert isinstance(uniform_data, glm.vec4), f"Uniform type with name: {uniform_name}
79         is not a 4x1 glm array of float32 type"
80         gl.glUniform4f(uniform_location, uniform_data.x, uniform_data.y, uniform_data.z,
81         uniform_data.w)
82         return
83     case 'ivec2':
84         assert isinstance(uniform_data, glm.ivec2), f"Uniform type with name: {uniform_name}
85         is not a 2x1 glm array of int32 type"
86         gl.glUniform2i(uniform_location, uniform_data.x, uniform_data.y)
87         return
88     case 'ivec3':
89         assert isinstance(uniform_data, glm.ivec3), f"Uniform type with name: {uniform_name}
90         is not a 3x1 glm array of int32 type"
91         gl.glUniform3i(uniform_location, uniform_data.x, uniform_data.y, uniform_data.z)
92         return
93     case 'ivec4':
94         assert isinstance(uniform_data, glm.ivec4), f"Uniform type with name: {uniform_name}
95         is not a 4x1 glm array of int32 type"
96         gl.glUniform4i(uniform_location, uniform_data.x, uniform_data.y, uniform_data.z,
97         uniform_data.w)
98         return
99     case 'uvec2':
100         assert isinstance(uniform_data, glm.uvec2), f"Uniform type with name: {uniform_name}
101         is not a 2x1 glm array of uint32 type"
102         gl.glUniform2ui(uniform_location, uniform_data.x, uniform_data.y)
103         return
104     case 'uvec3':
105         assert isinstance(uniform_data, glm.uvec3), f"Uniform type with name: {uniform_name}
106         is not a 3x1 glm array of uint32 type"
107         gl.glUniform3ui(uniform_location, uniform_data.x, uniform_data.y, uniform_data.z)
108         return
109     case 'uvec4':
110         assert isinstance(uniform_data, glm.uvec4), f"Uniform type with name: {uniform_name}
111         is not a 4x1 glm array of uint32 type"
112         gl.glUniform4ui(uniform_location, uniform_data.x, uniform_data.y, uniform_data.z,
113         uniform_data.w)
114         return
115     case 'dvec2':
116         assert isinstance(uniform_data, glm.dvec2), f"Uniform type with name: {uniform_name}
117         is not a 2x1 glm array of double type"
118         gl.glUniform2d(uniform_location, uniform_data.x, uniform_data.y)
119         return
120     case 'dvec3':
121         assert isinstance(uniform_data, glm.dvec3), f"Uniform type with name: {uniform_name}
122         is not a 3x1 glm array of double type"
123         gl.glUniform3d(uniform_location, uniform_data.x, uniform_data.y, uniform_data.z)
124         return
125     case 'dvec4':
126         assert isinstance(uniform_data, glm.dvec4), f"Uniform type with name: {uniform_name}
127         is not a 4x1 glm array of double type"
128         gl.glUniform4d(uniform_location, uniform_data.x, uniform_data.y, uniform_data.z,
129         uniform_data.w)
130         return
131     case 'mat2':
132         assert isinstance(uniform_data, glm.mat2), f"Uniform type with name: {uniform_name}
133         is not a 2x2 glm matrix type"
134         gl.glUniformMatrix2fv(uniform_location, 1, gl.GL_FALSE, glm.value_ptr(uniform_data))
135         return
136     case 'mat3':
137         assert isinstance(uniform_data, glm.mat3), f"Uniform type with name: {uniform_name}
138         is not a 3x3 glm matrix type"
139         gl.glUniformMatrix3fv(uniform_location, 1, gl.GL_FALSE, glm.value_ptr(uniform_data))
140         return
141     case 'mat4':
142         assert isinstance(uniform_data, glm.mat4), f"Uniform type with name: {uniform_name}
143         is not a 4x4 glm matrix type"
144         gl.glUniformMatrix4fv(uniform_location, 1, gl.GL_FALSE, glm.value_ptr(uniform_data))
145         return
146
147     if 'int[' in uniform_type or 'sampler2D[' in uniform_type:
148         number_pattern = re.compile(r'\[(\d+)\]')
149         match = number_pattern.search(uniform_type)
150         if match:
151             count = int(match.group(1))
152             assert len(uniform_data) <= count and isinstance(uniform_data[0], np.int32), f"
153             Uniform type with name: {uniform_name} is not a NumPy array of {count} integers"
154             gl.glUniform1iv(uniform_location, len(uniform_data), uniform_data)
155         elif 'float[' in uniform_type:
156             number_pattern = re.compile(r'\[(\d+)\]')
157             match = number_pattern.search(uniform_type)
158             if match:
159                 count = int(match.group(1))
160                 assert len(uniform_data) <= count and isinstance(uniform_data[0], np.float32), f"
161                 Uniform type with name: {uniform_name} is not a NumPy array of {count} floats"

```

```

143         gl.glUniform1fv(uniform_location, len(uniform_data), uniform_data)
144     elif 'mat2[' in uniform_type:
145         number_pattern = re.compile(r'\[(\d+)\]\>')
146         match = number_pattern.search(uniform_type)
147         if match:
148             count = int(match.group(1))
149             assert len(uniform_data) <= count and isinstance(uniform_data[0], glm.mat2), f"
Uniform type with name: {uniform_name} is not a NumPy array of {count} 2x2 glm matrices"
150         gl.glUniformMatrix2fv(uniform_location, len(uniform_data), gl.GL_FALSE, uniform_data.
ptr)
151     elif 'mat3[' in uniform_type:
152         number_pattern = re.compile(r'\[(\d+)\]\>')
153         match = number_pattern.search(uniform_type)
154         if match:
155             count = int(match.group(1))
156             assert len(uniform_data) <= count and isinstance(uniform_data[0], glm.mat3), f"
Uniform type with name: {uniform_name} is not an array of {count} 3x3 glm matrices"
157         gl.glUniformMatrix3fv(uniform_location, len(uniform_data), gl.GL_FALSE, uniform_data.
ptr)
158     elif 'mat4[' in uniform_type:
159         number_pattern = re.compile(r'\[(\d+)\]\>')
160         match = number_pattern.search(uniform_type)
161         if match:
162             count = int(match.group(1))
163             assert len(uniform_data) <= count and isinstance(uniform_data[0], glm.mat4), f"
Uniform type with name: {uniform_name} is not an array of {count} 4x4 glm matrices"
164         gl.glUniformMatrix4fv(uniform_location, len(uniform_data), gl.GL_FALSE, uniform_data.
ptr)
165     elif 'vec2[' in uniform_type:
166         number_pattern = re.compile(r'\[(\d+)\]\>')
167         match = number_pattern.search(uniform_type)
168         if match:
169             count = int(match.group(1))
170             assert len(uniform_data) <= count and isinstance(uniform_data[0], glm.vec2), f"
Uniform type with name: {uniform_name} is not an array of {count} 2x1 glm array of float"
171         gl.glUniform2fv(uniform_location, len(uniform_data), uniform_data.ptr)
172     elif 'vec3[' in uniform_type:
173         number_pattern = re.compile(r'\[(\d+)\]\>')
174         match = number_pattern.search(uniform_type)
175         if match:
176             count = int(match.group(1))
177             assert len(uniform_data) <= count and isinstance(uniform_data[0], glm.vec3), f"
Uniform type with name: {uniform_name} is not an array of {count} 3x1 glm array of float"
178         gl.glUniform3fv(uniform_location, len(uniform_data), uniform_data.ptr)
179     elif 'vec4[' in uniform_type:
180         number_pattern = re.compile(r'\[(\d+)\]\>')
181         match = number_pattern.search(uniform_type)
182         if match:
183             count = int(match.group(1))
184             assert len(uniform_data) <= count and isinstance(uniform_data[0], glm.vec4), f"
Uniform type with name: {uniform_name} is not an array of {count} 4x1 glm array of float"
185         gl.glUniform4fv(uniform_location, len(uniform_data), uniform_data.ptr)
186
187 def uniform_not_found(self, uniform_name: str):
188     """Prints a message stating that the uniform with the provided name was not found.
189
190     Args:
191         uniform_name (str): The name of the uniform that was not found.
192     """
193     logger.debug(f'Could not find {uniform_name} uniform for material: {self.name}!\n These are
the available uniforms for shader with id {self.shader_program}:\n {self.shader_params}')
194
195 def print_available_uniforms(self):
196     """Prints a message with all the available uniforms of the material.
197
198     """
199     logger.log(f'Available uniforms for material: {self.name} and shader id {self.shader_program
}:\n {self.shader_params}')
200
201 @dataclass
202 class MaterialDescriptor:
203     primitive: gl.Constant = gl.GL_TRIANGLES
204     cast_shadows = True
205     cull_enabled: bool = True
206     cull_face: gl.Constant = gl.GL_BACK
207     patch_resolution: int = 20
208     vertices_per_patch: int = 20
209     depth_enabled: bool = True
210     depth_func: gl.Constant = gl.GL_LEQUAL
211     depth_mask: gl.Constant = gl.GL_TRUE
212     blend_enabled: bool = True
213     blend_func_source: gl.Constant = gl.GL_SRC_ALPHA
214     blend_func_destination: gl.Constant = gl.GL_ONE_MINUS_SRC_ALPHA
215     blend_equation: gl.Constant = gl.GL_FUNC_ADD

```

```

216 class MaterialData:
217     def __init__(self, base_template: str, textures: list[str], color: glm.vec4 = glm.vec4(1.0, 1.0,
218         1.0, 1.0), glossiness = 3.0):
219         self.base_template = base_template
220         self.color = color
221         self.textures = textures
222         self.glossiness = glossiness
223
224     def __eq__(self, other):
225         if self.base_template != other.base_template:
226             return False
227         if self.color != other.color:
228             return False
229         if self.glossiness != other.glossiness:
230             return False
231         if len(self.textures) != len(other.textures):
232             return False
233         for i in range(len(self.textures)):
234             if (self.textures[i] != other.textures[i]):
235                 return False
236         return True
237
238     def __hash__(self):
239         return hash((self.base_template, self.color.r, self.color.g, self.color.b, self.color.a, len(
240             self.textures), self.glossiness, tuple(texture for texture in self.textures)))
241
242 class OpenGLMaterialLib(object):
243     def __new__(cls):
244         if not hasattr(cls, 'instance'):
245             cls.instance = super(OpenGLMaterialLib, cls).__new__(cls)
246             cls.instance.cached_materials: dict[MaterialData, MaterialInstance] = {} # type: ignore
247             cls.instance.materials: dict[str, MaterialInstance] = {} # type: ignore
248         return cls.instance
249
250     def build(cls, name: str, data: MaterialData, descriptor: MaterialDescriptor=MaterialDescriptor()
251         ) -> MaterialInstance:
252         """Builds a new material (if one does not already exists with that data) and returns its
253         instance.
254
255         Args:
256             name (str): The name of the material.
257             data (MaterialData): The data of the material, which consists of shader, color, textures
258             and glossiness.
259             descriptor (MaterialDescriptor, optional): The description of the material, which
260             consists of various options and flags.
261
262         Returns:
263             MaterialInstance: The material instance.
264         """
265         if cls.instance.cached_materials.get(data) != None:
266             material = cls.instance.cached_materials[data]
267             cls.instance.materials[name] = material
268             return material
269
270         shader_data = OpenGLShaderLib().get(data.base_template)
271
272         if shader_data == None:
273             logger.error(f"No such material exists: '{data.base_template}'")
274             return None
275
276         shader_program = shader_data.shader_program
277         shader_params_vertex = OpenGLShaderLib().parse(shader_data.vs_code)
278         shader_params_fragment = OpenGLShaderLib().parse(shader_data.fs_code)
279         shader_params_geometry = {}
280         shader_params_tess_control = {}
281         shader_params_tess_eval = {}
282
283         if shader_data.gs_code != None:
284             shader_params_geometry = OpenGLShaderLib().parse(shader_data.gs_code)
285         if shader_data.tcs_code != None:
286             shader_params_tess_control = OpenGLShaderLib().parse(shader_data.tcs_code)
287         if shader_data.tes_code != None:
288             shader_params_tess_eval = OpenGLShaderLib().parse(shader_data.tes_code)
289
290         cls.instance.cached_materials[data] = MaterialInstance(name, data, descriptor, shader_program
291             , shader_params_vertex | shader_params_fragment | shader_params_geometry |
292             shader_params_tess_control | shader_params_tess_eval)
293         cls.instance.materials[name] = MaterialInstance(name, data, descriptor, shader_program,
294             shader_params_vertex | shader_params_fragment | shader_params_geometry |
295             shader_params_tess_control | shader_params_tess_eval)
296
297         return cls.instance.materials[name]
298
299     def get(cls, name: str) -> MaterialInstance:

```

```

290     """Returns the material instance with the given name.
291
292     Args:
293         name (str): The name of the material instance to get.
294
295     Returns:
296         MaterialInstance: The material instance with the given name.
297     """
298     return cls.instance.materials.get(name)
299
300 def get_textures(cls, name: str):
301     mat: MaterialInstance = cls.instance.materials[name]
302
303     textures = []
304     for uniform_name in mat.shader_params.keys():
305         uniform_type = mat.shader_params[uniform_name]
306         if 'sampler' in uniform_type:
307             textures.append(uniform_name)
308
309     return textures
310
311 def get_materials(cls) -> dict[str, MaterialInstance]:
312     """Returns a dictionary the holds all the material instances. As the key is the name of the
313     material, as the value is the material instance.
314
315     Returns:
316         dict[str, MaterialInstance]: A dictionary the holds all the material instances.
317     """
318     return cls.instance.materials

```

Listing C.3: OpenGLMaterialLib Class

```

1 from pyGandalf.renderer.webgpu_renderer import WebGPURenderer
2
3 import wgpu
4 from PIL import Image
5
6 from pathlib import Path
7 from dataclasses import dataclass
8
9 @dataclass
10 class TextureDescriptor:
11     flip: bool = False
12     dimension: wgpu.TextureDimension = wgpu.TextureDimension.d2
13     view_aspect: wgpu.TextureAspect = wgpu.TextureAspect.all
14     view_format: wgpu.TextureFormat = None
15     view_dimension: wgpu.TextureViewDimension = wgpu.TextureViewDimension.d2
16     usage: wgpu.TextureUsage = wgpu.TextureUsage.COPY_DST | wgpu.TextureUsage.TEXTURE_BINDING | wgpu.
17         TextureUsage.RENDER_ATTACHMENT
18     format: wgpu.TextureFormat = wgpu.TextureFormat.rgba8unorm
19     address_mode_u: wgpu.AddressMode = wgpu.AddressMode.clamp_to_edge
20     address_mode_v: wgpu.AddressMode = wgpu.AddressMode.clamp_to_edge
21     address_mode_w: wgpu.AddressMode = wgpu.AddressMode.clamp_to_edge
22     min_filter: wgpu.FilterMode = wgpu.FilterMode.linear
23     mag_filter: wgpu.FilterMode = wgpu.FilterMode.linear
24     sampler_compare: wgpu.CompareFunction = None
25     array_layer_count: int = 1
26
27 @dataclass
28 class TextureData:
29     path: Path | list[Path] = None
30     width: int = 0
31     height: int = 0
32     image_bytes: bytes = None
33
34 @dataclass
35 class TextureInstance:
36     texture: wgpu.GPUTexture = None
37     view: wgpu.GPUTextureView = None
38     sampler: wgpu.GPUSampler = None
39     data: TextureData = None
40     descriptor: TextureDescriptor = None
41
42 class WebGPUTextureLib(object):
43     def __new__(cls):
44         if not hasattr(cls, 'instance'):
45             cls.instance = super(WebGPUTextureLib, cls).__new__(cls)
46             cls.instance.textures = {}
47             cls.instance.slots = {}
48             cls.instance.current_slot = 0
49         return cls.instance

```

```

50 def build(cls, name: str, data: TextureData, descriptor: TextureDescriptor = TextureDescriptor())
51 :
52     """Builds a new texture (if one does not already exists with that name) and returns its slot.
53
54     Args:
55         name (str): The name of the texture.
56         data (TextureData): The data of the texture. You can either give a path (or list of paths
57         if cubemap) or the byte data to use when creating the texture and the width and height.
58         descriptor (TextureDescriptor, optional): The description of the texture, which consists
59         of various options and flags.
60
61     Returns:
62         int: The texture slot.
63     """
64     if cls.instance.textures.get(name) != None:
65         return cls.instance.textures.get(name)
66
67     if data.path is not None:
68         if type(data.path) is not list:
69             img = Image.open(data.path)
70             if descriptor.flip:
71                 img = img.transpose(Image.FLIP_TOP_BOTTOM)
72             data.image_bytes = img.convert("RGBA").tobytes("raw", "RGBA", 0, -1)
73             data.width = img.width
74             data.height = img.height
75         else:
76             cubemap_data = []
77             for p in data.path:
78                 img = Image.open(p)
79                 if descriptor.flip:
80                     img = img.transpose(Image.FLIP_TOP_BOTTOM)
81                 cubemap_data.append(img.convert("RGBA").tobytes("raw", "RGBA", 0, -1))
82             data.width = img.width
83             data.height = img.height
84
85             data.image_bytes = bytes()
86             for cubemap in cubemap_data:
87                 data.image_bytes += cubemap
88
89     size = [data.width, data.height, descriptor.array_layer_count]
90
91     texture: wgpu.GPUTexture = WebGPURenderer().get_device().create_texture(
92         size = size,
93         usage = descriptor.usage,
94         dimension = descriptor.dimension,
95         format = descriptor.format,
96         mip_level_count = 1,
97         sample_count = 1,
98     )
99
100     view = texture.create_view(
101         dimension=descriptor.view_dimension,
102         aspect=descriptor.view_aspect,
103         format=descriptor.view_format,
104         array_layer_count=descriptor.array_layer_count,
105     )
106
107     sampler = WebGPURenderer().get_device().create_sampler(
108         address_mode_u=descriptor.address_mode_u,
109         address_mode_v=descriptor.address_mode_v,
110         address_mode_w=descriptor.address_mode_w,
111         min_filter=descriptor.min_filter,
112         mag_filter=descriptor.mag_filter,
113         compare=descriptor.sampler_compare,
114         mipmap_filter=wgpu.FilterMode.nearest,
115         max_anisotropy=1,
116     )
117
118     if data.image_bytes != None:
119         WebGPURenderer().get_device().queue.write_texture(
120             {
121                 "texture": texture,
122                 "mip_level": 0,
123                 "origin": (0, 0, 0)
124             },
125             data.image_bytes,
126             {
127                 "offset": 0,
128                 "bytes_per_row": data.width * 4,
129                 "rows_per_image": data.height,
130             },
131             size
132         )

```

```

131     cls.instance.textures[name] = TextureInstance(texture, view, sampler, data, descriptor)
132     cls.instance.slots[name] = cls.instance.current_slot
133
134     cls.instance.current_slot += 1
135
136     return cls.instance.slots[name]
137
138     def get_instance(cls, name: str) -> TextureInstance:
139         """Returns the instance of the texture with the given name.
140
141         Args:
142             name (str): The name of the texture.
143
144         Returns:
145             TextureInstance: The instance of the texture with the given name.
146         """
147         return cls.instance.textures.get(name)
148
149     def get_slot(cls, name: str):
150         """Returns the slot of texture with the given name.
151
152         Args:
153             name (str): The name of the texture.
154
155         Returns:
156             float: The texture slot.
157         """
158         return float(cls.instance.slots.get(name))

```

Listing C.4: WebGPUTextureLib Class

```

1 from pyGandalf.renderer.webgpu_renderer import WebGPURenderer
2 from pyGandalf.utilities.definitions import SHADERS_PATH
3
4 import wgpu
5
6 import os
7 import re
8 from pathlib import Path
9
10 class ShaderData:
11     def __init__(self, name: str, shader_module, pipeline_layout, bind_group_layouts, shader_path:
12         Path, shader_code: str):
13         self.name = name
14         self.shader_module = shader_module
15         self.pipeline_layout = pipeline_layout
16         self.bind_group_layouts: list = bind_group_layouts
17         self.shader_path = shader_path
18         self.shader_code = shader_code
19
20 class WebGPUShaderLib(object):
21     def __new__(cls):
22         if not hasattr(cls, 'instance'):
23             cls.instance = super(WebGPUShaderLib, cls).__new__(cls)
24             cls.instance.shaders : dict[str, ShaderData] = {} # type: ignore
25             return cls.instance
26
27     def create_shader_module(cls, shader_source):
28         shader_module: wgpu.GPUShaderModule = WebGPURenderer().get_device().create_shader_module(code
29             =shader_source)
30
31         uniform_buffers, storage_buffers, read_only_storage_buffers, other = cls.instance.parse(
32             shader_source)
33
34         # Create the wgpu binding objects
35         bind_groups_layout_entries = [[]]
36         for buffer_name in uniform_buffers.keys():
37             if len(bind_groups_layout_entries) <= uniform_buffers[buffer_name]['group']:
38                 bind_groups_layout_entries.append([])
39
40         bind_groups_layout_entries[uniform_buffers[buffer_name]['group']].append({
41             "binding": uniform_buffers[buffer_name]['binding'],
42             "visibility": wgpu.ShaderStage.VERTEX | wgpu.ShaderStage.FRAGMENT | wgpu.ShaderStage.
43             COMPUTE, # TODO: Fix this visibility properly
44             "buffer": {
45                 "type": wgpu.BufferBindingType.uniform
46             },
47         })
48
49         for buffer_name in storage_buffers.keys():
50             if len(bind_groups_layout_entries) <= storage_buffers[buffer_name]['group']:
51                 bind_groups_layout_entries.append([])

```

```

49         bind_groups_layout_entries[storage_buffers[buffer_name]['group']].append({
50             "binding": storage_buffers[buffer_name]['binding'],
51             "visibility": wgpu.ShaderStage.VERTEX | wgpu.ShaderStage.COMPUTE, # TODO: Fix this
visibility properly
52         "buffer": {
53             "type": wgpu.BufferBindingType.storage
54         },
55     })
56
57     for buffer_name in read_only_storage_buffers.keys():
58         if len(bind_groups_layout_entries) <= read_only_storage_buffers[buffer_name]['group']:
59             bind_groups_layout_entries.append({})
60
61     bind_groups_layout_entries[read_only_storage_buffers[buffer_name]['group']].append({
62         "binding": read_only_storage_buffers[buffer_name]['binding'],
63         "visibility": wgpu.ShaderStage.VERTEX | wgpu.ShaderStage.COMPUTE, # TODO: Fix this
visibility properly
64         "buffer": {
65             "type": wgpu.BufferBindingType.read_only_storage
66         },
67     })
68
69     for uniform_name in other.keys():
70         if len(bind_groups_layout_entries) <= other[uniform_name]['group']:
71             bind_groups_layout_entries.append({})
72
73     match other[uniform_name]['type']['name']:
74         case 'texture_2d<f32>':
75             bind_groups_layout_entries[other[uniform_name]['group']].append({
76                 "binding": other[uniform_name]['binding'],
77                 "visibility": wgpu.ShaderStage.FRAGMENT | wgpu.ShaderStage.COMPUTE,
78                 "texture": {
79                     "sample_type": wgpu.TextureSampleType.float,
80                     "view_dimension": wgpu.TextureViewDimension.d2,
81                 }
82             })
83         case 'texture_depth_2d':
84             bind_groups_layout_entries[other[uniform_name]['group']].append({
85                 "binding": other[uniform_name]['binding'],
86                 "visibility": wgpu.ShaderStage.FRAGMENT,
87                 "texture": {
88                     "sample_type": wgpu.TextureSampleType.depth,
89                     "view_dimension": wgpu.TextureViewDimension.d2,
90                 }
91             })
92         case 'texture_cube<f32>':
93             bind_groups_layout_entries[other[uniform_name]['group']].append({
94                 "binding": other[uniform_name]['binding'],
95                 "visibility": wgpu.ShaderStage.FRAGMENT,
96                 "texture": {
97                     "sample_type": wgpu.TextureSampleType.float,
98                     "view_dimension": wgpu.TextureViewDimension.cube,
99                 }
100             })
101         case 'texture_storage_2d<rgba8unorm, write>':
102             bind_groups_layout_entries[other[uniform_name]['group']].append({
103                 "binding": other[uniform_name]['binding'],
104                 "visibility": wgpu.ShaderStage.COMPUTE,
105                 "storage_texture": {
106                     "format": wgpu.TextureFormat.rgba8unorm,
107                     "access": wgpu.StorageTextureAccess.write_only,
108                     "view_dimension": wgpu.TextureViewDimension.d2,
109                 }
110             })
111         case 'sampler':
112             bind_groups_layout_entries[other[uniform_name]['group']].append({
113                 "binding": other[uniform_name]['binding'],
114                 "visibility": wgpu.ShaderStage.FRAGMENT,
115                 "sampler": {
116                     "type": wgpu.SamplerBindingType.filtering
117                 },
118             })
119         case 'sampler_comparison':
120             bind_groups_layout_entries[other[uniform_name]['group']].append({
121                 "binding": other[uniform_name]['binding'],
122                 "visibility": wgpu.ShaderStage.FRAGMENT,
123                 "sampler": {
124                     "type": wgpu.SamplerBindingType.comparison
125                 },
126             })
127
128     # Create the wgou binding objects
129     bind_group_layouts = []
130

```

```

131         for layout_entries in bind_groups_layout_entries:
132             bind_group_layouts.append(WebGPURenderer().get_device().create_bind_group_layout(entries=
layout_entries))
133
134         # No bind group and layout, we should not create empty ones.
135         pipeline_layout : wgpu.GPUPipelineLayout = WebGPURenderer().get_device().
create_pipeline_layout(bind_group_layouts=bind_group_layouts)
136
137         return shader_module, pipeline_layout, bind_group_layouts
138
139 def build(cls, name: str, shader_path: Path) -> int:
140     """Builds a new shader (if one does not already exists with that name) and returns its shader
program.
141
142     Args:
143         name (str): The name of the shader
144         vs_path (Path): The path to the vertex shader code
145         fs_path (Path): The path to the fragment shader code
146
147     Returns:
148         int: The shader module
149     """
150     if cls.instance.shaders.get(name) != None:
151         return cls.instance.shaders[name].shader_module
152
153     shader_source = cls.instance.load_from_file(shader_path)
154
155     shader_rel_path = Path(os.path.relpath(shader_path, SHADERS_PATH))
156
157     shader_module, pipeline_layout, bind_group_layouts = cls.instance.create_shader_module(
shader_source)
158     cls.instance.shaders[name] = ShaderData(name, shader_module, pipeline_layout,
bind_group_layouts, shader_rel_path, shader_source)
159
160     return shader_module
161
162 def parse(cls, shader_code: str) -> dict:
163     """Parses the provided shader code and identifies all the uniforms along with their types.
164
165     Args:
166         shader_code (str): The source code of the shader to parse.
167
168     Returns:
169         dict: A dictionary holding the uniform name as a key and the uniform type as a value
170     """
171     def parse_buffer(buffer_type: str):
172         pattern = re.compile(r"@group\((\d+)\) @binding\((\d+)\) var" + re.escape(buffer_type) +
r" (\w+):\s*(\w+(?:\<.*?>?));")
173         matches = pattern.findall(shader_code)
174
175         buffers = {}
176         buffer_members = {}
177         for match in matches:
178             group, binding, name, type = match
179             print(f"Buffer: {name}")
180             print(f"Group: {group}")
181             print(f"Binding: {binding}")
182             print(f"Type: {type}")
183             struct_pattern = r"struct " + re.escape(type) + r"\s*\{([^\}]*)\}"
184             struct_match = re.search(struct_pattern, shader_code)
185             if struct_match:
186                 buffer_members.clear()
187                 struct_body = struct_match.group(1)
188                 member_pattern = r"(\w+)\s*:\s*(\<?[\w\s,<>+>?),"
189                 members = re.findall(member_pattern, struct_body)
190                 print(f"Members:")
191                 for member in members:
192                     print(f" {member[0]}: {member[1]}")
193                     buffer_members[member[0]] = member[1]
194
195         buffers[name] = {
196             'group': int(group),
197             'binding': int(binding),
198             'type': {
199                 'name': type,
200                 'members': buffer_members
201             },
202         }
203
204     return buffers
205
206     uniform_buffers = parse_buffer('<uniform>')
207     storage_buffers = parse_buffer('<storage>')
208     read_write_storage_buffers = parse_buffer('<storage, read_write>')

```

```

209     read_only_storage_buffers = parse_buffer('<storage, read>')
210     other = parse_buffer('')
211
212     return uniform_buffers, storage_buffers | read_write_storage_buffers,
213         read_only_storage_buffers, other
214
215 def get(cls, name: str) -> ShaderData:
216     """Gets the shader data of the given shader name.
217
218     Args:
219         name (str): The name of the shader to get its data.
220
221     Returns:
222         ShaderData: the shader data of the given shader name.
223     """
224     return cls.instance.shaders.get(name)
225
226 def get_shaders(cls) -> dict[str, ShaderData]:
227     """Returns the dictionary that holds all the shader data
228
229     Returns:
230         dict[str, ShaderData]: the dictionary that holds all the shader data
231     """
232     return cls.instance.shaders
233
234 def load_from_file(cls, path_to_source):
235     """
236     Returns the file contents as a string.
237     """
238     with open(path_to_source) as file:
239         return file.read()

```

Listing C.5: WebGPUShaderLib Class

```

1 from pyGandalf.renderer.webgpu_renderer import WebGPURenderer
2 from pyGandalf.utilities.webgpu_shader_lib import WebGPUShaderLib
3 from pyGandalf.utilities.webgpu_texture_lib import WebGPUTextureLib, TextureInstance
4 from pyGandalf.utilities.logger import logger
5
6 import glm
7 import wgpu
8 import numpy as np
9
10 import re
11 from dataclasses import dataclass
12
13 class CPUBuffer:
14     def __init__(self, *args):
15         fields = []
16         current_offset = 0
17
18         # Use this to double check the padding: https://eliemichel.github.io/WebGPU-AutoLayout/
19     def add_padding():
20         nonlocal current_offset
21         if current_offset % 16 != 0:
22             padding = 16 - (current_offset % 16)
23             pad_field_name = f'__pad{len(fields)}'
24             fields.append((pad_field_name, np.float32, ((padding // 4),)))
25             current_offset += padding
26
27     for name, dtype, shape in args:
28         # Ensure shape is a tuple even for single elements
29         if not isinstance(shape, tuple):
30             shape = (shape,)
31
32         # Calculate the size in bytes of the current field
33         field_size = np.dtype(dtype).itemsize * np.prod(shape)
34
35         # Append field and increase the offset
36         fields.append((name, dtype, shape))
37         current_offset += field_size
38
39         # Align current field to 16 bytes
40         add_padding()
41
42     self.type = np.dtype(fields)
43     self.data = np.empty((1,), self.type)
44
45     @property
46     def nbytes(self):
47         return self.data.nbytes
48
49     @property

```

```

50     def mem(self):
51         return self.data
52
53     @property
54     def dtype(self):
55         return self.type
56
57     def has_member(self, member):
58         return member in self.data.dtype.names
59
60     def __getitem__(self, key):
61         return self.data[key]
62
63     def __setitem__(self, key, val):
64         self.data[key] = val
65
66 @dataclass
67 class MaterialDescriptor:
68     primitive: wgpu.PrimitiveTopology = wgpu.PrimitiveTopology.triangle_list
69     front_face: wgpu.FrontFace = wgpu.FrontFace.ccw
70     cull_mode: wgpu.CullMode = wgpu.CullMode.front
71
72     depth_enabled: bool = True
73     depth_write_enabled: bool = True
74     depth_format: wgpu.TextureFormat = wgpu.TextureFormat.depth24plus
75     depth_compare: wgpu.CompareFunction = wgpu.CompareFunction.less_equal
76
77     cast_shadows: bool = True
78
79 class MaterialData:
80     def __init__(self, base_template: str, textures: list[str], color: glm.vec4 = glm.vec4(1.0, 1.0,
81         1.0, 1.0), glossiness = 3.0):
82         self.base_template = base_template
83         self.color = color
84         self.textures = textures
85         self.glossiness = glossiness
86
87     def __eq__(self, other):
88         if self.base_template != other.base_template:
89             return False
90         if self.color != other.color:
91             return False
92         if self.glossiness != other.glossiness:
93             return False
94         if len(self.textures) != len(other.textures):
95             return False
96         for i in range(len(self.textures)):
97             if (self.textures[i] != other.textures[i]):
98                 return False
99         return True
100
101     def __hash__(self):
102         return hash((self.base_template, self.color.r, self.color.g, self.color.b, self.color.a, len(
103             self.textures), self.glossiness, tuple(texture for texture in self.textures)))
104
105 class MaterialInstance:
106     def __init__(self, name, data: MaterialData, descriptor: MaterialDescriptor, shader_module,
107         pipeline_layout, bind_groups, uniform_buffers, uniform_buffer_types, storage_buffers,
108         storage_buffer_types, other_uniforms, shader_params = []):
109         self.name = name
110         self.data: MaterialData = data
111         self.descriptor: MaterialDescriptor = descriptor
112         self.shader_module = shader_module
113         self.pipeline_layout = pipeline_layout
114         self.bind_groups = bind_groups
115         self.uniform_buffers = uniform_buffers
116         self.uniform_buffer_types = uniform_buffer_types
117         self.storage_buffers = storage_buffers
118         self.storage_buffer_types = storage_buffer_types
119         self.other_uniforms = other_uniforms
120         self.shader_params = shader_params
121
122     def has_uniform(self, uniform_name: str) -> bool:
123         """Returns True if the material has the uniform with the given name, otherwise False.
124
125         Args:
126             uniform_name (str): The name of the uniform.
127
128         Returns:
129             bool: True if the material has the uniform with the given name, otherwise False.
130         """
131         return uniform_name in self.uniform_buffers.keys()
132
133     def set_uniform(self, uniform_name: str):

```

```

130     """Sets the uniform buffer with the provided name (if valid), with the provided data.
131
132     Args:
133         uniform_name (str): The name of the uniform buffer to set.
134     """
135     uniform = self.other_uniforms[uniform_name]
136
137     if isinstance(uniform, TextureInstance):
138         WebGPURenderer().write_texture(uniform)
139
140 def set_uniform_buffer(self, uniform_name: str, uniform_data: CPUBuffer):
141     """Sets the uniform buffer with the provided name (if valid), with the provided data.
142
143     Args:
144         uniform_name (str): The name of the uniform buffer to set.
145         uniform_data (CPUBuffer): The new data for the uniform buffer.
146     """
147     uniform_buffer = self.uniform_buffers[uniform_name]
148     WebGPURenderer().write_buffer(uniform_buffer, uniform_data.mem)
149
150 def set_storage_buffer(self, storage_name: str, storage_data: CPUBuffer):
151     """Sets the storage buffer with the provided name (if valid), with the provided data.
152
153     Args:
154         uniform_name (str): The name of the storage buffer to set.
155         uniform_data (Any): The new data for the storage buffer.
156     """
157     storage_buffer = self.storage_buffers[storage_name]
158
159     temporary_buffer: wgpu.GPUBuffer = WebGPURenderer().get_device().create_buffer_with_data(
160         data=storage_data.mem, usage=wgpu.BufferUsage.COPY_SRC
161     )
162
163     WebGPURenderer().get_command_encoder().copy_buffer_to_buffer(
164         temporary_buffer, 0, storage_buffer, 0, storage_data.mem.nbytes
165     )
166
167     temporary_buffer.destroy()
168
169 def get_cpu_buffer_type(self, buffer_name: str) -> CPUBuffer | None:
170     if buffer_name in self.uniform_buffer_types.keys():
171         return self.uniform_buffer_types[buffer_name]
172     if buffer_name in self.storage_buffer_types.keys():
173         return self.storage_buffer_types[buffer_name]
174     return None
175
176 def uniform_not_found(self, uniform_name: str):
177     """Prints a message stating that the uniform with the provided name was not found.
178
179     Args:
180         uniform_name (str): The name of the uniform that was not found.
181     """
182     logger.debug(f'Could not find {uniform_name} uniform for material: {self.name}!\n These are
183         the available uniforms for shader with id {self.shader_program}:\n {self.shader_params}')
184
185 def print_available_uniforms(self):
186     """Prints a message with all the available uniforms of the material.
187
188     """
189     logger.log(f'Available uniforms for material: {self.name} and shader id {self.shader_program
190         }:\n {self.shader_params}')
191
192 class WebGPUMaterialLib(object):
193     def __new__(cls):
194         if not hasattr(cls, 'instance'):
195             cls.instance = super(WebGPUMaterialLib, cls).__new__(cls)
196             cls.instance.cached_materials: dict[MaterialData, MaterialInstance] = {} # type: ignore
197             cls.instance.materials: dict[str, MaterialInstance] = {} # type: ignore
198         return cls.instance
199
200 def build(cls, name: str, data: MaterialData, descriptor: MaterialDescriptor=MaterialDescriptor())
201     -> MaterialInstance:
202     """Builds a new material (if one does not already exists with that data) and returns its
203     instance.
204
205     Args:
206         name (str): The name of the material.
207         data (MaterialData): The data of the material, which consists of shader, color, textures
208         and glossiness.
209         descriptor (MaterialDescriptor, optional): The description of the material, which
210         consists of various options and flags.
211
212     Returns:
213         MaterialInstance: The material instance.
214     """

```

```

208         if cls.instance.cached_materials.get(data) != None:
209             material = cls.instance.cached_materials[data]
210             cls.instance.materials[name] = material
211             return material
212
213         shader_data = WebGPUShaderLib().get(data.base_template)
214
215         if shader_data == None:
216             logger.error(f"No such material exists: '{data.base_template}'")
217             return None
218
219         # Parse shader params
220         uniform_buffers_data, storage_buffers_data, read_only_storage_buffers_data, other =
WebGPUShaderLib().parse(shader_data.shader_code)
221
222         uniform_buffers = {}
223         uniform_buffer_types = {}
224         storage_buffers = {}
225         storage_buffer_types = {}
226         other_uniforms = {}
227
228         bind_groups_entries = [[]]
229         for buffer_name in uniform_buffers_data.keys():
230             uniform_buffer_data = uniform_buffers_data[buffer_name]
231
232             if len(bind_groups_entries) <= uniform_buffer_data['group']:
233                 bind_groups_entries.append([])
234
235             # Find uniform buffer layout and fields from shader reflection.
236             fields = []
237             for member_name in uniform_buffer_data['type']['members']:
238                 member_type = uniform_buffer_data['type']['members'][member_name]
239                 fields.append(cls.instance.compute_field_layout(member_type, member_name))
240
241             # Instantiate a struct for the uniform buffer data with the given fields
242             uniform_data = CPUBuffer(*fields)
243             uniform_buffer_types[buffer_name] = uniform_data
244
245             # Create a uniform buffer
246             uniform_buffer: wgpu.GPUBuffer = WebGPURenderer().get_device().create_buffer(
247                 size=uniform_data.nbytes, usage=wgpu.BufferUsage.UNIFORM | wgpu.BufferUsage.COPY_DST
248             )
249
250             # Append uniform buffer to dictionary holding all uniform buffers
251             uniform_buffers[buffer_name] = uniform_buffer
252
253             # Create a bind group entry for the uniform buffer
254             bind_groups_entries[uniform_buffers_data[buffer_name]['group']].append({
255                 "binding": uniform_buffers_data[buffer_name]['binding'],
256                 "resource": {
257                     "buffer": uniform_buffer,
258                     "offset": 0,
259                     "size": uniform_buffer.size,
260                 },
261             })
262
263         for buffer_name in storage_buffers_data.keys():
264             storage_buffer_data = storage_buffers_data[buffer_name]
265
266             if len(bind_groups_entries) <= storage_buffer_data['group']:
267                 bind_groups_entries.append([])
268
269             # Find storage buffer layout and fields from shader reflection.
270             fields = []
271             for member_name in storage_buffer_data['type']['members']:
272                 member_type = storage_buffer_data['type']['members'][member_name]
273                 fields.append(cls.instance.compute_field_layout(member_type, member_name))
274
275             # Instantiate a struct for the uniform buffer data with the given fields
276             storage_data = CPUBuffer(*fields)
277
278             storage_buffer_types[buffer_name] = storage_data
279
280             # Create storage buffer - data is uploaded each frame
281             storage_buffer: wgpu.GPUBuffer = WebGPURenderer().get_device().create_buffer(
282                 size=storage_data.nbytes, usage=wgpu.BufferUsage.STORAGE | wgpu.BufferUsage.COPY_DST
283             )
284
285             # Append storage buffer to dictionary holding all storage buffers
286             storage_buffers[buffer_name] = storage_buffer
287
288             bind_groups_entries[storage_buffer_data['group']].append({
289                 "binding": storage_buffer_data['binding'],
290                 "resource": {

```

```

291         "buffer": storage_buffer,
292         "offset": 0,
293         "size": storage_buffer.size,
294     },
295 )
296
297 for buffer_name in read_only_storage_buffers_data.keys():
298     read_only_storage_buffer_data = read_only_storage_buffers_data[buffer_name]
299
300     if len(bind_groups_entries) <= read_only_storage_buffer_data['group']:
301         bind_groups_entries.append([])
302
303     # Find storage buffer layout and fields from shader reflection.
304     fields = []
305     for member_name in read_only_storage_buffer_data['type']['members']:
306         member_type = read_only_storage_buffer_data['type']['members'][member_name]
307         fields.append(cls.instance.compute_field_layout(member_type, member_name))
308
309     # Instantiate a struct for the uniform buffer data with the given fields
310     storage_data = CPUBuffer(*fields)
311
312     storage_buffer_types[buffer_name] = storage_data
313
314     # Create storage buffer - data is uploaded each frame
315     storage_buffer: wgpu.GPUBuffer = WebGPURenderer().get_device().create_buffer(
316         size=storage_data.nbytes, usage=wgpu.BufferUsage.STORAGE | wgpu.BufferUsage.COPY_DST
317     )
318
319     # Append storage buffer to dictionary holding all storage buffers
320     storage_buffers[buffer_name] = storage_buffer
321     bind_groups_entries[read_only_storage_buffer_data['group']].append({
322         "binding": read_only_storage_buffer_data['binding'],
323         "resource": {
324             "buffer": storage_buffer,
325             "offset": 0,
326             "size": storage_buffer.size,
327         },
328     })
329
330 texture_index = 0
331 texture_index_use_count = 0
332
333 for uniform_name in other.keys():
334     other_data = other[uniform_name]
335
336     if len(bind_groups_entries) <= other_data['group']:
337         bind_groups_entries.append([])
338
339     # Retrieve texture.
340     texture_inst: TextureInstance = WebGPUTextureLib().get_instance(data.textures[
341 texture_index])
342
343     if texture_inst == None:
344         logger.error(f"No such texture exists: '{data.textures[texture_index]}'")
345         continue
346
347     match other_data['type']['name']:
348         case 'texture_2d<f32>':
349             # Append uniform buffer to dictionary holding all uniform buffers
350             other_uniforms[uniform_name] = texture_inst
351
352             # Create a bind group entry for the uniform buffer
353             bind_groups_entries[other_data['group']].append({
354                 "binding": other_data['binding'],
355                 "resource": texture_inst.view
356             })
357             texture_index_use_count += 1
358         case 'texture_depth_2d':
359             # Append uniform buffer to dictionary holding all uniform buffers
360             other_uniforms[uniform_name] = texture_inst
361
362             # Create a bind group entry for the uniform buffer
363             bind_groups_entries[other_data['group']].append({
364                 "binding": other_data['binding'],
365                 "resource": texture_inst.view
366             })
367             texture_index_use_count += 1
368         case 'texture_cube<f32>':
369             # Append uniform buffer to dictionary holding all uniform buffers
370             other_uniforms[uniform_name] = texture_inst
371
372             # Create a bind group entry for the uniform buffer
373             bind_groups_entries[other_data['group']].append({
374                 "binding": other_data['binding'],

```

```

374         "resource": texture_inst.view
375     })
376     texture_index_use_count += 1
377
378     case 'sampler':
379         # Append uniform buffer to dictionary holding all uniform buffers
380         other_uniforms[uniform_name] = texture_inst
381
382         # Create a bind group entry for the uniform buffer
383         bind_groups_entries[other_data['group']].append({
384             "binding": other_data['binding'],
385             "resource": texture_inst.sampler,
386         })
387         texture_index_use_count += 1
388     case 'sampler_comparison':
389         # Append uniform buffer to dictionary holding all uniform buffers
390         other_uniforms[uniform_name] = texture_inst
391
392         # Create a bind group entry for the uniform buffer
393         bind_groups_entries[other_data['group']].append({
394             "binding": other_data['binding'],
395             "resource": texture_inst.sampler,
396         })
397         texture_index_use_count += 1
398
399     if texture_index_use_count == 2:
400         texture_index_use_count = 0
401         texture_index += 1
402
403     bind_groups: list[wgpu.GPUBindGroup] = []
404     for index, bind_group_entry in enumerate(bind_groups_entries):
405         bind_groups.append(WebGPURenderer().get_device().create_bind_group(
406             layout=shader_data.bind_group_layouts[index],
407             entries=bind_group_entry
408         ))
409
410     cls.instance.cached_materials[data] = MaterialInstance(name, data, descriptor, shader_data.shader_module,
411                                                             shader_data.pipeline_layout, bind_groups, uniform_buffers, uniform_buffer_types,
412                                                             storage_buffers, storage_buffer_types, other_uniforms, [])
413     cls.instance.materials[name] = MaterialInstance(name, data, descriptor, shader_data.shader_module,
414                                                     shader_data.pipeline_layout, bind_groups, uniform_buffers, uniform_buffer_types,
415                                                     storage_buffers, storage_buffer_types, other_uniforms, [])
416
417     return cls.instance.materials[name]
418
419 def get(cls, name: str) -> MaterialInstance:
420     """Returns the material instance with the given name.
421
422     Args:
423         name (str): The name of the material instance to get.
424
425     Returns:
426         MaterialInstance: The material instance with the given name.
427     """
428     return cls.instance.materials.get(name)
429
430 def get_materials(cls) -> dict[str, MaterialInstance]:
431     """Returns a dictionary the holds all the material instances. As the key is the name of the
432     material, as the value is the material instance.
433
434     Returns:
435         dict[str, MaterialInstance]: A dictionary the holds all the material instances.
436     """
437     return cls.instance.materials
438
439 def extract_array_size(cls, declaration: str) -> tuple[str, int]:
440     # Define the regular expression pattern to match the array declaration
441     pattern = r'array<([^\s,]+), (\d+)>'
442
443     # Use re.search to find the match
444     match = re.search(pattern, declaration)
445
446     if match:
447         # Extract the number from the match group and convert it to an integer
448         array_type = match.group(1).strip()
449         array_size = int(match.group(2))
450         return (array_type, array_size)
451     else:
452         raise ValueError(f"Invalid declaration format: {declaration}")
453
454 def compute_field_layout(cls, member_type, member_name):
455     match member_type:
456         case 'f32':
457             return (member_name, np.float32, (1,))

```

```

453         case 'vec2f':
454             return (member_name, np.float32, (2,))
455         case 'vec2<f32>':
456             return (member_name, np.float32, (2,))
457         case 'vec3f':
458             return (member_name, np.float32, (3,))
459         case 'vec3<f32>':
460             return (member_name, np.float32, (3,))
461         case 'vec4f':
462             return (member_name, np.float32, (4,))
463         case 'vec4<f32>':
464             return (member_name, np.float32, (4,))
465         case 'mat4x4f':
466             return (member_name, np.float32, (4, 4))
467         case 'mat4x4<f32>':
468             return (member_name, np.float32, (4, 4))
469
470     if 'array' in member_type:
471         if ',' not in member_type:
472             logger.error(f'Array type with unspecified array size is not supported right now,
please specify the array size of array: {member_name}')
473             exit(-1)
474
475     array_type, array_size = cls.instance.extract_array_size(member_type)
476
477     match array_type:
478         case 'mat4x4f':
479             return (member_name, np.float32, (array_size, 4, 4))
480         case 'mat4x4<f32>':
481             return (member_name, np.float32, (array_size, 4, 4))
482         case 'vec4f':
483             return (member_name, np.float32, (array_size, 1, 4))
484         case 'vec4<f32>':
485             return (member_name, np.float32, (array_size, 1, 4))
486         case 'vec3f':
487             return (member_name, np.float32, (array_size, 1, 3))
488         case 'vec3<f32>':
489             return (member_name, np.float32, (array_size, 1, 3))
490         case 'vec2f':
491             return (member_name, np.float32, (array_size, 1, 2))
492         case 'vec2<f32>':
493             return (member_name, np.float32, (array_size, 1, 2))
494         case 'f32':
495             return (member_name, np.float32, (array_size, 1, 1))

```

Listing C.6: WebGPUMaterialLib Class

```

1  from pyGandalf.utilities.logger import logger
2  from pyGandalf.utilities.definitions import MODELS_PATH
3
4  import numpy as np
5  import trimesh
6
7  import os
8  from pathlib import Path
9
10 class MeshInstance:
11     def __init__(self, name, path, vertices, indices, normals, texcoords):
12         self.name = name
13         self.path = path
14         self.vertices = vertices
15         self.indices = indices
16         self.normals = normals
17         self.texcoords = texcoords
18
19 class MeshLib(object):
20     def __new__(cls):
21         if not hasattr(cls, 'instance'):
22             cls.instance = super(MeshLib, cls).__new__(cls)
23             cls.instance.meshes: dict[str, MeshInstance] = {} # type: ignore
24             cls.instance.meshes_names: dict[str, str] = {} # type: ignore
25             return cls.instance
26
27     def build(cls, name: str, path: Path):
28         filename = str(path)
29         if cls.instance.meshes.get(filename) != None:
30             cls.instance.meshes_names[name] = filename
31             return cls.instance.meshes[filename]
32
33     mesh = None
34     vertices = None
35     indices = None
36     normals = None

```

```

37     texcoords = None
38
39     mesh: trimesh.Trimesh = trimesh.load(filename, force='mesh')
40     vertices = np.asarray(mesh.vertices, dtype=np.float32)
41     indices = np.asarray(mesh.faces, dtype=np.uint32)
42     normals = np.asarray(mesh.vertex_normals, dtype=np.float32)
43
44     if hasattr(mesh.visual, 'uv'):
45         texcoords = np.asarray(mesh.visual.uv, dtype=np.float32)
46
47     rel_path = Path(os.path.relpath(path, MODELS_PATH))
48
49     cls.instance.meshes_names[name] = filename
50     cls.instance.meshes[filename] = MeshInstance(name, rel_path, vertices, indices, normals,
51     texcoords)
52
53     return cls.instance.meshes[filename]
54
55 def get(cls, name: str) -> MeshInstance | None:
56     if name not in cls.instance.meshes_names.keys():
57         return None
58
59     filename = cls.instance.meshes_names[name]
60
61     if filename not in cls.instance.meshes.keys():
62         return None
63
64     return cls.instance.meshes[filename]
65
66 def get_meshes(cls) -> dict[str, MeshInstance]:
67     return cls.instance.meshes

```

Listing C.7: MeshLib Class